计算机系统与程序执行的层次 1

章节脉络



计算机系统将硬件、软件和运行规则结合起来，使程序能够按确定的步骤处理数据。本章从功能部件出发，沿存储程序与指令执行理解各层接口，再用 CPU 时间、吞吐量和容量计算分析系统性能。

课程连接·功能部件接到数据通路

把 c=a+b 分成取 a、取 b、相加、存 c 后，第 5 章会逐步标出寄存器传送和控制信号。这里的功能划分仍成立；一块芯片可以同时实现多种功能，不能按芯片数倒推五部件。

1.1 从程序看计算机的功能部件

定义 1.1.1 (概念与适用对象)

计算机系统由硬件和软件共同构成。硬件提供保存信息、执行运算、控制步骤和输入输出的物理装置，软件用程序和数据规定装置怎样完成任务。仅凭处理器芯片无法判断一个应用能否运行；还要看指令集、操作系统、库及文件格式等条件。

计算机的五个基本功能部件是运算器、控制器、存储器、输入设备和输出设备。这个分类按**功能**划分，不是按芯片这一物理实体划分。CPU 通常包含运算器、控制器和寄存器，还可能集成若干级 Cache；一个存储控制器也可能集成在处理器芯片内。

部件	在执行 c=a+b 时做什么
运算器	对取来的两个数进行加法，形成结果及有关标志
控制器	解释指令，在合适时刻发出选择、读写和运算等控制信号
存储器	保存指令、a、b、c 及执行所需的其他信息
输入、输出设备	将系统外部的信息送入系统，或把系统内的结果送出

中央处理器 CPU 的数据通路包括寄存器、运算部件及其连接，控制器决定数据在通路上何时、向哪里流动。第 5 章将把一次加法展开为寄存器传送；这里先把 “处理数据” 和 “控制步骤” 分清。

主存储器能由 CPU 直接按地址访问，程序执行时所需的指令和数据通常要进入主存。磁盘、SSD 等外存提供较大且非易失的存储空间；文件保存在磁盘，并不意味着 CPU 每执行一条指令都直接向磁盘取一次。

最小反例·同 ISA 不保证文件通用

最小反例：两系统使用同一指令集，但系统调用号、动态库和可执行文件格式不同。一份应用文件仍可能无法直接运行。ISA 兼容解决指令语义，运行环境兼容还需操作系统与 ABI 条件。

1.1.1 ISA 规定什么，微体系结构决定什么

指令集体系结构 ISA 是软件可见的机器接口，规定机器指令的功能和编码、软件可见的寄存器、数据类型、寻址方式以及相关的存储访问规则。微体系结构则指在该接口约束下怎样组织与实现机器，例如采用几级流水线、怎样转发数据、Cache 怎样实现、加法器采用何种进位结构。

同一个整数加法的机器指令，可以由不同硬件通路实现。只要软件可见的效果符合 ISA，程序不必知道内部用了串行进位还是先行进位。不过，机器码能否执行还取决于运行环境；“同一 ISA” 不自动保证不同操作系统的可执行文件可直接互用。硬件与软件能实现相同逻辑功能，也不表示两者的时间、成本和可修改性相同。

1.2 存储程序怎样变成一次次指令执行

存储程序的基本思想是：执行前把程序指令和数据存入存储器，运行时自动取出指令、解释并执行。指令和数据都可以用二进制编码，机器根据当前操作和访问位置解释其含义。同一串位的 “指令身份” 或 “数据身份” 都离不开上下文。

冯·诺依曼型计算机按指令控制执行过程，常称控制流驱动。遇到转移、调用或异常中断时，下一步不再简单地取顺序地址上的指令，但仍由确定的控制机制改变执行位置。“存储程序” 并不要求程序所在的存储内容在指令执行期间保持不变。

程序计数器 PC 保存指令执行所需的地址信息。最小的顺序执行过程可以写成：

1. 根据 PC 访问指令所在存储单元，取得指令。
2. 解释操作码和有关字段，取得操作数。
3. 执行运算、访存或控制转移，保存结果。
4. 按顺序地址或转移目标更新执行位置，继续取指。

这里的顺序地址增量取决于指令长度和地址单位。按字节编址、指令长 4 B 时，顺序地址通常增加 4；不能把所有机器的 PC 更新都写成 “加 1”。流水线中取指、译码、执行还可能重叠，程序员看到的指令语义与内部时序应分开。

1.2.1 从 C 源文件到 CPU 能执行的内容

在常见的静态编译流程中，预处理展开头文件和宏等指令；编译把高级语言转换为汇编形式；汇编把汇编指令编码成可重定位目标文件；链接合并目标文件与所需库，处理符号引用和重定位，形成可执行文件。装入程序再按可执行文件的组织方式建立进程的地址空间，准备执行入口。

汇编语句 `add` 是便于人阅读的符号，CPU 取到的是其机器编码。宏、伪指令还可能展开为零条、一条或多条机器指令。一条 C 语句也可能展开为多条指令；反过来，编译优化可能把多条 C 语句合并或删除。

1.2.2 例 1-1　硬件直接执行哪一种程序

例 1.2.1 (配套例题)

【配套例题】计算机硬件能够直接执行的是（　）。

I. 机器语言程序　II. 汇编语言程序　III. 硬件描述语言程序

A. 仅 I　B. 仅 I、II　C. 仅 I、III　D. I、II、III

解析。执行普通指令的硬件按 ISA 识别机器编码，因此 I 成立。汇编语言必须经过汇编，硬件描述语言用于描述和设计硬件，不能作为普通机器指令直接送入 CPU 执行。答案为 **A**。题目问的是执行接口，而不是哪种语言更接近硬件。

解释执行是另一种程序执行方式：解释器读取源程序或中间表示，并在运行时完成相应操作。实际解释器本身仍由机器指令运行。现代语言实现也可能结合解释、预编译和即时编译，不能用 “高级语言都逐行解释” 概括。

不变量·先数动态执行次数

总周期数来自真正执行过的每一条指令。循环体写一份不等于只执行一次；分支未走到的代码不贡献本次动态指令数。比较性能时，要让任务、输入和计数区间保持相同。

1.3 性能计算：先确定谁的时间在改变

响应时间是从提出任务到得到结果经历的时间；吞吐量是单位时间内完成的任务量。增加处理器可以让多个独立任务同时进行，因而可能提高系统吞吐量；一个不能并行的单线程任务却不因此自动缩短执行时间。

程序的总运行时间还可能包含 I/O 等待、系统调度等。CPU 时间是 CPU 为该程序工作所花的时间。本节的基本性能式只计算指定口径的 CPU 时间：

$$T_{\text{CPU}} = \frac{N \times \overline{\text{CPI}}}{f} = N \times \overline{\text{CPI}} \times T_{\text{clk}}.$$

N 是实际执行的指令条数，不能用程序文件中的静态指令条数代替；f 是主频，单位 Hz；$T_{\text{clk}} = 1/f$ 是时钟周期；平均 CPI 是每条指令平均占用的时钟周期数。若第 i 类指令执行 n_i 条、平均 CPI 为 c_i，则

$$\overline{\text{CPI}} = \frac{\sum_i n_i c_i}{\sum_i n_i}.$$

不同类指令的 CPI 必须按条数加权平均。CPI 还受 Cache 缺失、流水线停顿等影响，并非仅由指令操作码决定。流水线稳态下 CPI 可以接近 1；多发射处理器的平均 CPI 也可以小于 1，这并不表示一条指令的全部处理延迟小于一个时钟周期。

1.3.1 例 1-2　主频相同，怎样由指令组成求时间

例 1.3.1 (配套例题)

【配套例题】某计算机主频为 1 GHz，程序 P 运行过程中共执行 10000 条指令，其中 80% 的指令执行平均需 1 个时钟周期，20% 的指令执行平均需 10 个时钟周期。程序 P 的平均 CPI 和 CPU 执行时间分别是（　）。

A. 2.8，28 μs　B. 28，28 μs　C. 2.8，28 ms　D. 28，28 ms

解析。条件给的是动态指令比例，先求加权平均：$0.8\times1+0.2\times10=2.8$。总周期数为 $10000\times2.8=28000$；1 GHz 对应每周期 1 ns，因此 CPU 时间为 28000 ns，即 28 μs。答案为 **A**。百分比必须乘各类周期数，GHz 与 μs 的换算不能略去。

1.3.2 例 1-3　CPU 加快后，I/O 时间怎样处理

例 1.3.2 (配套例题)

【配套例题】假定基准程序 A 在某计算机上的运行时间为 100 s，其中 90 s 为 CPU 时间，其余为 I/O 时间。若 CPU 速度提高 50%，I/O 速度不变，则运行基准程序 A 所耗费的时间是（　）。

A. 55 s　B. 60 s　C. 65 s　D. 70 s

解析。“速度提高 50%”是原速度的 1.5 倍，因此原来的 90 s 变为 $90/1.5=60$ s，而不是减为 45 s。I/O 仍占 10 s，总时间为 70 s，选 **D**。

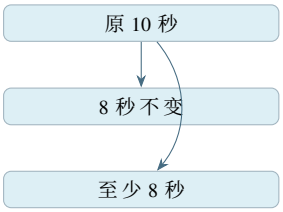
一般地，原运行时间中比例 p 的部分加快为原速度的 s 倍，其余不变，且两部分时间可以这样相加时，加速比为

$$S=\frac{1}{(1-p)+p/s}.$$

该式的上限是 $1/(1-p)$，但前提是加速部分划分和时间模型成立。若 I/O 与计算本来重叠，或并行化新增了通信与同步开销，应先画出实际时间关系，不能直接把各部分都相加。

最小反例·速度翻倍不等于总时减半

最小反例：10 秒任务中只有 2 秒可加速，即使这 2 秒缩短为 0，总时间仍至少 8 秒。只有整个统计对象都按相同倍率加速，总历时才按该倍率缩短；先画出未改变的部分。



可加速部分即使归零，总时间仍不小于 8 秒。

1.3.3 比较性能时，为什么不能只看主频或 MIPS

两台机器的时间之比同时取决于指令条数、平均 CPI 和主频。相同源程序在不同 ISA、不同编译器下可能有不同的动态指令条数。即使平均 CPI 相等，也不能仅凭主频断言谁完成同一任务更快。

性能变化也应放在同一任务与同一统计口径下比较：编译优化可能同时减少动态指令条数、改变指令组成；改善 Cache 通常减少访存停顿，从而改变平均 CPI；提高主频改变每周期时间，但若伴随更多流水级或等待，CPI 也可能变化；增加核心则先要问任务能并行到什么程度。只有题干明确其他量不变时，才把时间比约成某一个因子的比例。

这些变化还对应不同系统层次。ISA 规定一条指令对软件可见的效果，编译器决定怎样用指令表达任务，微体系结构决定这些指令怎样在资源和时序上实现，操作系统决定进程可使用的运行环境与资源。问机器码能否执行，要查 ISA 和运行环境；问同一机器码为何快慢不同，要沿动态指令数、CPI、主频和等待时间寻找变化。用同一个“机器更先进”概括，无法支持具体推算。

MIPS 表示每秒执行多少百万条指令，$\text{MIPS}=f/(\overline{\text{CPI}}\times10^6)$；IPC 是每周期平均完成的指令数，在统计区间一致时与平均 CPI 互为倒数。MFLOPS 等指标统计浮点运算次数，与机器指令条数不是同一量。跨机器比较应优先比较同一任务的实际时间，再看指标如何解释差异。

机器字长主要反映 CPU 一次处理整数数据的基本位宽。它关系到整数表示范围和相关运算能力，但不能单独推出主存容量、总线宽度或某程序的速度。“64 位机器”也不表示所有数据类型都占 64 位。

1.4 从层次图走到可核算的模型

性能题四步

1. 写清任务和统计范围：CPU 执行时间、总历时还是单位时间完成数。
2. 数动态指令，按条数加权求 CPI；把 GHz、ns、微秒统一。
3. 标出变化因子：指令数、CPI、主频或可并行比例；不变条件必须来自题设。
4. 用原方案回代，再检查加速后的总时间是否仍包含不可加速部分。

1.4.1 为什么 CPI 按条数加权

从总周期数出发

第 i 类指令执行 n_i 条，每条占 c_i 周期，故该类总周期为 $n_i c_i$；各类相加得到 $C = \sum_i n_i c_i$。平均每条周期必为 $C/(\sum_i n_i)$，因此权重是指令条数比例，而非各类指令所占时间比例。乘上共同的时钟周期，才得到 CPU 时间；若各阶段频率不同，应分阶段计算。

□

层次结构图中的 “机器” 是一种抽象视角。高级语言机器依靠编译或解释实现，汇编语言机器依靠汇编工具，操作系统提供资源与服务接口，传统机器执行 ISA 指令，微程序机器描述某类控制器内部实现。并非所有 CPU 都采用微程序控制；不可把五层图误读成每个程序必经五种独立硬件。

1.4.2 从地址位数到容量

若有 m 位完整地址、按字节编址且所有地址均可用，则最多可区分 2^m 个字节位置；若按 w 位字编址，容量应为 $2^m \times w$ 位。MDR 或数据总线宽度描述一次交换的数据位数，不能单独代替地址位数。以 20 位字节地址为例，可编址 1MiB；若同为 20 位地址却每地址对应 16 位字，则可编址 2MiB。

课程图中容量常按 2^{10} 递增，而频率、传输率通常按 10^3 递增。为避免含混，容量推导写成 KiB/MiB 或明确 “本题 K=2^{10}”；带宽则把每次字节数乘每秒次数，最后再按题设进制换单位。

1.4.3 发展历程的背景定位

电子管、晶体管、中小规模集成电路、大规模与超大规模集成电路依次推动器件集成、可靠性和成本变化。它们帮助理解实现技术的演变；性能判断仍要回到具体任务的执行时间。教材与课件把部分发展史标为参考内容，本章保留这一背景，不以年代细节替代核心性能模型。

层次	向上说明什么	不能直接推出什么
ISA	软件可见指令与状态	具体流水段数、Cache 组织
微体系结构	指令在部件和时序上的实现	应用跨操作系统直接兼容
操作系统	资源、进程与运行接口	所有应用可并行加速

最小反例·字长不能推出地址容量

同样是 32 位整数处理器，可以配较小物理内存，也可以有更宽物理地址。最小反例已表明 “字长 32 位所以最多 4GiB” 缺少地址位数前提。容量必须由地址数与每地址对应的单位共同确定。

1.5 五大部件怎样共同完成一次计算

课程连接·五部件的连接

存储器提供指令与数据，控制器协调传送与运算；同一条指令还需明确数据经过哪些寄存器。

冯·诺依曼计算机把指令和数据都存放在存储器中，用地址选择存储单元。指令与数据在存储形式上都表现为二进制位串；CPU 取出的内容被当作指令还是操作数，取决于当前执行阶段和指令语义。早期机器以运算器为中心，外设和主存之间的数据常经运算器；现代组织通常以存储器为中心，外设可借助接口、总线和 DMA 与主存交换数据。

功能部件	基本任务	与程序执行的关系
运算器	算术、逻辑运算以及移位	ALU 完成运算，寄存器保存操作数和中间结果
控制器	取指、分析指令、产生控制信号	决定何时传送数据、执行哪种运算以及下一步做什么
主存储器	保存正在使用的指令与数据	提供按地址读写的存储单元
输入设备	将外部信息转换成机器能处理的形式	经接口把程序和数据送入系统
输出设备	将处理结果转换成人能识别或外部需要的形式	经接口接收输出数据和控制信息

主机通常指 CPU 和主存；CPU 包括运算器与控制器。辅存、输入设备、输出设备合称外部设备。按功能划分的部件与按物理芯片划分的部件未必一一对应：一个 CPU 芯片可以包含多个核、多级 Cache 和其他控制逻辑。

1.5.1 存储字、机器字和指令字的区别

一个存储单元中保存的位串称为存储字，位数为存储字长。机器字长一般指 CPU 一次能处理的二进制数据位数，常与通用寄存器、运算部件的数据通路宽度相关。指令字长是某条指令所占的位数；一种 ISA 可以采用固定长度或可变长度的指令。

设按字编址的主存共有 2^{10} 个单元、每单元 16 位，则容量为 $2^{10}\times 16$ bit，即 2 KiB；地址至少需要 10 位。若同样的 2 KiB 容量改按字节编址，则共有 2^{11} 个可编址单元，地址至少 11 位。求容量时先写“单元数 $\times$ 每单元位数”，能避免把地址线数与数据线数互换。

运算器中的 ACC 可暂存操作数或结果，MQ 可辅助保存乘数或乘积低位，X 可暂存另一个输入；这些是课件示意机器的组织约定。现代机器未必使用完全相同的专用寄存器，分析具体数据通路仍应以题设为准。

1.5.2 例：从主存中的程序求出 $y=ab+c$

设主存按字编址，地址 0～4 依次保存取数、乘法、加法、存数和停机指令；地址 5～8 分别存放 $a=2$、$b=3$、$c=1$ 和 $y=0$。取指时，PC 给出的是指令地址；执行访存指令时，地址字段给出的是数据地址。

执行的指令	主要数据变化	完成后的状态
0：取地址 5 中的数	$M[5]\rightarrow$ ACC	ACC=2，PC 指向 1
1：乘地址 6 中的数	ACC $\times M[6]\rightarrow$ ACC	ACC=6，PC 指向 2
2：加地址 7 中的数	ACC $+M[7]\rightarrow$ ACC	ACC=7，PC 指向 3
3：将结果存入地址 8	ACC $\rightarrow M[8]$	$M[8]=7$，PC 指向 4
4：停机	停止继续取执行指令	计算结束

同一条访存指令至少要区分 “取指令” 和 “取操作数”：前者由 PC 形成地址，后者由 IR 的地址字段及寻址规则形成地址。MAR 在不同时刻可保存这两类地址，MDR 在不同时刻可保存指令或数据；寄存器名称并未限定其中只能保存一种内容。

课程连接·程序如何到机器上

编译得到目标代码，链接处理符号与所需模块，加载将程序置入内存；不要把这三个阶段混为一次操作。

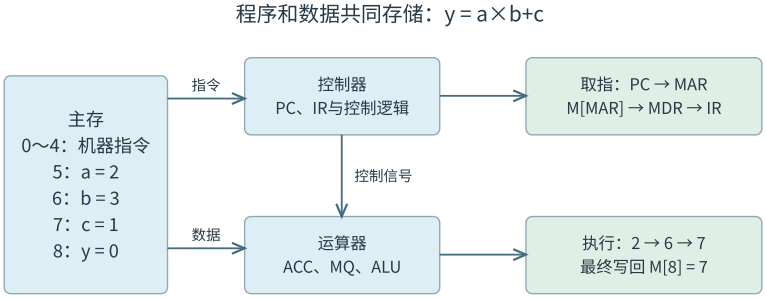


Figure 1.1: 主存按地址同时保存程序与数据，控制器组织取指与执行

1.6 系统软件、应用软件与程序翻译

系统软件负责管理资源、提供运行环境和支撑程序开发，包括操作系统、语言处理程序、数据库管理系统及有关工具；应用软件面向具体任务。编译程序把高级语言程序翻译成目标语言程序，解释程序则在解释过程中组织执行，高级语言语句与机器指令通常不是一一对应关系。

软件与硬件的逻辑功能等价，是指同一逻辑功能可以用软件算法实现，也可以增加专门硬件实现。选择哪种实现取决于性能、成本、功耗和灵活性。它不意味着软件无需硬件，也不意味着两种实现的时间、空间开销相同。

从下向上可以把系统理解为微程序机器级、传统机器语言级、操作系统级、汇编语言级和高级语言级。层次中的 “虚拟机器” 是下层向上层提供的一套使用规则；某一功能在下层实现时，对上层可以是透明的。“透明” 在体系结构语境中通常指使用者无需看见或处理其实现细节。

1.7 性能指标的统计口径与单位

主频 f 与时钟周期 T 互为倒数。对含有多类指令的程序，若第 i 类执行 n_i 条、每条平均需要 CPI_i 个周期，则

$$N=\sum_i n_i,\qquad C=\sum_i n_i \mathrm{CPI}_i,\qquad t_{\mathrm{CPU}}=\frac{C}{f}.$$

吞吐量衡量单位时间完成多少工作，响应时间衡量从请求开始到结果完成经历多久。提高并行度可能增加吞吐量，而单个请求的响应时间还受到等待、通信和 I/O 影响。比较两台机器运行同一任务的性能时，性能比等于执行时间的反比。

MFLOPS 表示每秒百万次浮点运算，GFLOPS、TFLOPS 分别采用 10^9、10^{12} 量级；它们侧重浮点工作负载。MIPS 为每秒百万条指令，依赖 ISA、编译结果和指令组成。容量中的 KiB 采用 2^{10} B，而频率中的 MHz 采用 10^6 Hz，计算时应保持量纲一致。

1.7.1 拓展：硬件技术的演进

电子管、晶体管、中小规模集成电路、大规模和超大规模集成电路是常用的发展阶段划分。集成度提高使存储容量、运算速度和可靠性提升，并推动微型化、多核、网络化以及高性能并行计算。摩尔定律描述历史上集成电路晶体管数量增长的经验趋势，不能直接据此把某个程序的运行时间按相同比例缩短。

课程连接·自查顺序

先写公式和单位，再代入数值；题目限定的工作负载决定可以比较的指标。

1.7.2 计算与辨析自查

1. 为什么同一个 MDR 既能接收指令又能接收数据?
2. 主频 2 GHz，运行 10^8 条指令，平均 CPI 为 1.5，CPU 时间是多少?
3. 4 KiB 主存、存储字长 32 位，分别按字和按字节编址需要几位地址?

答案。1. MDR 保存存储器本次交换的数据位串，内容的用途由执行阶段决定。2. $10^8 \times 1.5/(2 \times 10^9) = 0.075$ s。3. 按字编址有 1024 个单元，需 10 位；按字节编址有 4096 个单元，需 12 位。

1.8 主动回忆自测

1.8.1 配套自测

1. 【配套例题】程序 P 在机器 M 上的执行时间是 20 s。编译优化后，P 执行的指令数减少到原来的 70%，而 CPI 增加到原来的 1.2 倍，则 P 在 M 上的执行时间是（ ）。A. 8.4 s　B. 11.7 s　C. 14 s　D. 16.8 s。
2. 原创补充：两个任务都由同一单核 CPU 各执行 2 s。换成两核后，每个任务仍占用一个核执行 2 s，且可同时开始。两个任务的总完成时间和单个任务的响应时间分别怎样变化?

答案。1. D。主频不变，时间变为 $20 \times 0.7 \times 1.2 = 16.8$ s。2. 两个任务全部完成的时间从 4 s 降为 2 s；无排队时单个任务本身的执行时间仍为 2 s。该小例用于区分吞吐量改善与单任务加速。

1.8.2 遮住答案复述知识关系

主动回忆·自拟

1. 不看正文，画出五大功能部件，并圈出 CPU 与主机各自包含什么。
2. 同一 ISA 的两台机器为什么能执行同一类机器指令，却可能有不同 CPI?
3. 一段程序 CPU 时间占 80%，只把 CPU 部分加速为 2 倍，总加速比是多少?
4. 20 位地址能否直接推出主存为 1MiB? 还缺哪个条件?
5. 用一句话分别说明响应时间、吞吐量、IPC 和 FLOPS 的统计对象。

1.8.3 参考答案

1. 五部件为输入、输出、存储、运算、控制；常见模型 CPU 含运算与控制及寄存器，主机还含主存。

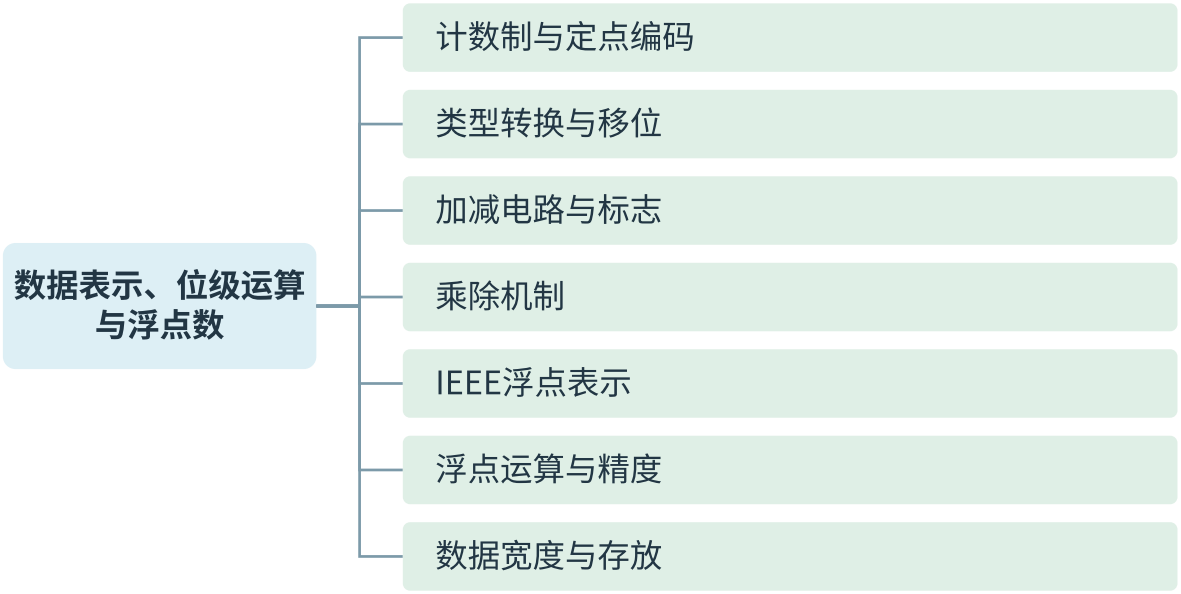
2. ISA 规定可见语义，流水线、Cache、功能部件与等待时序由具体微架构决定。
3. 原时间记 1，变为 $0.2 + 0.8/2 = 0.6$，加速比 $5/3$。
4. 须说明按字节编址、完整 20 位地址均可用；已安装物理容量还可能更小。
5. 响应时间按单任务历时，吞吐按单位时间完成任务数，IPC 按周期完成指令数，FLOPS 按秒完成浮点运算数。

课程连接·自查顺序

先写公式和单位，再代入数值；题目限定的工作负载决定可以比较的指标。

数据表示、位级运算与浮点数 2

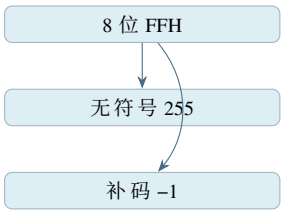
章节脉络



本章围绕“计数制与定点编码 → 类型转换与移位 → 加减电路与标志 → 乘除机制 → IEEE 浮点表示 → 浮点运算与精度 → 数据宽度与存放”展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

最小反例·最高位 1 并不总是负数

最小反例：同一 8 位串 FFH 按无符号解释是 255，按补码解释是 −1。编码和位宽必须与位串一同给出。直接把最高位称作负号，会把无符号扩展、浮点阶码和地址都误读成有符号数。



同一位串依编码得到不同真值。

2.1 位串与数值之间还差一个解释规则

定义 2.1.1 (概念与适用对象)

位是取 0 或 1 的二进制位，字节 B 在本讲义中为 8 位，bit 写作 b。同一位串可以解释成无符号整数、补码整数、浮点数或字符编码。计算前必须同时确定**位宽、编码和小数点位置**，不能看见最高位为 1 就断言“这个数是负数”。

r 进制采用位置权，$(a_k \cdots a_0.a_{-1} \cdots)_r$ 的值是 $\sum_i a_i r^i$，其中 $0 \le a_i < r$。二进制与八进制、十六进制转换时，从小数点向两侧分组：每 3 位一组对应一位八进制，每 4 位一组对应一位十六进制；整数端左侧补 0，小数端右侧补 0。

2.1.1 例 2-1　整数与小数分别换基数

例 2.1.1 (配套例题)

【配套例题】若十进制数为 137.5，则其八进制数为（　）。

A. 89.8　　B. 211.4　　C. 211.5　　D. 1011111.101

解析。$137 = 2\times 8^2 + 1\times 8 + 1$，小数部分 $0.5\times 8 = 4$，因此为 $(211.4)_8$，选 **B**。八进制数字只能是 0 至 7，A 本身就不是合法八进制数。分组也可验证：$137.5 = (10001001.1)_2 = (010\ 001\ 001.100)_2$。

十进制整数转二进制可反复除 2 取余、倒序排列；小数则反复乘 2 取整数、顺序排列。有限十进制小数未必能写成有限二进制小数，例如 0.1 的二进制表示无限循环。保存有限位时会发生近似，这正是后面浮点舍入出现的原因之一。

不变量·范围来自位权

补码最高位权为负，其余位权为正；最小值只有最高位 1，最大值只有最高位 0。每次推范围先写位权，就能同时解释零唯一、负数多一个，以及最小负数同宽取负不可表示。

2.2 定点编码：范围先于运算规则

真值是按通常数学符号写出的数值；机器数是采用约定编码保存的位串。定点表示约定小数点位置固定，并不表示存储器真的另存了一个小数点。

对 n 位整数，常用编码如下。

编码	最高位和数值含义	表示范围
无符号	所有位均有正权	0 至 2^n-1
原码	最高位为符号，其余为绝对值	$-(2^{n-1}-1)$ 至 $2^{n-1}-1$，两个零
反码	正数同原码，负数的数值位取反	与原码相同，两个零
补码	最高位权为 -2^{n-1}，其余为正权	-2^{n-1} 至 $2^{n-1}-1$，一个零

8 位补码 `1111 1101` 的值为 $-128+64+32+16+8+4+1=-3$。也可先把它作为无符号数 253，再因最高位为 1 减去 256，得到 -3。后一方法适合十六进制机器数：对 n 位补码，若无符号解释为 u，则最高位为 0 时取 u，为 1 时取 $u-2^n$。

求一个负整数的补码时，可在给定位宽下把它的绝对值逐位取反再加 1；位宽不能省。8 位中 -128 编码为 `1000 0000`，它的数学相反数 128 不能用 8 位补码表示。“取反加 1” 得到的仍是同一位串，这说明运算按模 256 进行，并不表示已经得到可表示的正 128。

原码中的 `1000 0000` 表示 -0，补码中则表示 -128。反码的 -0 是 `1111 1111`。对于常见的偏置 2^{n-1} 的整数移码，有 $E = x + 2^{n-1}$，与对应补码相比只需翻转最高位；IEEE 754 阶码的偏置通常为 $2^{k-1}-1$，不可套用 “翻转最高位” 这一特殊关系。

若采用一位符号、$n-1$ 位小数的补码定点小数，最高位权为 -1，其余权从 2^{-1} 至 $2^{-(n-1)}$，范围为 $[-1, 1-2^{-(n-1)}]$。同样的位串改变小数点位置，真值就按 2 的幂缩放。

2.3 截断、扩展与 C 类型转换

位宽变小时，硬件截取低位；能否保留原数值，要检查被丢掉的位。位宽增大时，无符号数在高位补 0，有符号补码数复制符号位。原码、反码各有自己的扩展规则，补码规则并不适用于所有编码。

8 位补码 -3 扩为 16 位是 `1111 1111 1111 1101`，无符号 253 扩为 16 位则是 `0000 0000 1111 1101`。扩展后低 8 位相同，真值不同。**扩展方式首先取决于源数怎样解释，而不是目标类型名字中有没有 unsigned**。

2.3.1 例 2-2 无符号短整数扩展

错项排除·扩展看源数解释

FFFAH 若是 unsigned short 则为 65530，扩宽补 0；若是 signed short 补码则为 −6，扩宽复制符号。目标名字中出现 unsigned 不代表先补 0：数值转换还需按目标模数处理。

例 2.3.1 (配套例题)

【配套例题】假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，执行下列 C 语言语句：

```
unsigned short x = 65530;
unsigned int y = x;
```

得到 y 的机器数为（ ）。

A. 0000 7FFAH B. 0000 FFFAH C. FFFF 7FFAH D. FFFF FFFAH

解析。$65530 = 65536 - 6 =$ `FFFAH`，源类型是无符号 16 位数，扩展补 0，得到 `0000 FFFAH`，选 **B**。最高位为 1 在这里是普通数值位，不是负号。

2.3.2 例 2-3 负的 short 转成 unsigned int

例 2.3.2 (配套例题)

【配套例题】在 32 位计算机上执行下列 C 语言代码段后，ui 的值是（ ）。

```
short si = -32767;
unsigned int ui = si;
```

A. $2^{15}-1$ B. $2^{15}+1$ C. $2^{32}-2^{15}-1$ D. $2^{32}-2^{15}+1$

解析。按题目的 16 位 short、32 位 unsigned int 模型，-32767 的 16 位补码为 `8001H`。数值转换为 32 位无符号整数后，得到与 -32767 模 2^{32} 同余的数，即 $2^{32}-32767 = 2^{32}-2^{15}+1$，机器数为 `FFFF 8001H`，选 **D**。把 `8001H` 直接补 0 会把负的源值误当成无符号 32769。

C 中的 usual arithmetic conversions 也会改变比较方式。若 32 位 `int i=-1` 与 32 位 `unsigned u=1` 比较，常见模型下 i 转为无符号 $2^{32}-1$，所以 `i<u` 为假。需要先完成整数提升和通常算术转换，再决定按哪种值比较。

本讲义中 “截断取低位” 的位级推演按题设机器模型进行。C 语言本身不保证有符号溢出按补码绕回：有符号加法溢出属于未定义行为；按本讲义采用的 C11/C17 口径，超范围转换到有符号类型还涉及实现规定。考题若明确给机器位宽和截断模型，可据模型计算；编写可移植 C 程序时不能把这些题设当成语言的普遍保证。

2.3.3 移位先问逻辑还是算术

逻辑右移高位补 0，算术右移复制符号位；左移均在低位补 0，但如何解释结果及是否溢出仍取决于编码。8 位补码 -3 算术右移 1 位得到 -2，相当于向负无穷方向取整；整数除法 $-3/2$ 通常向零截断得到 -1，两者不能无条件互换。左移时被丢弃的高位无法恢复，“低位补 0” 也不保证

数值恰好乘 2。

不变量·减法中的一只加一

控制 Sub=1 时第二路逐位取反，最低进位再加 1，总体才是 A−B。若已经送入 B 的负数补码，又把最低进位设 1，就把那一个 1 重复加入。追踪输入端比只写 “取补码” 更精确。

2.4 加法器怎样同时支持加与减

一位全加器接收 A_i, B_i, C_i，产生和位 S_i 与向高位的进位 C_{i+1}：

$$S_i = A_i \oplus B_i \oplus C_i, \qquad C_{i+1} = A_iB_i + (A_i \oplus B_i)C_i.$$

定义进位生成 $G_i = A_iB_i$、进位传递 $P_i = A_i \oplus B_i$，即可逐层展开进位表达式。串行进位需等低位进位逐级传播；先行进位用更复杂的组合逻辑并行形成若干进位，缩短关键路径。更快的进位结构没有改变加法的数值规则。

加减运算器用控制位 Sub 将第二输入按位与 Sub 异或，同时令最低进位 $C_0 = \text{Sub}$。Sub=0 时执行 $A + B$；Sub=1 时执行 $A + \overline{B} + 1$，即模 2^n 的减法。算术逻辑部件 ALU 还通过选择控制实现与、或、异或等逻辑操作；移位器是否算在 ALU 内部，要看具体的数据通路图。

2.4.1 例 2-4　减法器的两路输入不直接送负数符号

例 2.4.1 (配套例题)

【配套例题】某计算机字长为 8 位，CPU 中有一个 8 位加法器。已知无符号数 $x = 69, y = 38$，若在该加法器中计算 $x - y$，则加法器的两个输入端信息和输入的低位进位信息分别为（　）。

A. 0100 0101、0010 0110、0

B. 0100 0101、1101 1001、1

C. 0100 0101、1101 1010、0

D. 0100 0101、1101 1010、1

解析。$69 = 0100\,0101$, $38 = 0010\,0110$。第二路取反为 `1101 1001`，另由最低进位补上 1，所以选 **B**。若第二路已取反加 1 又令最低进位为 1，就重复加了 1。运算实际为 $69 + 217 + 1 = 287$，低 8 位 31、最高进位 1，符合 $69 - 38 = 31$。

2.5 CF 与 OF 分别判断什么

同一次加法器运算可以同时按有符号和无符号两种规则解释，因此产生不同用途的标志。ZF 表示结果低 n 位是否全 0，SF 等于结果最高位；SF=1 并不自动表示数学真值为负，还要确认按有符号解释且未发生有符号溢出。

对于加法, CF 反映最高位向外进位; 对于本讲义采用的 “CF=1 表示借位” 约定, 减法的 CF 为最高进位的反值。可统一写为 $\text{CF} = C_n \oplus \text{Sub}$。某些 ISA 用 “无借位” 作为减法进位标志，遇到这类题设时，必须按该 ISA 的约定判断。

OF 判断有符号结果是否越出补码范围：加法时同号相加而结果异号会溢出；减法时两操作数异号且结果符号与被减数不同会溢出。也可用符号位的进位输入与进位输出异或：$\text{OF} = C_{n-1} \oplus C_n$。它不要求把符号位当成独立的符号去做原码运算。

2.5.1 同一个结果，按哪一种数判断

位串运算、数值解释和条件判断应分三步。先按位宽完成加减，保留低 n 位；再按无符号或补码规则解读；最后选择与这种解释对应的标志。不能为了得到希望的正负号，改写已经产生的机器结果。

8 位运算	保留结果	按无符号解释	按补码解释
`7FH+01H`	`80H`	$127+1=128$，可表示	$127+1$ 超过 127
`FFH+01H`	`00H`	$255+1$ 超过 255	$-1+1=0$，可表示
`80H-01H`	`7FH`	$128-1=127$，无借位	$-128-1$ 小于 -128
`00H-01H`	`FFH`	$0-1$ 需借位	$0-1=-1$，可表示

最小反例·结果负号不能代替 OF

最小反例：7FH 加 01H 得 80H，SF=1 而精确和 128 为正，只因结果已越界。相反 FFH 加 01H 得 00H，CF=1 却没有补码溢出。只看结果正负，会同时误判这两类题。

同一过程的其余状态（按首列对应）：

8 位运算	CF、OF
`7FH+01H`	0、1
`FFH+01H`	1、0
`80H-01H`	0、1
`00H-01H`	1、0

ZF 和 SF 只看保留下来的结果：上表第二行 ZF=1，第一行与第四行 SF=1。若要由减法结果判断补码数的有符号小于或大于关系，应结合 OF 修正结果符号；例如第三行 SF=0，却不能据此说原来的 -128 大于 1。是否相等只需看 ZF，不必额外使用 OF。编码范围决定“能否表示”，运算电路决定“产生哪些位”，转移条件决定“怎样使用标志”，三者要连起来判断。

2.5.2 例 2-5　十六进制补码相减

例 2.5.1 (配套例题)

【配套例题】假定有符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x、y 的值及 x−y 的机器数分别是（　）。

A. x=−65，y=41，x−y 的机器数溢出
B. x=−33，y=65，x−y 的机器数为 FFFF FF9DH
C. x=−33，y=65，x−y 的机器数为 FFFF FF9EH
D. x=−65，y=41，x−y 的机器数为 FFFF FF96H

解析。`FFFFFFDFH` 距 2^{32} 差 33，故 x=−33；`41H` 是十进制 65。数学结果为 -98，在 32 位补码范围内，机器数为 $2^{32}-98=$ `FFFFFF9EH`，选 **C**。十六进制的 41 不能按十进制 41 读。

2.5.3 例 2-6　有符号减法为何仍会产生借位

例 2.5.2 (配套例题)

【配套例题】已知 x、y 为 int 类型，当 x=100、y=200 时，执行“x 减 y”指令得到的溢出标志 OF 和借位标志 CF 分别为 0、1。那么当 x=10、y=−20 时，执行该指令得到的 OF 和 CF 分别是（　）。

A. OF=0, CF=0　B. OF=0, CF=1　C. OF=1, CF=0　D. OF=1, CF=1

解析。有符号真值为 $10-(-20)=30$，可表示，OF=0。若按无符号解释输入位串，-20 对应 $2^{32}-20$，10 小于它，减法需要借位，CF=1，选 **B**。题目已经用 100−200 说明了借位标志约定，不能把两个标志都按有符号真值判断。

条件转移利用这些标志完成比较。先计算 $a-b$：无符号 $a<b$ 对应 CF=1，无符号 $a>b$ 对应 CF=0 且 ZF=0；带符号 $a<b$ 对应 SF $\neq$ OF，带符号 $a>b$ 对应 ZF=0 且 SF=OF。用 SF 单独判断带符号大小，会在溢出时失败。

不变量·完整积保留双倍宽

两个 n 位输入的完整积最多 2n 位。判断截为 n 位是否有符号溢出，要看高半部是否恰为低半部符号扩展；只看高半部是否全 0，会把合法负积误判。

2.6 乘除运算必须保留的中间量

2.6.1 无符号乘法：为什么需要部分积和双倍位宽

两个 n 位无符号数的精确乘积最多需要 $2n$ 位。以王道书 §2.2 中的 13 乘 11 为例：$13=1101_2$，$11=1011_2$。乘数从低位到高位为 1、1、0、1，部分积依次是 13、26、0、104，总和 143，即 `1000 1111`。

顺序乘法器可以每轮检查乘数最低位，决定是否把被乘数加到部分积高半部，再将 “进位—部分积—乘数” 组合右移。部分积开始为 0，循环 n 次后得到 $2n$ 位积。相应的等价算法也可让被乘数左移、乘数右移，把选中的部分积加到双倍宽累加器；两种寄存器布局的逐轮数值不同，不应把一种布局的移位规则套用到另一种。

对 13×11，用双倍宽累加器的轨迹如下：

本轮乘数低位	本轮被乘数	累加器更新后	乘数右移后
1	13	13	5
1	26	39	2
0	52	39	1
1	104	143	0

带符号乘法不能把负数的补码直接当成同宽无符号数求完整积。Booth 法在乘数末端附一位 0，考察相邻两位 q_iq_{i-1}：01 时加被乘数，10 时减被乘数，00 或 11 不加减，再对组合寄存器算术右移。这里的前提是符号扩展到位、中间寄存器足够宽；对最小负数取负时，尤其需要保护位。

以 4 位补码的 -3×5 说明 Booth 法（原创机制示例）。乘数寄存器 Q 为 4 位，附加位 q_{-1} 为 1 位，A 采用 5 位补码以保留保护位，被乘数 M=−3 也扩成 5 位。初态 A=`00000`、Q=`0101`、$q_{-1}=0$。每轮先看 Q_0q_{-1}，对 A 加减 M，再把 A、Q、q_{-1} 连接成 10 位整体算术右移；A 的最低位移入 Q 最高位，Q 最低位移入 q_{-1}。

轮次	判断位	移位前操作及 A 真值	移位后的 A
1	10	A−M=3	00001
2	01	A+M=−2	11111
3	10	A−M=2	00001
4	01	A+M=−2	11111

同一过程的其余状态（按首列对应）：

轮次	移位后的 Q	q_{-1}
1	1010	1
2	0101	0
3	0010	1
4	0001	0

不变量·余数必须最终回验

无符号除法结束后，同时核 N=QD+R 及 0≤R<D。恢复法与不恢复法可以出现不同中间余数，但最终条件相同；不恢复法若结束时 R 为负，常需补做一次加 D 校正。

四轮后 AQ 为 9 位 `11111 0001`，去掉冗余符号保护位即 8 位 `1111 0001`，真值为 -15。移位时必须复制 A 的符号位，并在寄存器边界传位；把 A、Q 分别右移会丢掉跨界位，得到错误乘积。

阵列乘法器用组合电路完成部分积生成与压缩求和，可能在一个足够长的时钟周期内完成；顺序乘法器复用 ALU 和移位器，通常跨多个周期。“一个周期”不能脱离所用电路和时钟周期长度讨论。常数乘法可以在保持规定语义的前提下化为移位与加减，例如 $13x = (x \ll 3) + (x \ll 2) + x$。

2.6.2 除法：商来自每一步余数比较

以无符号 7÷3 为例，按被除数的有效位 `111` 从高到低处理，初始余数 R=0。每读入一位，先令 $R \leftarrow 2R + b$，若 R≥3 则减 3 并在商中记 1，否则记 0。

读入位	比较前余数	本位商	本步余数
1	1	0	1
1	3	1	0
1	1	0	1

商为 `010` 即 2，余数 1，满足 $7 = 2 \times 3 + 1$ 且 $0 \le R < 3$。硬件通常固定处理整个位宽，在前面补零位，不会因此改变结果。恢复余数法先试减，结果为负时再加回除数；不恢复余数法根据余数符号决定下一轮加还是减，用不同轨迹完成同一任务。

不恢复余数法也可用同一 7÷3 观察。令余数 R 为 4 位有符号寄存器、Q 为 3 位，除数 D=3，初态 R=0、Q=`111`。每轮先把 R、Q 连接左移一位，Q 最高位移入 R 最低位；若上一轮 R 非负，本轮减 D，若上一轮 R 为负则加 D；运算后 R 非负就在 Q 最低位置 1，否则置 0。

轮次	左移后 R 真值	加减操作	操作后 R
1	1	R−D	−2
2	−3	R+D	0
3	1	R−D	−2

同一过程的其余状态（按首列对应）：

轮次	记商后的 Q
1	110
2	101
3	010

最后 R 仍负，补做 R←R+D，得到余数 1，Q=`010` 为商 2。第二轮没有先把 -2 恢复成 1 再试减，而把恢复效果合并进下一轮加法，这正是不恢复余数法的含义；最终余数校正不能漏掉。硬件布局或商编码约定不同会改变表中轨迹，做题时应遵循同一套规则。

有符号除法还要确定商、余数符号和舍入方向。按 C 向零截断模型，-7÷3 的商为 -2、余数为 -1，仍满足被除数 = 商 × 除数 + 余数。

除数为 0 时需要异常处理；$-2^{n-1}/(-1)$ 的商超出 n 位补码范围。某种除法器可能检测到被除数绝对值小于除数，就提前输出商 0；另一种固定轮数除法器仍运行完整迭代。只有题干电路给出提前退出逻辑时，才能比较哪组数据用时最短。

错项排除·字段长度不等于精度

binary32 尾数字段 23 位，正规格化数隐含前导 1，因此有效精度 24 位；非规格化数隐含 0。把所有编码都当 1.F，会把最小正数、零和特殊值混在一起。先判断阶码是否全 0 或全 1。

2.7 IEEE 754：字段既规定精度，也规定特殊情况

IEEE 754 binary32 即通常的单精度格式，由 1 位符号 s、8 位阶码 E、23 位尾数字段 F 构成；binary64 分别为 1、11、52 位。这里 F 的整数值要除以 2^{23} 或 2^{52} 才是小数尾数部分。

项目	binary32（单精度）	binary64（双精度）
总位数	32 位	64 位
符号位 s	1 位	1 位
阶码 E	8 位	11 位
尾数字段 F	23 位	52 位
阶码偏置 B	127	1023
规格化数的真阶码 e	$-126 \le e \le 127$	$-1022 \le e \le 1023$
规格化数的有效精度	24 位，含隐含前导 1	53 位，含隐含前导 1

项目	binary32（单精度）	binary64（双精度）
最小正规格化数	2^{-126}	2^{-1022}
最小正非规格化数	2^{-149}	2^{-1074}
最大有限正数	$(2-2^{-23})2^{127}$	$(2-2^{-52})2^{1023}$

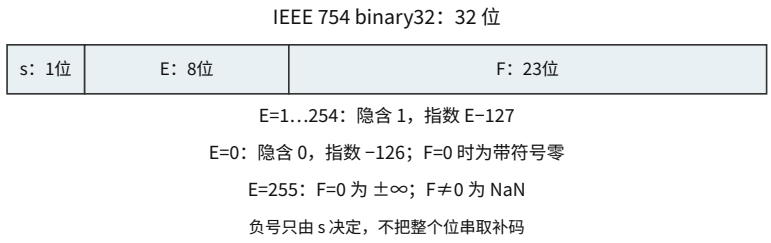


Figure 2.1: 单精度字段及三类指数编码

解码时先看阶码是否全 0 或全 1，再看尾数字段。下表中 p 是尾数字段位数，binary32 取 23、binary64 取 52；B 是对应的阶码偏置，F 按无符号整数解释。

类别	阶码 E	尾数字段 F	数值或含义
正零、负零	全 0	0	$(-1)^s 0$，符号位区分正负零
非规格化数	全 0	非 0	$(-1)^s(F/2^p)2^{1-B}$，隐含前导 0
规格化数	既非全 0，也非全 1	任意	$(-1)^s(1+F/2^p)2^{E-B}$，隐含前导 1
正无穷、负无穷	全 1	0	$(-1)^s\infty$
NaN	全 1	非 0	非数；不能当作无穷或普通实数

对于 binary32：

$$x=\begin{cases}(-1)^s(1+F/2^{23})2^{E-127}, & 1\le E\le 254;\\ (-1)^s(F/2^{23})2^{-126}, & E=0;\\ (-1)^s\infty, & E=255, F=0;\\ \text{NaN}, & E=255, F\neq 0.\end{cases}$$

不变量·阶码字段先减偏置

普通 IEEE 浮点数的指数是 E 减偏置，尾数字段另按二进制小数解释。整数移码常见的翻转符号位关系，不能直接套到偏置 127 的 IEEE 阶码。编码和解码各走一遍可验证字段。

规格化数有隐含的前导 1，有效精度 24 位。非规格化数前导为 0，最小指数同样取 -126，数值逐渐接近 0；当 F=0 时是带符号零。非规格化数不能再用指数 E−127 配隐含 1 计算。

最小正规格化数为 2^{-126}，最小正非规格化数为 2^{-149}，最大有限正数为 $(2-2^{-23})2^{127}$。这些是数值范围；“能连续精确表示的整数” 是另一问题。binary32 能精确表示 $[-2^{24},2^{24}]$ 内所有整数，更大的某些整数仍能精确表示，但相邻可表示数的间隔可能大于 1。

2.7.1 例 2-7 负小数怎样组成单精度机器数

例 2.7.1 (配套例题)

【配套例题】−0.4375 的 IEEE 754 单精度浮点数表示为（ ）。

A. BEE0 0000H B. BF60 0000H C. BF70 0000H D. C0E0 0000H

解析。$0.4375=0.0111_2=1.11_2\times 2^{-2}$。

s=1，E=−2+127=125=`01111101`，F 为 `110` 后补 20 个 0。

连接为 `1|01111101|11000000000000000000000`，十六进制为 `BEE00000H`，选 **A**。负数只改变符号位，不对浮点数全部位取补码。

2.7.2 例 2-8 同为负浮点数时怎样比较

例 2.7.2 (配套例题)

【配套例题】float 型数据常用 IEEE 754 单精度浮点格式表示。假设两个 float 型变量 x 和 y 分别存放在 32 位寄存器 f1 和 f2 中，若 `(f1)=CC90 0000H`、`(f2)=B0C0 0000H`，则 x 和 y 之间的关系为（ ）。

A. x<y 且符号相同 B. x<y 且符号不同 C. x>y 且符号相同 D. x>y 且符号不同

解析。两个最高位都为 1，均为负。x 的阶码为 153，实际指数 26；y 的阶码为 97，实际指数 −30。两者均为有限规格化数，x 的绝对值远大于 y，因此 x<y，选 **A**。此处不必把尾数展开为完整十进制数。对负数，绝对值更大的反而更小。

int 转 float 可能丢低位，32 位 int 转 binary64 则可精确表示，因为 binary64 有 53 位有效精度。float 转 double 在通常 IEEE 格式下能精确保留原来的有限数值，但不能恢复 float 形成时已经丢掉的信息。浮点转整数按语言规则截断，且须检查范围；NaN 与任何数（包括自身）的相等比较都为假，不能把 `x/x==1` 用在 x=0 或无穷等情形。

2.8 浮点加减：对阶丢位以后还要判断舍入

最小反例·不溢出仍可不精确

最小反例：binary32 能精确存 2 的 24 次方，却不能精确存它加 1。指数远未到上限，有效位已经不足。浮点溢出与舍入损失要分别检查，答案有限并不保证与数学精确值相等。

两个浮点数只有指数相同，才能直接把有效数相加。通常先把较小指数的有效数右移，使指数与较大者一致；这叫对阶。大阶向小阶对齐会要求更多高位和更大的寄存器，不符合固定精度运算的通常实现。

对阶之后进行尾数运算，再规格化和舍入。加法可能得到 $10.xxx_2$，须右移 1 位、阶码加 1；异号相加发生抵消，可能得到 $0.00\cdots1xxx_2$，须左移、阶码减少。舍入使尾数加 1 时还可能再次进位，必须再检查规格化和阶码范围。若得到精确的 0，也不能继续无止境地左移寻找前导 1。

用一个仅保留 4 位有效数的教学格式说明舍入。$x = 1.000_2 \times 2^0$，$y = 1.000_2 \times 2^{-4}$，精确和为 1.0001_2，恰好位于保留为 `1.000` 和 `1.001` 的中点。采用 “舍入到最近，正中取偶数”，保留最低位为 0 的 `1.000`。若 x 改为 `1.001`，精确和为 `1.0011`，则正中取偶数进为 `1.010`。偶数指保留有效数的最低位为 0，不是判断整个真值是奇整数还是偶整数。

实际实现常保留保护位、舍入位和粘滞位。粘滞位记录更低位中是否出现过 1，因此 `1000`…与 `1000`…`1` 的舍入结论可能不同，不能只保留移出的第一位。

2.8.1 例 2-9　哪些步骤可能造成指数越界

例 2.8.1 (配套例题)

【配套例题】下列有关浮点数加减运算的叙述中, 正确的是（　）。

I. 对阶操作不会引起阶码上溢或下溢
II. 右规和尾数舍入都可能引起阶码上溢
III. 左规时可能引起阶码下溢
IV. 尾数溢出时，结果不一定溢出

A. 仅 II、III　B. 仅 I、II、IV　C. 仅 I、III、IV　D. I、II、III、IV

解析。对阶把较小阶提高到较大阶，最终阶码仍是原有合法阶码之一；右移尾数可能损失精度，但不使该阶码越界，I 正确。右规和舍入后的再右规会使指数加 1，可能越过上界，II 正确。抵消后的左规要减小指数，可能进入非规格化区或触及题设的下溢界，III 正确。尾数暂时形成 `10.xxx` 还可以右规，只要指数容纳得下，结果仍可表示，IV 正确。选 **D**。尾数的暂时进位与最终浮点范围溢出是不同层次。

采用 IEEE 754 的渐进下溢时，指数过小并不立即把所有结果变成 0，还可能形成非规格化数。题目若采用 “阶码、尾数均为补码” 的教学格式，就应按其位宽、小数点位置、双符号位和舍入方法计算，不能把该格式的尾数补码接到 IEEE 字段中。

浮点加法不满足数学实数的结合律。例如在 binary32 中，$a = 2^{25}, b = -2^{25}, c = 1$：$(a + b) + c = 1$，而 $a + (b + c)$ 中 b+c 可能先舍入回 b，结果为 0。发生差异的原因是中间舍入，不是加法指令随意改变数学规则。

2.8.2 定点整数与浮点数的范围、精度怎样比较

32 位补码整数把 31 位用于正数的数值范围，范围内整数连续、没有空缺；binary32 把位分给符号、指数和有效数，扩大数值范围，同时只保留 24 位有效二进制数字。浮点数在某个数量级的间隔随指数增大，因

而 “32 位” 相同不表示能精确保存同一批整数。

例如 2^{24} 可被 binary32 精确表示，但 $2^{24}+1$ 正好落在相邻两个可表示数之间；按最近偶数舍入回 2^{24}。而 2^{25} 仍可精确表示。问 “所有不超过某界的整数都精确” 与问 “某个大整数是否精确”，必须分别作答。整数溢出检查编码范围；浮点溢出检查舍入、规格化后的指数；尚未溢出的浮点结果也可能已经不精确。

课程连接·编程中的循环边界

无符号 n=0 时 n−1 绕到最大值，会改变循环终止；有符号溢出又涉及语言规则。把循环、编码范围、浮点精度三步分开检查，可以说明同一算法在不同类型中为何产生不同结果。

2.8.3 例 2-10　同一求和程序中的循环边界、整数范围与浮点精度

例 2.8.2 (配套例题)

【配套例题】已知 $f(n)=\sum_{i=0}^{n}2^i=2^{n+1}-1=\underbrace{11\cdots1}_{n+1\text{位}}{}_2$，计算 $f(n)$ 的 C 语言函数 f1 如下。

```c
int f1(unsigned n) {
    int sum = 1, power = 1;
    for (unsigned i = 0; i <= n - 1; i++) {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

将 f1 中的 int 都改为 float，可得到计算 $f(n)$ 的另一个函数 f2。假设 unsigned 和 int 型数据都占 32 位，float 采用 IEEE 754 单精度标准。请回答下列问题。

（1）当 n=0 时，f1 会出现死循环，为什么？若将 f1 中的变量 i 和 n 都定义为 int 型，则 f1 是否还会出现死循环？为什么？

（2）f1(23) 和 f2(23) 的返回值是否相等？机器数各是什么（用十六进制表示）？

（3）f1(24) 和 f2(24) 的返回值分别为 33 554 431 和 33 554 432.0，为什么不相等？

（4）$f(31)=2^{32}-1$，而 f1(31) 的返回值却为 -1，为什么？若使 f1(n) 的返回值与 f(n) 相等，则最大的 n 是多少？

（5）f2(127) 的机器数为 7F80 0000H，对应的值是什么？若使 f2(n) 的结果不溢出，则最大的 n 是多少？若使 f2(n) 的结果精确（无舍入），则最大的 n 是多少？

解析。本题把同一数列放进不同表示中：先查循环是否结束，再查运算范围，最后查浮点有效位。以下对 f1(31) 的既定结果和 n=0 时持续循环的轨迹，都按 32 位补码机器执行模型解释；标准 C 中有符号溢出属于未定义行为，n=0 时循环体也可能先发生这种溢出，不能把本题机器推演当成可移植 C 保证。

（1）i 和 n 均为 unsigned，n=0 时，`n-1` 按模 2^{32} 运算成为 $2^{32}-1$。任何 32 位无符号 i 都满足 `i<=n-1`，i 达到最大值后再加 1 又回到 0，因此循环条件一直为真。若两者都改为 int，n−1 为 -1，第一次测试 $0\le -1$ 就为假，循环一次也不执行，不会因这个边界形成死循环。

（2）$f(23)=2^{24}-1=16\,777\,215$，是 24 个连续的二进制 1。它在 32 位 int 范围内，且 binary32 恰有 24 位有效精度，故两者数值相等。

f1(23) 的补码机器数为 `00FF FFFFH`。浮点形式为 $1.\underbrace{11\cdots1}_{23位}\times2^{23}$，s=0、E=23+127=150、F 全 1，故 f2(23) 的机器数为 `4B7F FFFFH`。相同数值不要求机器编码相同。

不变量·字节序和对齐分两步

先依成员类型与对齐规则求对象布局，再把每个值拆成字节按大小端放入。字节序不改变结构体填充，也不反转数组元素次序。两步分别核对后，才能回答某地址保存的具体字节。

（3）$f(24)=2^{25}-1$ 需要 25 位有效数字，int 仍能精确保存，binary32 却只能保留 24 位。最后一次加法的精确结果为 33 554 431，处于 33 554 430 与 33 554 432 之间的正中；最近偶数舍入选后者，得到 2^{25}，机器数 `4C00 0000H`。这里尚未浮点溢出，只丢失了低位精度。

（4）$2^{32}-1$ 超过 32 位补码 int 最大值 $2^{31}-1$。按题设保留的低 32 位全为 1，解释为补码 -1。要求所有返回值与数学数列一致，须 $2^{n+1}-1\le 2^{31}-1$，所以最大 n=30；不能因为无符号 32 位能表示 $2^{32}-1$，便认为 int 也能表示。

（5）`7F80 0000H` 的阶码全 1、尾数全 0、符号 0，表示正无穷。n=126 时，精确数 $2^{127}-1$ 被舍入为 2^{127}，仍是有限数；n=127 时，结果越过最大有限数并舍入为正无穷。因此不溢出的最大 n=126。要完全精确，数列中 n+1 个连续 1 必须全部容纳在 24 位有效数中，最大 n=23。两个上界分别来自指数范围与有效精度，不能互换。

2.9 大端、小端和对齐解决不同问题

多字节数据的大端方式把最高有效字节放在最低地址，小端方式把最低有效字节放在最低地址。字节序只改变一个多字节对象内部的字节排列，不反转每个字节内的位，也不改变数组元素的顺序。

32 位值 `12345678H` 从低地址到高地址，大端为 12、34、56、78，小端为 78、56、34、12。讨论 “高位字节” 时指数字意义上的高位，讨论 “高地址” 时指存储位置，两个维度应分开。

对齐指对象的起始地址满足规定的倍数要求。常用的自然对齐模型中，2 B 对象的起始地址是 2 的倍数，4 B 对象是 4 的倍数。编译器还可能为结构体在成员之间和末尾加入填充，使各成员地址和相邻数组元素都满足对齐规则。具体要求属于 ABI/编译器约定；题目给出 “按边界对齐” 时，才按给定类型大小计算。

2.9.1 例 2-11　先确定结构布局，再看低字节

例 2.9.1 (配套例题)

【配套例题】某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

```
struct {
    int a;
    char b;
    short c;
} record;
record.a = 273;
```

若 record 变量的首地址为 0xC008，则地址 0xC008 中的内容及 record.c 的地址分别为（　）。

A. 0x00，0xC00D　　B. 0x00，0xC00E

C. 0x11，0xC00D　　D. 0x11，0xC00E

解析。a 占偏移 0 至 3，b 占偏移 4。下一地址偏移 5 不是 2 的倍数，须填充 1 B，c 占偏移 6、7，故 c 地址为 C00EH。$273 = \texttt{00000111H}$，小端最低地址保存低字节 11H，选 **D**。填充来自对齐，低字节 11 来自字节序，两步不能互相代替。

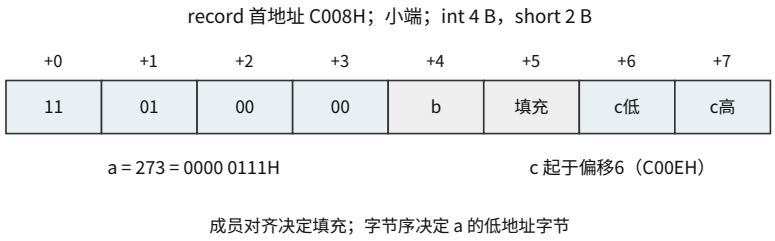


Figure 2.2: 结构体成员、填充与低地址字节分别标明

二维数组按行优先存放时，`a[i][j]` 的元素偏移为 $i \times$ 列数 $+ j$，字节偏移还要乘元素字节数。对于取负值的 short，先写出完整的 16 位补码，再按字节序拆开。例如 -6 为 `FFFAH`，小端低地址是 FA、下一字节是 FF，不是把真值“−6”拆成十进制字符。

不变量·相邻间隔随指数变大

正规格化 binary32 在同一指数区间内间隔是 2 的 e−23 次方。指数每增 1，相邻间隔也翻倍。先算间隔再判某整数是否可表示，比只记 “24 位精度” 更容易处理大整数与舍入中点。

2.10 把位串、真值和条件分开核算

表示与运算题的统一路径

先写位宽与编码，再求数学真值或完整精确结果；按题设完成截断、扩展或舍入，最后解释结果位串并检查标志。浮点解码先识别阶码的全 0/全 1，普通数公式不适用于特殊编码。涉及 C 代码时再核语言语义，不能把某种硬件结果提升为标准 C 保证。

2.10.1 补码为什么恰好多一个负数

由位权得到范围

n 位补码的值是 $-b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$。最小时只有符号位为 1，值为 -2^{n-1}；最大时符号位 0 而其余全 1，值为 $\sum_{i=0}^{n-2} 2^i = 2^{n-1} - 1$。这既说明负数多一个，也说明最小负数的相反数无法放回同宽补码中。

□

2.10.2 精度与范围各问什么

binary32 在 $[2^e, 2^{e+1})$ 内，相邻正规格化数相差 2^{e-23}。原因是尾数字段每增加 1，有效数增加 2^{-23}，再乘指数倍率 2^e。在 $[2^{24}, 2^{25})$ 间距为 2，所以 $2^{24}+1$ 无法精确保存；2^{25} 本身却是精确的二进制幂。有限数是否溢出取决于指数边界，能否精确取决于有效位，两者不可互代。

最近偶数舍入先比较被舍弃部分与半个间隔：小于一半向较近值，大于一半向另一侧；恰好一半时选保留末位为 0 的候选。只写 “四舍五入” 会漏掉二进制等距点规则。

2.10.3 对齐和容量的单位

课程连接·单位衔接存储系统

数据宽度、地址单位和块大小会在下一章共同决定偏移位数与容量。按字节编址时 4B 整数跨 4 个地址，但同一字节内 8 个位并不产生 8 个独立字节地址。每个对数前都先写清以字节还是以字计数。

机器字长是 CPU 一次处理整数的基本宽度，存储字长是一个存储单元保存的位数，指令字长是某条指令的编码位数；三者可以不同。按字节编址时，32 位整数通常占 4 个连续地址；按 32 位字编址时，同一个字只占一个地址。先把地址单位写出来，再计算数组偏移、总容量和跨界访问次数。

自然对齐要求对象起点满足某个倍数，但具体倍数来自题设或 ABI。结构体末尾填充使数组中下一对象也满足对齐；删除末尾填充可能让第一个对象内部正确而第二个对象起点出错。字节序则只回答多字节值的高低有效字节如何落到地址，不能决定填充量。

2.11 数字逻辑：从真值表到运算部件

逻辑运算把 0、1 解释为假、真。与运算只有输入全为 1 才输出 1；或运算只要有输入为 1 就输出 1；非运算反转一个输入；异或在两个输入不同时输出 1，同或在两个输入相同时输出 1。逻辑式中的 “+” 表示或，“乘积” 表示与，不能直接当作整数加乘。

A, B	与 AB	或 $A+B$	异或 $A \oplus B$
0,0	0	0	0
0,1	0	1	1
1,0	0	1	1
1,1	1	1	0

非、与、或的通常优先级依次降低；括号和非号覆盖范围优先明确。德摩根律为 $\overline{A+B}=\bar{A}\bar{B}$、$\overline{AB}=\bar{A}+\bar{B}$。异或满足交换律和结合律，多个位异或的结果为 1，当且仅当其中 1 的个数为奇数；这既可生成一位二进制加法的本位和，也可构造奇偶校验。

2.11.1 选择器和三态门怎样控制通路

k 选 1 多路选择器 MUX 有 k 组数据输入、一组输出和选择信号，至少需要 $\lceil \log_2 k \rceil$ 位选择编码。2 选 1 且选择位为 s 时，输出可写为 $Y=\bar{s}D_0+sD_1$。若数据总线有 32 位，应把它理解为 32 路对应位一起被选择。

三态门除 0、1 外，还有高阻态。使能时输出跟随输入；关闭时输出端呈高阻，对公共总线不施加有效驱动。高阻态不是数值 0。一条内部总线可连接多个输出端，但同一时刻必须按控制约束选择驱动源；多个源同时输出相反电平会发生总线争用。

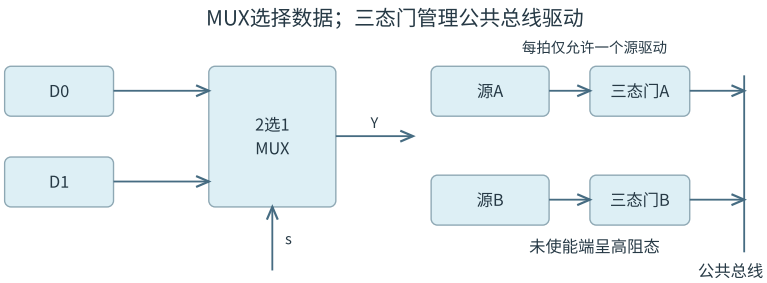


Figure 2.3: 选择器选择数据源，三态门决定输出端是否驱动公共线

2.11.2 全加器、进位产生与 ALU 选择

课程连接·乘法轮次

每轮先按寄存器低位或相邻位决定加减操作，再按约定整体移位，特别关注符号扩展。

一位全加器输入 A_i, B_i, C_{i-1}，输出本位和与进位：

$$S_i = A_i \oplus B_i \oplus C_{i-1}, \qquad C_i = A_iB_i + (A_i \oplus B_i)C_{i-1}.$$

令 $G_i = A_iB_i$ 为进位产生信号，$P_i = A_i \oplus B_i$ 为进位传播信号，则 $C_i = G_i + P_iC_{i-1}$。继续代入可以写出 $C_2 = G_2 + P_2G_1 + P_2P_1C_0$。串行进位加法器逐级等待前一级进位；先行进位电路通过组合逻辑提前形成进位，以增加电路复杂度换取速度。

“并行加法器” 可指多个输入位同时送入，并不保证进位也同时生成。ALU 则在加法器和逻辑电路基础上，由操作控制位选择加、减、与、或、移位等功能；数据位宽与操作选择编码位宽是不同的量。

2.12 乘法：按寄存器状态逐轮核算

2.12.1 无符号乘法：1101×1011

设 $n = 4$，被乘数寄存器 $X = 1101$，初始部分积 $P = 0000$、乘数 $Y = 1011$、进位 $C = 0$。每轮先根据 Y 最低位决定是否把 X 加到 P，保留最高进位，再把 (C, P, Y) 整体逻辑右移，重复 4 轮。

轮次	Y 最低位与加法结果 C, P	整体右移后的 P, Y
初始	—	0000,1011
1	1；0,1101	0110,1101
2	1；1,0011	1001,1110
3	0；0,1001	0100,1111
4	1；1,0001	1000,1111

最终 $(P, Y) = 10001111_2 = 143 = 13 \times 11$。完整乘积需要 $2n$ 位；若结果字段仅保留低 n 位，则高 n 位非零表示无符号乘积不能由 n 位完整表示。课件中的硬件标志行为应与具体指令定义对应；C 语言无符号类型运算按模 2^n 处理，而有符号溢出的语言语义另行规定。

2.12.2 Booth 补码一位乘法：-3×6

补码乘法让符号位参加运算。令 $X = 1101$ 表示 -3，初始 $P = 0000$、$Y = 0110$、辅助位 $q_{-1} = 0$。观察 (Y_0, q_{-1})：00、11 执行 $+0$，01 执行 $+X$，10 执行 $-X$。随后把 (P, Y, q_{-1}) 整体算术右移，保留最左符号位。

轮次	观察位与操作	算术右移后的 P, Y, q_{-1}
初始	—	0000,0110,0
1	00，$+0$	0000,0011,0
2	10，$-X$	0001,1001,1
3	11，$+0$	0000,1100,1
4	01，$+X$	1110,1110,0

乘积为 $11101110_2 = -18$。若只保留 4 位，则低 4 位 1110 表示 -2，与真实乘积不同。对 $2n$ 位补码乘积保留低 n 位，未溢出的条件是高 n 位均为低 n 位符号位的扩展，即最高 $n+1$ 位全部相同。处理最小负数等边界时，需要足够的内部符号扩展，不能在过窄的临时寄存器中直接对最小负数求正值。

2.12.3 硬件实现的三种组织

逐位乘法用同一加法器多次完成加法、移位，硬件少而周期多；阵列乘法器用更多组合电路并行形成部分积并累加，延迟取决于电路路径；流水线乘法器在组合逻辑中插入阶段寄存器，允许多组乘法交叠。流水线主要提高持续输入时的吞吐率，单次乘法仍要经历完整延迟。

课程连接·商余数约束

先写被除数、除数、余数和商的位宽与符号规则；逐轮试商后检验被除数等于商乘除数加余数。

2.13 除法：先明确被除数、商和余数的位宽

无符号双精度除法通常以 (R,Q) 保存 $2n$ 位被除数，除数寄存器保存 n 位除数，结束后 Q 保存商、R 保存余数。单精度被除数先零扩展，带符号被除数则按约定作符号扩展。除数为 0 属于除法异常；商还必须落在目标字段的表示范围。

恢复余数法可理解为：移入下一被除数位，试减除数；若结果非负，上商 1；若为负，加回除数恢复余数，上商 0。不恢复余数法则利用当前余数符号决定下一轮加或减，减少“立即恢复”动作，但最终须按算法规则校正余数。

以 $1111_2 \div 0010_2$，即 $15 \div 2$ 为例，下面用逐位长除法呈现同一数学过程：

处理的被除数位	移入后的中间值	本位商、余数
第 1 位 1	1	0，1
第 2 位 1	$2\times1+1=3$	1，1
第 3 位 1	3	1，1
第 4 位 1	3	1，1

得到商 0111、余数 0001，满足 $15=7\times2+1$ 且 $0\le1<2$。表中的长除法是数学核算模型，和硬件 (R,Q) 整体移位时的寄存器切分应分别标明。

2.13.1 双精度除法的商溢出与寄存器轨迹

无符号 $2n$ 位被除数先分装到 R,Q，各 n 位，Y 保存 n 位除数。$Y>0$ 时，若初始高半部 $R\ge Y$，则商至少为 2^n，不能装入 n 位 Q；因此正式迭代前可先作商溢出检查。课件中的 $255/15=17$ 超出 4 位商范围，而 $200/15=13$ 余 5 可以表示。单精度无符号被除数先零扩展，高半部为 0；除数非零时，商不会超出同宽无符号范围。

对 $15/2$，初态为 $R=0000,Q=1111,Y=0010$。每轮将 RQ 整体左移，使下一被除数位进入 R，再试算 $R-Y$：非负时保留差并把 Q 最低位置 1；负时恢复原 R 并置 0。循环 4 轮后，R 为余数、Q 为商。

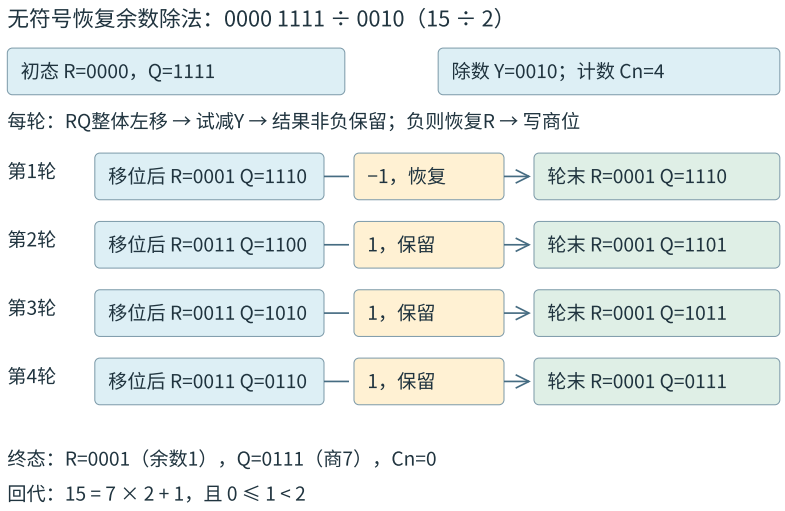


Figure 2.4: 王道课件 15 除以 2 的四轮寄存器状态，包含试减、恢复与商位写入

课程连接·商余数约束

先写被除数、除数、余数和商的位宽与符号规则；逐轮试商后检验被除数等于商乘除数加余数。

“试减为负” 是临时结果，并不表示被除数变为带符号负数。电路须保存用于试算符号或借位判断的状态；把负的试算低位直接当作最终无符号余数，会破坏 $0 \le R < Y$。

2.13.2 补码除法中的符号扩展和商修正

王道课件以 4 位补码 $-7/3$ 说明有符号除法。被除数 1001 先符号扩展为 8 位，初态 $R = 1111, Q = 1001$；除数 $Y = 0011$。下面列出这一实例的状态，R 栏按 4 位补码解读。被除数与正除数异号，试算为 $R + Y$，本例试算结果若改变余数原来的负号就恢复，否则保留并生成商的绝对值位。

轮次	移位后的 R、Q	试加后的 R	轮末 R、Q
1	1111、0010	0010，符号改变	恢复为 1111、0010
2	1110、0100	0001，符号改变	恢复为 1110、0100
3	1100、1000	1111，保留	1111、1001
4	1111、0010	0010，符号改变	恢复为 1111、0010

四轮后 $Q = 0010$ 是商的绝对值 2。因原操作数异号，再对 Q 取补得到 1110，即商 -2；$R = 1111$ 即余数 -1，满足 $-7 = (-2) \times 3 + (-1)$。课件的另外两例为 $6/3 = 2$ 余 0，以及 $3/(-7) = 0$ 余 3。表中没有出现试算恰为 0 的情形；一般电路还需单独正确处理整除，不能把 “结果符号位变 0” 机械地判为试算失败。

也可先在足够宽的内部表示中求两数绝对值，按无符号规则求商余数，最后恢复商符号和余数符号；这是一种便于核算的等价模型。最小负数的绝对值需要保护位，不能直接在原来的 n 位补码寄存器中取正值。单精度补码除法还须预先排除除数 0 及最小负数除以 -1 两类异常。

带符号整数除法中，商符号由两操作数符号异或决定；按 C 语言向 0 截断的约定，非零余数与被除数同号，始终满足 $X = QY + R$。例如 $-15/2 = -7$、余数 -1；$15/(-2) = -7$、余数 1。对 n 位补码，最小负数除以 -1 的商超出可表示范围，应单独判断。

2.14 浮点数舍入的临界情形

课程连接·舍入看被舍部分

最近偶数舍入的正中间情形，要检查保留结果末位的奇偶；规格化溢出后还要重新检查阶码。

IEEE 754 单精度规格化数含 24 位有效二进制精度。在 $[2^{24}, 2^{25})$ 范围内，相邻可表示数的间距为 2；因此 $2^{24}+1$ 恰好处于两个可表示数的中点。

最近偶数舍入的步骤是：先比较被舍部分与半个最低有效位；小于半单位截断，大于半单位进 1；恰好半单位时，选择保留部分末位为 0 的结果。保留保护位、舍入位、粘滞位，可判断被舍部分的大小；粘滞位是更低所有被舍位的或。

加法	精确结果相对相邻浮点数的位置	单精度最近偶数结果
$2^{24}+0.5$	距 2^{24} 更近	16777216
$2^{24}+1$	正好居中，低端末位偶	16777216
$2^{24}+1.5$	距 $2^{24}+2$ 更近	16777218
$(2^{24}+2)+1$	正好居中，高端末位偶	16777220

对阶导致移位不等于立刻丢弃全部低位；舍入可能再次产生尾数进位，需要右规并使指数增加。判断上溢应在规格化、舍入后检查指数；接近 0 的结果还要根据表示格式区分次正规数和下溢处理。

2.14.1 计算与辨析自查

1. 高阻态为什么不能当成数据 0？ 2. n 位补码乘积保留低 n 位时怎样判断溢出？ 3. $-15 = Q \times 2 + R$ 按向 0 截断时 Q, R 分别是多少？ 4. 单精度数 16777218 加 1 为什么成为 16777220？

答案。1. 高阻表示输出端不主动驱动总线，而 0 是主动输出低电平。2. 检查完整 $2n$ 位结果是否为低 n 位的符号扩展。3. $Q=-7, R=-1$。4. 精确结果居中，最近偶数舍入选择有效尾数末位为 0 的高端数。

2.15 主动回忆自测

2.15.1 配套自测

1. 原创补充：8 位机器计算 `7FH+01H`，按本章标志约定求低 8 位结果、CF、OF、SF、ZF。
2. 原创补充：binary32 位串 `0000 0001H` 与 `0080 0000H` 各表示什么正数？两者是否都具有隐含前导 1？
3. 【配套例题】已知 32 位 int 变量 i=12345，float 变量 $f = 1.2345 \times 2^3$。下列结果为假的是：A. `i==(int)(double)i`；B. `f==(float)(double)f`；C. `i==(int)(float)i`；D. `f==(float)(int)f`。

答案。1. 结果 `80H`，CF=0、OF=1、SF=1、ZF=0；127+1 越出补码范围，但未越出 8 位无符号范围。2. 分别为 2^{-149} 与 2^{-126}，前者非规格化、隐含 0，后者规格化、隐含 1。3. D。12345 能被 binary32 精确表示；float 到 double 再回来也保留原 float 值，但 f 截断为整数会丢小数部分。

2.15.2 遮住答案复述知识关系

课程连接·从条件出发

回答转换题时先写位宽、编码及符号，再进行位操作和真值计算。

主动回忆·自拟

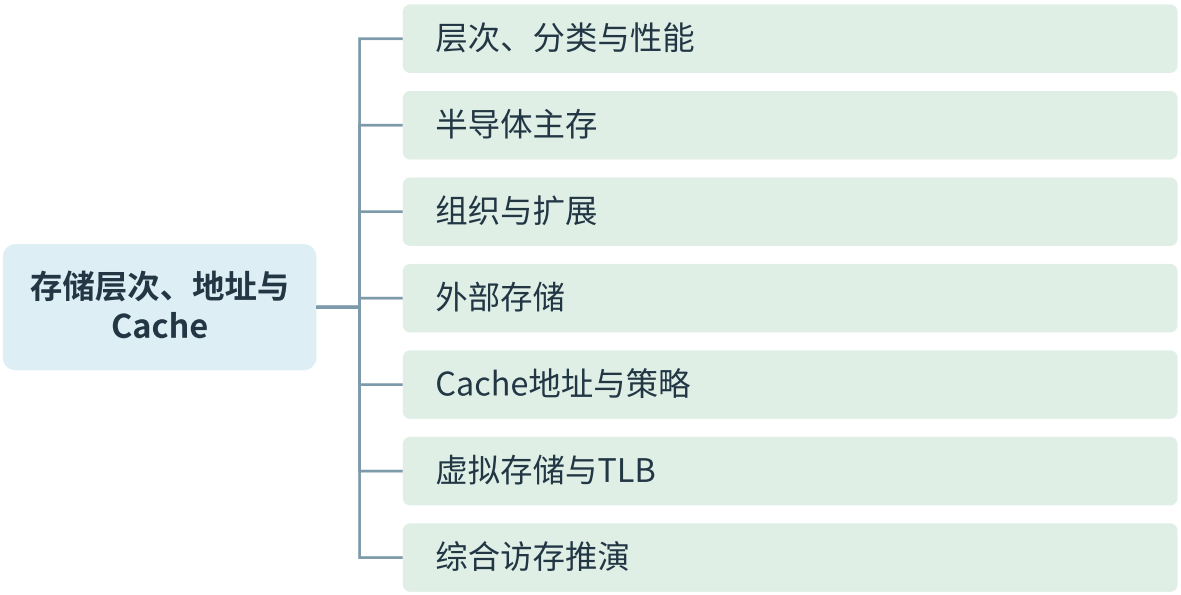
4. 配套结构体案例：某 32 位计算机按字节编址、小端、int 为 4B，按自然边界对齐。结构为 `struct record { int id; char name[10]; int salary; } employee[200];`，数组首址 A0B0H。若 employee[1].id 为 12345678H，其中 56H 的地址是多少?
5. 自拟：8 位补码最小负数取反加 1 得到自身，为什么不能据此说数学相反数等于自身?
6. 自拟：解释 8 位补码 -3 算术右移 1 位和 C 整数 $-3/2$ 为何可能不同。
7. 自拟：binary32 在 $[2^{25}, 2^{26})$ 的相邻间隔是多少？$2^{25}+1$ 按最近偶数舍入到哪里?
8. 自拟：列出恢复余数和不恢复余数法都必须满足的最终商余数关系。
9. 自拟：12345678H 采用大端和小端时，最低地址分别保存哪一字节？字节中的位需要反转吗?

2.15.3 参考答案

4. A0C5H。结构大小 20B，第二元素首址 A0C4H，小端 56H 位于其后 1B。
5. 同宽运算按模 256 保存位串；+128 不在 8 位补码范围，不能把位串绕回当作数学等式。
6. 算术右移复制符号位，-3 变 -2，相当于向负无穷取整；C 整数除法向零截断得 -1。还须核语言版本对负数右移的规定。
7. 间隔 4，$2^{25}+1$ 离 2^{25} 更近，舍入到 2^{25}。
8. 无符号被除数 $N = QD + R$，$D > 0$ 时 $0 \le R < D$；不恢复法可能需最终负余数校正。有符号还需给定商、余数符号及舍入约定。
9. 大端 12H、小端 78H；不反转字节内部位。

存储层次、地址与 Cache 3

章节脉络



本章围绕“层次、分类与性能 → 半导体主存 → 组织与扩展 → 外部存储 → Cache 地址与策略 → 虚拟存储与 TLB → 综合访存推演”展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

课程连接·分层连接操作系统

Cache 利用短期访问局部性，虚拟存储还承担地址空间、隔离和换页。两者都能体现层次思想，但缓存对象与缺失处理不同：Cache 常缓存数据块，TLB 缓存映射，页表记录虚页到页框关系。

3.1 为什么一个程序同时使用几层存储器

定义 3.1.1 (概念与适用对象)

寄存器、Cache、主存和外存的速度、容量及单位成本不同。寄存器紧贴运算部件、容量小；Cache 保存近期可能使用的主存块；主存容纳正在运行的程序和数据；外存提供较大且通常非易失的空间。层次化存储利用程序访问的局部性，使大量访问落在较快的层中，同时保留较大容量。

随机存取指访问时间基本不随单元位置改变，不等于“随机挑一个位置”。顺序存取要按顺序经过前面的内容；直接存取先定位一个区域，再在区域内访问，例如磁盘。相联存储器按内容匹配，TLB 的相联查找就是常见用途。

存取时间是完成一次读或写所需的时间；存取周期是连续两次访问之间允许的最短间隔。DRAM 读后需要恢复等操作，存取周期可能大于单纯读出延迟。计算连续访问的总时间时，不能只拿一次存取时间乘访问次数。

容量要写成“单元数 × 每单元位数”，再结合编址单位换算。2^{20} 个 8 位单元为 1 MiB；本讲义在芯片容量、地址空间、Cache 中使用二进制单位，主频和传输率中的 M、G 按十进制。题源沿用 KB、MB 时，根据地址幂次说明实际口径。

3.2 SRAM、DRAM 与 Flash 保存信息的方式

最小反例·动态不等于访问变化

最小反例：一个程序只反复读同一 DRAM 单元，芯片仍需刷新。动态描述电荷保持机制，而非程序写入频度；随机描述按地址选取能力，也不意味着无需刷新。把这两条分类轴分开。

SRAM 用稳定电路状态保存位，只要供电就不需周期刷新，通常速度快、集成密度相对低，适合 Cache。DRAM 用电容电荷表示位，电荷会泄漏，需要刷新；读出还伴随感知、放大和恢复。它的高密度使其适合主存。

“动态” 说的是保存机制，不是程序是否经常改写数据；“随机” 说的是访问方式，不是是否需要刷新。SRAM、DRAM 都是易失性存储器，掉电后不能保持原信息。

Flash 是非易失性半导体存储器。读、编程和擦除的粒度与代价不同，通常不能像 SRAM 那样直接覆盖写入任意旧位；例如 NAND Flash 常按页读写、按块擦除。SSD 控制器借助映射、垃圾回收和磨损均衡，把复杂的物理操作组织成主机看到的逻辑块访问。掉电保持不等于写入次数无限，也不等于写延迟与读延迟相同。

比较存储器时，要固定同一维度。SRAM 与 DRAM 在“能否随机访问”上相同，在“怎样保持一位、是否刷新、速度与密度”上不同；DRAM 与 Flash 在“是否掉电保持”上不同；Cache 与主存则首先是系统中的层次角色，常分别由 SRAM 和 DRAM 实现，而角色的划分与器件技术的划分不是一回事。芯片题再把器件组织写成单元数 $\times$ 位宽：一片 $4\text{M}\times 8$ 的容量是 4 MiB，不能因它用于主存，就把 “地址线、数据线、存储单元数” 当成同一个数。

3.3 地址引脚不总等于地址位数

SRAM 的 $2^k \times w$ 表示有 2^k 个可选单元，每次选中输出 w 位，通常需 k 根地址线和 w 根数据线，另有片选和读写等控制线。DRAM 常将行地址、列地址分时送入同一组地址引脚，由行、列选通信号分别锁存，因此逻辑地址位数不能直接作为地址引脚数。

3.3.1 例 3-1　DRAM 的复用地址怎样计数

例 3.3.1 (配套例题)

【配套例题】某容量为 256 MB 的存储器由若干 $4\text{M}\times 8$ 位的 DRAM 芯片构成，该 DRAM 芯片的地址引脚和数据引脚总数是（　）。

A. 19　B. 22　C. 30　D. 36

解析。$4\text{M}=2^{22}$，芯片内部需要 22 位单元地址。按题目采用的行列各 11 位、地址复用模型，需 11 根地址引脚；数据宽 8 位，需要 8 根数据引脚，共 19 根，选 **A**。题目给出的整机 256 MB 并不改变单片引脚数。若问组成整机需要多少片，则另算 $256/4=64$ 片。

实际 DRAM 还可能含多个 Bank，地址、命令及引脚结构要按题目给出的组织判断。刷新按行进行，不是每保存一个位就分别发一次刷新。集中刷新把若干刷新操作连在一起，期间可能有一段较长的时间不能正常访问；分散刷新把一次正常存取和一次刷新组合到扩大的存取周期内，不形成集中刷新的长间隙，但会降低正常访存速率。异步刷新在保持时间内把各行刷新均匀分散，定期插入短的刷新周期，其间仍可进行正常访问。若有 R 行、保持时间为 T_r，把每行各刷新一次并均匀分布，平均间隔为 T_r/R；还须检查每次刷新占用多久。

3.3.2 MAR 的位数由地址空间决定

最小反例·已安装容量不决定 MAR

最小反例：机器给出 64MiB 字节地址空间，只安装 32MiB 内存。MAR 仍需容纳 26 位地址，而非按已安装容量只取 25 位。芯片单元地址、可编址空间、已安装容量是三个问题。

MAR 需容纳可由该机器给出的主存地址。它与 MDR 的位数没有 “必须相等” 的关系。MDR 反映某次数据传送的位宽，MAR 反映地址编码位宽。

3.3.3 例 3-2　已安装容量不是全部可编址空间

例 3.3.2 (配套例题)

【配套例题】某计算机存储器按字节编址，主存地址空间大小为 64 MB，现用 $4\text{M}\times 8$ 位的 RAM 芯片组成 32 MB 的主存储器，则存储器地址寄存器 MAR 的位数至少是（　）。

A. 22 位　B. 23 位　C. 25 位　D. 26 位

解析。题目问覆盖 64 MB 地址空间需要多少位，而不是按已安装的 32 MB 去算。$64\text{MB}=2^{26}\text{B}$，每个字节一个地址，所以 MAR 至少 26 位，选 **D**。芯片地址位数 22、已安装容量地址位数 25 都不能回答这一问题。

3.4 单体多字和多体交叉怎样提高带宽

单体多字存储器扩宽一次读写的数据量，一个存储单元包含多个机器字。顺序使用相邻字时，一次访问可以提供多个字；若频繁跳转，取来的其他字未必有用。它主要改善带宽，不能仅凭 “一次取多个字” 就当成解决地址空间不足的方案。

多体存储器则包含能相对独立工作的存储模块。高位交叉（顺序编址）用地址高位选模块，各模块保存连续的一段地址；低位交叉用低位选模块，连续字地址轮流落到不同模块。对 m 个模块且按字编址的常规模型：

$$\text{模块号}=\text{字地址 mod } m,\qquad \text{模块内地址}=\left\lfloor\frac{\text{字地址}}{m}\right\rfloor.$$

若给的是字节地址，每字 w B，且各模块每次存取一字，通常应先由 $\lfloor A/w\rfloor$ 求出字地址，再据此分模块；最低的字节偏移不参与模块编号。多体交叉与单体多字可以结合，但两者不是同一种机制。

设单模块存取周期为 T，连续启动相邻模块的间隔为 r，模块数 m。连续顺序访问不冲突的条件是 $mr\ge T$。若第一字在 T 后到达，后续每隔 r 得到一个字，读 n 个字的时间为 $T+(n-1)r$。第一字延迟仍为 T，提高的是连续吞吐率。

3.4.1 例 3-3　同一模块不等于立即发生冲突

例 3.4.1 (配套例题)

【配套例题】某计算机使用 4 体交叉编址存储器，假定在存储器总线上出现的主存地址（十进制）序列为 8005、8006、8007、8008、8001、8002、8003、8004、8000，则可能发生访存冲突的地址对是（　）。

A. 8004 和 8008　B. 8002 和 8007　C. 8001 和 8008　D. 8000

和 8004

解析。先求地址模 4：1、2、3、0、1、2、3、0、0。最后两次 8004、8000 连续访问模块 0，模块可能尚未结束前次存取，所以选 **D**。8008 与 8004 也在模块 0，却隔了四个访问位置，不能只凭同体就忽略时间间隔。这里沿题目的低位交叉约定解释 “可能冲突”，具体是否冲突仍由模块周期与请求间隔决定。

不变量·位扩展共享片选

同一地址上的多片芯片分担不同数据位，因此片选和片内地址应同步；不同地址组才由高位译码区分。先画组，再连数据线，能避免把位扩展误画成容量分区。

3.5 主存芯片怎样连到 CPU

位扩展把若干芯片并联在同一地址上，各提供数据的一部分；字扩展让若干组芯片承担不同地址范围，用片选区分。组成 $M \times W$ 的存储器，单片容量 $m \times w$，且两个方向都能整除时，位方向需要 W/w 片，字方向需要 M/m 组，总片数为两者乘积。

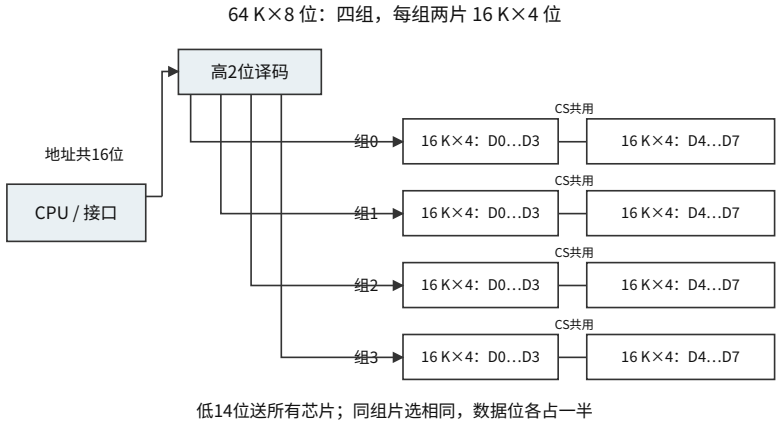


Figure 3.1: 两片并联扩位，四组译码扩字；同组选中信号相同

图中每片 16K × 4 位，同行两片并成 16K × 8 位，四组组成 64K × 8 位。低 14 位送给所有芯片选择片内单元，高 2 位经 2–4 译码器选一组。位扩展增加数据宽度，字扩展增加可选单元数，两者所用连线不同。

3.5.1 例 3-4　ROM 与 RAM 分区要分别计算

例 3.5.1 (配套例题)

【配套例题】某计算机主存容量为 64 KB，其中 ROM 区为 4 KB，其余为 RAM 区，按字节编址。现要用 2K × 8 位的 ROM 芯片和 4K × 4 位的 RAM 芯片来设计该存储器，则需要上述规格的 ROM 芯片数和 RAM 芯片数分别是（　）。

A. 1、15　　B. 2、15　　C. 1、30　　D. 2、30

解析。 ROM 单片 2 KB 且已有 8 位宽，需 $4/2 = 2$ 片。RAM 总量 60 KB；单片虽有 4 K 单元，每单元只有 4 位，要两片并成 4K × 8 位的一组，再用 $60/4 = 15$ 组，共 30 片，选 **D**。只算 60/4 会漏掉位扩展。

确定地址范围时，先确定编址单位，再拼接片选地址和片内地址。首地址由片选编码后接全 0 构成，末地址由片选编码后接全 1 构成。例如按字编址，32 K×16 位芯片的片内字地址为 15 位；若高 3 位片选为 111，则字地址范围为 `38000H` 至 `3FFFFH`。若改按字节编址，每个字占 2 B，片内字节范围需 16 位，范围随之改变，不能沿用同一答案。

地址线的编号方式同样要从题干读。通常本书令 A0 为最低位；有的题明确写 A0 为最高位，此时 4 K 片内地址使用最后 12 根低位地址线，片选应取其上方的线。下标数字不是判断高低位的可靠依据。

错项排除·平均旋转等半圈

转速 r 转每分意味着一圈 60/r 秒；均匀随机角位置的平均等待为半圈，传一扇区另按一圈中该扇区比例计算。SSD 没有这段机械旋转过程，不能给它机械套用半圈等待。

3.6 磁盘与 SSD 的一次访问花在哪些时间上

磁盘由盘面、磁道和扇区组织信息。同一半径处各盘面的磁道组成柱面；选择盘面由磁头完成，寻道把磁头移动到相应磁道。旋转等待使目标扇区转到磁头下方，随后才传输数据。磁盘地址的典型几何字段是柱面号、磁头号、扇区号；磁头号已经指定盘面，不必再把 “盘面号” 另算一个定位维度。

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}.$$

只有题设要求时再计控制器或排队开销。转速为 r 转/分时，一圈为 60/r 秒；随机到达且目标角位置均匀时，平均旋转等待为半圈。传输一个扇区的时间只占一圈的相应部分，不能把等待的半圈再算进传输时间。

3.6.1 例 3-5　7200 转每分不是 7200 转每秒

例 3.6.1 (配套例题)

【配套例题】若磁盘转速为 7200 转/分，平均寻道时间为 8 ms，每个磁道包含 1000 个扇区，则访问一个扇区的平均存取时间大约是（　）。

A. 8.1 ms　B. 12.2 ms　C. 16.3 ms　D. 20.5 ms

解析。一圈 $60/7200$ s ≈ 8.333 ms，平均旋转等待约 4.167 ms；一扇区传输约 $8.333/1000 = 0.008333$ ms。总时间约 12.175 ms，选 **B**。把一整圈当成平均等待，会得到另一数量级的选项。

SSD 没有机械寻道和旋转等待，但仍有接口传输、闪存访问、控制器与写放大等开销。因此不能照搬磁盘的平均旋转等待公式。RAID 把多个盘组织起来：条带化提升并行访问能力，镜像提供冗余，奇偶校验可支持某些盘故障后的恢复。RAID0 没有数据冗余；即便采用冗余，也不等于可以代替备份：误删的内容仍可能被同步写到冗余盘。

3.7 Cache 命中率怎样变成时间

时间局部性指近期访问过的信息可能很快再访问，例如循环累加器；空间局部性指相邻位置可能很快被访问，例如顺序扫描数组。Cache 按块调入，因此顺序数组中一次缺失可能为随后几个元素准备数据。

命中率 $h = N_{\text{hit}}/(N_{\text{hit}} + N_{\text{miss}})$。平均访问时间必须先明确 “未命中” 指的是哪一段时间。若先查 Cache 花 t_c，未命中后再用 t_m 完成取块，则

$$\text{AMAT} = t_c + (1 - h)t_m.$$

若题目给出的 “未命中访问时间” 已经包含查 Cache 的时间，记为 t_{miss}，则写为 $ht_c + (1-h)t_{\text{miss}}$。两式只是对同一事件的不同分解，第

二式中不能再额外加一次 t_c。多级 Cache 则应区别局部缺失率与相对 CPU 总访问的全局缺失率。

最小反例·循环轮数不是访问次数

最小反例：一条 a[k]=a[k]+32 每轮读一次又写一次，因此四元素八次数据访问。空 Cache 顺序扫描时一块首次读缺失，后续写命中。分母错用四轮，就把正确 12.5% 误算成 25%。

3.7.1 例 3-6　读后写的数组访问怎样计缺失率

例 3.7.1 (配套例题)

【配套例题】有如下 C 语言程序段：

```
for (k = 0; k < 1000; k++)
    a[k] = a[k] + 32;
```

若数组 a 和变量 k 均为 int 型，int 型数据占 4 B，数据 Cache 采用直接映射方式，数据区大小为 1 KB、块大小为 16 B，该程序段执行前 Cache 为空，则该程序段执行过程中访问数组 a 的 Cache 缺失率约为（　）。

A. 1.25%　　B. 2.5%　　C. 12.5%　　D. 25%

解析。每块容纳 4 个 int。按顺序扫描，每块第一次读缺失，其余 3 次读命中；第一次读已把块取入，随后对应的写也命中。每 4 个元素共 8 次数组访问、1 次缺失，因此约为 12.5%，选 **C**。若数组首地址未按块对齐，块数可能多 1，不影响本题 “约为” 的结论。不能把循环 1000 轮误当作只访问数组 1000 次。

3.7.2 一次缺失损失要不要反复付启动代价

若每次取 8 B 需发送地址 1 周期、准备 8 周期、传输 1 周期，64 B 块包含 8 次独立访问，总计 $8(1+8+1)=80$ 周期。若支持突发，只发一次首地址，等待首数据 8 周期后连续传 8 次，则共 $1+8+8=17$ 周期。是否支持突发会改变这里的计算结果，详见第 6 章；不能只看总线同为 64 位就给出同一个时间。

3.8 地址拆分决定块能放在哪里

先由数据容量 C 和块大小 B 求行数 $L=C/B$。若每组 r 行，则组数 $G=L/r$。按字节编址且 B、G 为 2 的幂时，块内偏移为 $b=\log_2 B$ 位，组号为 $g=\log_2 G$ 位，剩下高位为标记。若 Cache 总容量已经包含标记等附加位，这个总量就不能再当作 C。

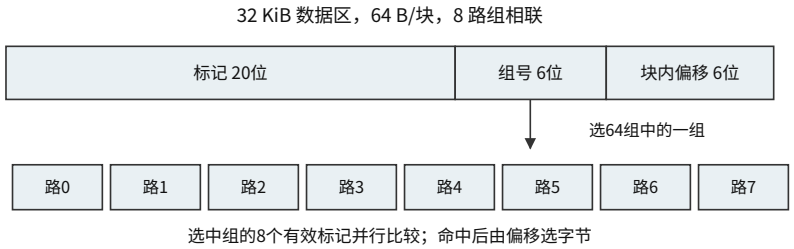


Figure 3.2: 先去块内偏移，再用块号确定组和标记

常规低位选组的规则为：主存块号 $q=\lfloor A/B \rfloor$，组号 $q \bmod G$，标

记 $\lfloor q/G \rfloor$。直接映射是 r=1，全相联是 G=1；r 路组相联中的 “路数” 是每组行数，不是组数。

在地址位数、数据容量 C 和块大小 B 都相同的条件下，三种映射可放在同一框架比较。表中 C 为数据区容量，与 B 采用相同单位；块大小和组数按 2 的幂组织，字段位数才可直接写成以 2 为底的对数。

映射	每块可放的位置	地址中的组选字段	一次查找的比较范围
直接映射	唯一一行	$\log_2(C/B)$ 位	选中行的一个标记
r 路组相联	唯一一组内的 r 行	$\log_2(C/(Br))$ 位	该组 r 个有效标记
全相联	全 Cache 任一行	0 位	全部行的有效标记

同一过程的其余状态（按首列对应）：

映射	满时怎样选牺牲行
直接映射	无行间选择
r 路组相联	仅在该组内替换
全相联	在全 Cache 内替换

最小反例·命中会改变 LRU

最小反例：两路一组访问 A、B、A、C。LRU 在第三次命中 A 后淘汰 B，FIFO 仍按装入次序淘汰 A。只记录装入顺序无法模拟 LRU，命中也必须更新最近使用状态。

提高相联度时，块内偏移不变，组选字段变短，标记相应变长；比较器和替换状态的成本也可能增加。这样做能减少某些冲突缺失，却不消除首次访问的强制缺失，也不能让有限容量容纳任意大的工作集。映射决定候选位置，替换决定满组时选谁，写策略决定何时更新下一层，应分别求解。

访问时用组号选一组，并行比较该组各行的有效标记。组号已经用于选组，不需要再保存到每行标记中；块内偏移只负责从已找到的块中选字节。

3.8.1 例 3-7　8 路组相联需要几个比较器

例 3.8.1 (配套例题)

【配套例题】若计算机主存地址为 32 位，按字节编址，某 Cache 的数据区容量为 32 KB，主存块大小为 64 B，采用 8 路组相联映射方式，该 Cache 中比较器的个数和位数分别为（　）。

A. 8，20　　B. 8，23　　C. 64，20　　D. 64，23

解析。行数 $2^{15}/2^6 = 2^9$，组数 $2^9/8 = 2^6$。偏移 6 位、组号 6 位，标记 $32-6-6 = 20$ 位。一次只在选中的一组内比较 8 个标记，因此 8 个 20 位比较器，选 **A**。组数为 64，并不表示同一次访问就要比较 64 个组。

3.9 LRU 更新的是访问次序，不只是装入次序

当目标组有空行时先用空行；满了才按替换算法决定牺牲哪一行。直接映射每个块只有一个位置，不需要在多行间选择。FIFO 记录装入次序，命中一般不改变队列；LRU 记录最近访问次序，命中也要更新；随机替换不跟踪精确最近次序。

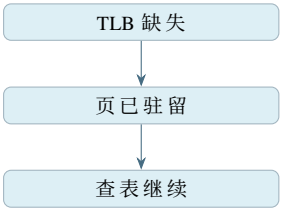
下面以两路的一个组解释 LRU 状态，顺序从最近使用到最久未使用。开始为空，访问 A 后为 [A]，访问 B 后为 [B,A]，再次访问 A 后为 [A,B]。此时访问 C，应淘汰 B，变为 [C,A]。若按 FIFO，则第三次访问 A 不改变装入队列，访问 C 会淘汰 A。这组状态足以说明两算法的分歧。

执行替换题的表格应至少记录本次地址、块号、组号、命中与否，以及该组更新后的状态。替换仅发生在对应组内，不能从其他组挑一个

“全 Cache 最久未用” 的块。若题干改用高位或特殊函数选组，先按题干给出的映射分组；LRU 本身不负责决定组号。

最小反例·映射没缓存不等于没驻留

最小反例：TLB 未命中，但页表存在位为 1。查表后即可得到页框号，无需从磁盘调页。TLB 缺失只说明映射未缓存，缺页才说明合法目标页当前不驻留。



TLB 未命中且存在位为 1，不发生缺页。

3.10 写命中与写缺失是两个独立问题

写命中时，直写同时更新 Cache 和下一层；回写只改 Cache 并置修改位，替换脏块时才写回。直写常配写缓冲，以减少等待，但缓冲满时仍可能阻塞。回写时主存内容暂时旧于 Cache，这是允许的受控状态，不能凭 “二者立即不同” 就断定设计有错。

写缺失时，写分配先调入块再更新 Cache，非写分配直接写下一层而不把该块装入 Cache。回写常配写分配、直写常配非写分配，这是常用组合，不是逻辑上仅有的两种组合。

3.10.1 例 3-8　Cache 总容量还包括哪些位

例 3.10.1 (配套例题)

【配套例题】假定主存地址为 32 位，按字节编址，主存和 Cache 之间采用直接映射方式，主存块大小为 4 个字，每字 32 位，采用回写方式，则能存放 4 K 字数据的 Cache 的总容量的位数至少是（　）。

A. 146 K　　B. 147 K　　C. 148 K　　D. 158 K

解析。一块 $4 \times 4 = 16$ B，块内偏移 4 位。4 K 字数据共 16 KB，Cache 有 $4096/4 = 1024 = 2^{10}$ 行，行号 10 位，标记 18 位。每行数据 128 位，还需有效位 1 和修改位 1，共 $128 + 18 + 1 + 1 = 148$ 位；总计 $1024 \times 148 = 148$ K 位，选 **C**。直接映射无需 LRU 替换位。本题 “至少” 按这些必要字段计算，不额外猜测 ECC 等未给出的实现。

多个 CPU 各有 Cache 时，一个核的写入还涉及其他副本怎样失效或更新；DMA 与 Cache 也可能有一致性处理。单核回写正确不等于多核副本会自动一致。协议细节超出本章基本模型，此处保留 “需要一致性机制” 的边界，不把全写法当作解决任意多副本问题的证明。

3.11 虚拟存储器怎样让程序看到地址空间

虚拟存储器把进程使用的虚拟地址映射到物理地址，允许部分内容暂存外存，也支持隔离与保护。它不是 “用硬盘代替所有内存”：CPU 最终仍需要可访问的物理存储，缺页时还要调入并等待。

操作系统建立、维护页表等映射信息，决定分配、换入换出及保护；运行时 MMU 根据这些信息实施地址转换并检查权限。把 “映射由 OS 管理” 与 “每次访存都调用 OS 软件转换” 混为一谈，会误判硬件路径。正常的 TLB 命中不需要逐次陷入内核。

3.12 页表与 TLB：地址偏移为何原样保留

页式系统把虚拟地址分为虚页号 VPN 和页内偏移 d；页表查得物理页框号 PPN 后，物理地址为 PPN $\times P + d$，P 是页大小。因为页和页框大小相同，页内位置不变。虚拟地址位数和物理地址位数可以不同，改变的是页号部分。

TLB 是页表项的高速缓存。TLB 未命中表示缓存里没有本次所需的映射，不表示页面一定不在主存；随后可以查页表并填入 TLB。对于地址合法且映射已建立的页面，存在位为 0 表示尚未驻留，按题设触发缺页异常。非法地址、无效映射或权限不足的情况须另行检查，不能一律按可调页修复处理；保护异常也可能发生在页面已经在主存时。

不变量·写回与写分配独立

直写或回写回答命中时怎样更新下一层，分配或不分配回答缺失时是否调入块。常见搭配不是逻辑上的唯一搭配。读清题设组合，再决定缺失数、主存写次数和脏块处理。

3.12.1 TLB、页表与 Cache 各自缺少什么

结构	查找所依据的信息	保存的主要对象	未命中或无效后继续做什么
TLB	虚页号；组相联时再拆组号和标记	虚页到物理页框的映射及权限等	查页表；此时还不能断定缺页
页表	虚页号，可能经过多级索引	页框号、存在位、权限及管理状态等	地址合法且未驻留时按题设缺页；非法映射与保护失败另行处理
物理地址访问的 Cache	物理块号；再拆组号和标记	主存数据块或指令块的副本	访问下一存储层，按写策略调块或写出

因此，TLB 命中仍可能 Cache 缺失；TLB 未命中也可能页面已在主存。不能把三个结构的 “有效” 都当成 “原数据已经在 Cache”。TLB 表项存的是地址映射，不是一页完整数据；Cache 行的标记也不等同于物理页框号。

按 “先转换、后查物理 Cache” 的串行教学模型，一次普通数据读可这样展开：先由虚拟地址分出 VPN 与页内偏移；用 VPN 查 TLB，未命中则走题设页表层数；取得有效页框号后拼出物理地址；再由物理地址分 Cache 标记、组号、块内偏移，决定从 Cache 还是下一层取得数据。若中间缺页，先处理并重试指令，不把尚未形成的物理地址继续送去查数据 Cache。计算次数或时间时，就沿这条路径逐项累计；页表访问是否又经过 Cache、TLB 与 Cache 能否重叠，均须服从题设。

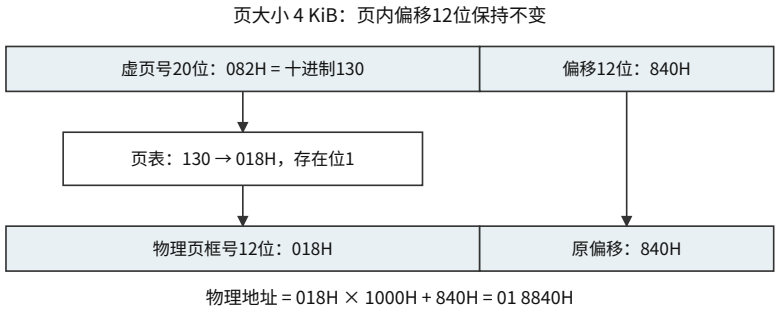


Figure 3.3: 虚拟地址先查映射，再用页框号与原偏移形成物理地址

3.12.2 例 3-9　页号的十六进制不能直接当表中十进制

例 3.12.1 (配套例题)

【配套例题】某计算机主存地址为 24 位，采用分页虚拟存储管理方式，虚拟地址空间大小为 4 GB，页大小为 4 KB，按字节编址。某进程的页表部分内容如下。

虚页号	实页号（页框号）	存在位
82	024H	0
…	…	…
129	180H	1
130	018H	1

当 CPU 访问虚拟地址 0008 2840H 时，虚—实地址转换的结果是（ ）。

A. 得到主存地址 02 4840H　B. 得到主存地址 18 0840H
C. 得到主存地址 01 8840H　D. 检测到缺页异常

解析。页大小 2^{12} B，低 12 位是偏移 `840H`，虚页号 `82H` 等于十进制 130，并非表中第一行的十进制 82。查到页框 `018H` 且存在位 1，物理地址为 `018H×1000H+840H=018840H`，选 **C**。计算前逐列辨明进制，可避免把本题误判为缺页。

3.12.3 例 3-10　TLB 标记不包含页内偏移

例 3.12.2 (配套例题)

【配套例题】某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址为 32 位，主存地址为 30 位，页大小为 1 KB。若 TLB 共有 32 个表项，采用 4 路组相联映射方式，则 TLB 表项中标记字段的位数至少是（ ）。

A. 17　B. 18　C. 19　D. 20

解析。偏移 10 位，VPN 为 22 位。TLB 共有 8 组，选组用 VPN 的 3 位，标记为 $22-3=19$ 位，选 **C**。物理地址 30 位决定页框号位数，不决定用来匹配虚页的标记位数；也不能把 Cache 块偏移的位数套用到 TLB 标记上。

计算含 TLB 的平均时间时，应画出题设路径。例如，设访问串行进行，TLB 命中率为 h，TLB 耗时 t_T，单级页表和主存访问各耗时 t_M，则不计缺页时的平均时间为 $h(t_T+t_M)+(1-h)(t_T+2t_M)$。多级页表、页表项缓存、TLB 与 Cache 并行查找都会改变路径。缺页还包括外存和操作系统处理，通常远大于一次普通主存访问。

3.13 段式与段页式地址为什么需要界限检查

不变量·段界限先约束偏移

段式地址包含段号与段内偏移，应先核段表和偏移是否在界限内，再形成物理地址或进入分页。扩大基址不能修复越界偏移；非法地址与合法页未驻留也不能混成同一种缺页。

段式系统按代码、数据等逻辑单位划分不等长段。逻辑地址含段号和段内偏移，段表记录段基址、段长及权限。若段长表示可用字节数 L，必须先验证 $0 \le d < L$，再计算物理地址 base $+ d$；当 d=L 时已经越界，不能把末地址写成 base+L。

段页式先按段号找到该段页表，再用段内页号查页框，最后拼接页内偏移。段表基址、页表基址和最终数据地址不是同一个量。若段本身有界限，页号合法仍不保证最后一页的所有偏移均合法。

例如某段基址 10000 B、段长 3000 B，偏移 2999 合法、物理地址 12999 B；偏移 3000 非法。若段页式采用 1 KB 页，该段的逻辑长度仍为 3000 B，最后一个页号 2 中的偏移 952 对应段内 3000，应被界限检查拒绝，不能因页框还有空间就允许访问。

3.14 综合访存：数组次序、TLB 替换与 Cache 查找

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

3.14.1 例 3-11　交换内外层循环，命中率一定降低吗

例 3.14.1 (配套例题)

【配套例题】改编（两个赋值式统一为常量 10）已知计算机 M 字长为 32 位，按字节编址，采用请求调页策略的虚拟存储管理方式，虚拟地址 32 位，页面大小为 4 KB；数据 Cache 采用 4 路组相联映射方式，数据区大小为 8 KB，主存块大小为 32 B。现有 C 语言程序段如下：

```
int a[24][64];
/* ... */
for (i = 0; i < 24; i++)
    for (j = 0; j < 64; j++) a[i][j] = 10;
```

已知二维数组 a 按行优先存放，在虚拟地址空间中分配的起始地址为 0042 2000H，`sizeof(int)=4`。假定在 M 上执行上述程序段之前数组 a 不在主存，且在该程序执行过程中不会发生页面置换。请回答下列问题。

（1）数组分布在几个页面中？对于数组 a 的访问，会发生几次缺页异常？页故障地址各是什么？

（2）不考虑变量 i 和 j，该程序段的数据访问是否具有时间局部性？为什么？

（3）计算机 M 的虚拟地址（A31～A0）中哪几位用作块内地址？哪几位用作 Cache 组号？a[1][0] 的虚拟地址是多少？其所在主存块对应的 Cache 组号是多少？

（4）数组 a 占用多少主存块？假设上述程序段执行过程中数组 a 的访问不会和其他数据发生 Cache 访问冲突，则数组 a 的 Cache 命中率是多少？若将循环中 i 和 j 的次序按如下方式调换，则数组 a 的 Cache 命中率又是多少？

```
for (j = 0; j < 64; j++)
    for (i = 0; i < 24; i++) a[i][j] = 10;
```

解析。先把数组元素下标化为字节地址，再分别按页和 Cache 块划分。按行优先，每行 64 个 int，共 256 B，因此

$$\mathrm{addr}(a[i][j]) = \texttt{00422000H} + (64i + j) \times 4.$$

本题参考解按写分配模型处理首次写缺失：写一个尚不在 Cache 中的块时先调入该块。后续命中率均采用这一约定；若明确改为非写分配，结论须重算。数组初始不在主存，因此不能把数组块预先算作有效命中。

（1）数组大小为 $24 \times 64 \times 4 = 6144$ B，即 6 KiB。起始地址低 12 位为 0，恰在 4 KiB 页边界；前 16 行占第一页 4096 B，后 8 行占第二页

2048 B，所以共 2 页。

第一次执行 a[0][0] 时缺页，故障地址为 0042 2000H；第一次执行 a[16][0] 时进入下一页，故障地址为 0042 3000H。无页面置换，每页首次调入后保留，共 2 次缺页。6 KiB 除以 4 KiB 后向上取整在这里成立，是因为首地址页对齐；若数组从页中部开始，必须把首地址页内偏移也计入。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

（2）以数组元素为数据对象，各元素只被写一次，没有重复访问同一元素，因此没有时间局部性。按行连续访问相邻元素，具有空间局部性。同一 Cache 块中相邻元素的后续访问能够命中，正是按块调入利用空间局部性的结果；不应据此认为同一元素 “又访问了一次”。

（3）块为 2^5 B，块内地址为 A4～A0。Cache 行数 $8192/32 = 256$，4 路共有 64 组，组号 6 位，对应 A10～A5。虽然映射最终作用于物理块，但这些位都在低 12 位页内偏移内，地址转换不改变它们，因此可以直接从虚拟地址取出本题组号。

a[1][0] 在起点之后 256 B，即 0042 2100H。低 11 位中的组号为 `(2100H>>5) mod 64=8`，二进制 001000。不要把 “第 1 行” 当成只跨过一个 int。

（4）每块 32 B 可容纳 8 个 int，起点块对齐，总块数 $6144/32 = 192$。按行访问时，每块第一次写缺失，随后 7 次写命中，共 1536 次写、192 次缺失，命中率为

$$\frac{1536 - 192}{1536} = \frac{7}{8} = 87.5\%.$$

交换循环后，每个 j 先跨 24 行访问，同一列相邻两次地址相差 256 B，即相差 8 个块，组号也按模 64 增加 8。固定 j 时，24 个块轮流落在 8 个组中，每组恰好 3 块，**4 路容得下这 3 块**。因此不能仅凭 “按列走” 就断言块被赶走。

例如 j=0 时，行号 0、8、16 的块都映射到组 0，分别占据该组 3 路；第 4 路仍空。j=1 至 7 仍访问这 24 个已装入块，均命中。j=8 首次使用下一批 24 块，其组号在原来基础上加 1，与前批分属不同组；以后同理。全数组 192 块按映射可以同时驻留，仍只有 192 次首次缺失，命中率仍为 87.5%。

这个结论同时依赖数组大小、起始对齐、组数、路数和无其他数据冲突这些条件。不能只凭 “数组 6 KiB 小于 Cache8 KiB” 就下结论，因为总容量装得下，仍可能在少数组内发生冲突。

关键变式（原创）。若仅把 Cache 改成 4 KiB、2 路，保留 64 组和 32 B 块，则每个相关组仍轮流访问 3 块，却只有 2 路。按列遍历、LRU 替换时，三块循环会互相淘汰，下一列也难以复用，数组写访问全部缺失；按行遍历仍能在同块的 8 次访问中命中 7 次。决定解法的条件是每组能容纳的块数，而不是 “换循环必然降低命中率” 这类笼统说法。

3.14.2 例 3-12　TLB 的组选、标记与连续替换

例 3.14.2 (配套例题)

【配套例题】假设计算机 M 的主存地址为 24 位，按字节编址；采用分页存储管理方式，虚拟地址为 30 位，页大小为 4 KB；TLB 采用 2 路组相联方式和 LRU 替换策略，共 8 组。请回答下列问题。

（1）虚拟地址中哪几位表示虚页号？哪几位表示页内地址？

（2）已知访问 TLB 时虚页号高位部分用作 TLB 标记，低位部

分用作 TLB 组号，M 的虚拟地址中哪几位是 TLB 标记？哪几位是 TLB 组号？

（3）假设 TLB 初始时为空，访问的虚页号依次为 10、12、16、7、26、4、12 和 20，在此过程中，哪一个虚页号对应的 TLB 表项被替换？说明理由。

（4）若将 M 中的虚拟地址位数增加到 32 位，则 TLB 表项的位数增加几位？

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

解析。这里替换的是地址映射表项，先按 VPN 选 TLB 组，再在所选组内更新 LRU；不依据物理页框号选组，也不对全部 16 项作一次全局 LRU。

（1）4 KB=2^{12} B，故虚拟地址 A11～A0 为页内偏移，A29～A12 共 18 位为 VPN。物理地址 24 位只决定页框号为 12 位，不能据此把虚页号也截成 12 位。

（2）8 组需 3 位组号，取 VPN 最低 3 位，即虚拟地址 A14～A12；余下 A29～A15 共 15 位为 TLB 标记。把字段写成位数核对：$15+3+12=30$。

（3）题中虚页号均为十进制。组号为 VPN mod 8，标记为 $\lfloor \mathrm{VPN}/8 \rfloor$。下表的状态从最久未使用到最近使用，只列本次选中的组。

次序	VPN	组号	标记
1	10	2	1
2	12	4	1
3	16	0	2
4	7	7	0
5	26	2	3
6	4	4	0
7	12	4	1

次序	VPN	组号	标记
8	20	4	2

同一过程的其余状态（按首列对应）：

次序	命中与否	本次访问后该组的 VPN 次序
1	未命中，空行装入	[10]
2	未命中，空行装入	[12]
3	未命中，空行装入	[16]
4	未命中，空行装入	[7]
5	未命中，第二路装入	[10,26]
6	未命中，第二路装入	[12,4]
7	命中，更新最近次序	[4,12]

次序	命中与否	本次访问后该组的 VPN 次序
8	未命中，淘汰 4	[12,20]

只有组 4 遇到第三个不同虚页。第 7 次命中 12 后，最久未用的是 4，所以访问 20 时替换 VPN=4 的表项。若只按装入次序排队，会错误淘汰 12；若只看全部 16 项里已用了几项，还会误以为未满就无需替换。

（4）页大小、组数和主存地址位数都不变。新增的 2 位虚拟地址属于 VPN 高位，全部进入 TLB 标记；页框号仍 12 位，故每个 TLB 表项增加 2 位。增加虚址位宽扩大了可区分的虚页范围，不会自动增加实际主存容量或 TLB 项数。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

3.14.3 例 3-13　TLB 未命中后，还要经过哪些判断

例 3.14.3 (配套例题)

典型示例（据王道《操作系统》§3.2.9 摘编、重排）。某系统按字节编址，虚拟地址为 14 位，物理地址为 12 位，页大小为 64 B。TLB 共 16 项，采用 4 路组相联；数据 Cache 使用物理地址，采用直接映射，共 16 行，每行 4 B。

按“先完成地址转换，再查数据 Cache”的串行模型分析。所有待访问的虚拟地址均合法，且允许读；页表存在位 $P=1$ 表示页面已在主存，$P=0$ 表示合法但尚未驻留，需要处理缺页。读 Cache 未命中时，从主存调入对应的 4 B 数据块。

以下给出本题所需的初始状态。表中页号、页框号、标记、Cache 行号和字节数据均用十六进制，后缀 H 表示十六进制；V 为有效位，“—”表示没有可用信息。TLB 的组号和路号均在 0 至 3 之间。

表 A　TLB 初始状态：每行对应一个表项

组号	路号	标记	有效位 V
0	0	03H	0
0	1	00H	0
0	2	09H	1
0	3	07H	1
3	0	07H	0
3	1	0AH	1
3	2	03H	1

组号	路号	标记	有效位 V
3	3	02H	0

同一过程的其余状态（按首列对应）：

组号	页框号
0	—
0	—
0	0DH
0	02H
3	—
3	34H
3	0DH

组号	页框号
3	—

表 B　页表中与本题有关的表项

虚页号	存在位 P	页框号
03H	1	02H
08H	0	—

表 C　Cache 初始状态：数据列按块内字节偏移排列

行号	标记	V	偏移 0
5H	0DH	1	36H
CH	02H	0	—

同一过程的其余状态（按首列对应）：

行号	偏移 1	偏移 2	偏移 3
5H	72H	F0H	1DH
CH	—	—	—

（1）写出查 TLB 时虚拟地址的字段划分，以及查 Cache 时物理地址的字段划分，并求各字段位数。

（2）分别读取虚拟地址 03D4H、00F1H、0229H 处的一个字节，依次说明 TLB 查找、必要的页表检查和 Cache 查找，指出最终从哪里取数，或在哪一步转入异常处理。能确定字节值时给出结果。

三次访问相互独立，均从上述初始状态开始。某次访问引起的回填不影响另两次；不推演缺页后的页框分配与替换，也不计算访问时间。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

解析。先分别拆分虚拟地址和物理地址，再沿实际访问路径检查各表项。

（1）两种地址的字段划分。页大小为 $64 = 2^6$ B，页内偏移占 6 位，虚页号 VPN 占 $14 - 6 = 8$ 位。TLB 有 $16/4 = 4$ 组，用 VPN 的低 2 位选组，其余 6 位作为标记。因此，从高位到低位，虚拟地址分为 **TLB 标记 6 位、组号 2 位、页内偏移 6 位**。

Cache 每行 4 B，块内偏移占 2 位；共 16 行，行号占 4 位，其余 $12 - 4 - 2 = 6$ 位作为标记。因此，物理地址分为 **Cache 标记 6 位、行号 4 位、块内偏移 2 位**。页内偏移和 Cache 块内偏移的长度不同，不能混用。

（2）三次独立的读访问。判断顺序如下。

- **查 TLB：**在选中组内，只有 $V = 1$ 且标记相同的表项才算命中。命中可取得页框号；未命中则查页表，此时还不能断定缺页。
- **必要时查页表：**本题已保证地址合法且有读权限。若 $P = 1$，取得页框号，并可回填 TLB；若 $P = 0$，转入缺页处理，当前查找停止。
- **形成物理地址后查 Cache：**用物理地址选行，再同时检查有效位和标记。$V = 1$ 且标记相同才命中；否则从主存调块，按块内偏移取出目标字节。

读取 03D4H：TLB 命中，Cache 也命中。虚页号为 0FH，页内偏移为 14H。VPN 的低 2 位给出 TLB 组号 3，其余位给出标记 03H。表 A 的组 3、路 2 有效且标记为 03H，取得页框号 0DH，无需再查页表。物理地址为

$$0\mathrm{DH} \times 40\mathrm{H} + 14\mathrm{H} = 354\mathrm{H}.$$

拆分 354H，得到 Cache 标记 0DH、行号 5H、块内偏移 0。表 C 中第 5H 行的 $V=1$，标记也是 0DH，因此从该行偏移 0 的位置读得 **36H**。

读取 00F1H：TLB 未命中，页面已驻留，Cache 未命中。虚页号为 03H，页内偏移为 31H；TLB 组号为 3、标记为 00H。组 3 没有有效的匹配项，因此查表 B。虚页 03H 的 $P=1$、页框号为 02H，说明页面已在主存，**不发生缺页**。物理地址为

$$02\text{H}\times 40\text{H}+31\text{H}=0\text{B1H}.$$

拆分 0B1H，得到 Cache 标记 02H、行号 CH、块内偏移 1。第 CH 行虽保留着相同标记，但 $V=0$，所以仍未命中。须从主存调入地址 0B0H 至 0B3H 所在的 4 B 块，再取偏移 1 的字节。题中没有给出该主存字节的内容，**只能确定取数路径，不能确定字节值**。

读取 0229H：TLB 未命中，随后发生缺页。虚页号为 08H，页内偏移为 29H；TLB 组号为 0、标记为 02H。组 0 没有有效的匹配项，转查页表。表 B 中虚页 08H 的 $P=0$，因此触发缺页异常。此时尚无可用页框号，**不形成物理地址，也不继续查数据 Cache**。由操作系统完成缺页处理、更新映射并重新执行该访问后，才继续取数。

三次访问分别在 TLB、页表和 Cache 处作出不同判断。TLB 缓存的是映射，页表存在位说明页面是否驻留，Cache 保存的是数据块：**TLB 未命中不等于缺页，页面已驻留不等于 Cache 命中，标记相同也不能代替有效位检查**。真实系统还须区分非法地址和访问越权；本题通过 “地址合法、允许读” 的条件将这两类异常排除。

课程连接·从路径列事件

先列访问哪些层级、命中或缺失时发生哪些访问，再按模型计算平均时间；不要把不同路径的额外时间重复计入。

3.15 先画访问路径，再写时间与容量

存储系统综合题的顺序

1. 写明地址位数、按字节还是按字编址、块与页大小，把地址先拆成有意义的字段。
2. 按题设画 TLB、页表、Cache 和主存的实际查找路径；分清并行与串行，缺页与普通缺失。
3. 每一步同时检查标记、有效或存在位、权限；得到物理地址后再按指定 Cache 模型查数据。
4. 时间式以完整事件分支加权；容量式先分数据位、标记和有效/脏/替换等元数据。
5. 回代一个边界：空 Cache、组数 1、路数 1、页面不驻留或跨页，查看路径是否仍一致。

3.15.1 地址拆分为什么不遗漏信息

从整除和余数恢复地址

设字节地址 A，块大小 B，组数 G，各量为适用范围内的整数。先作 $A=qB+d$，其中 $0\le d<B$；再作 $q=tG+s$，其中 $0\le s<G$。因此 $A=(tG+s)B+d$，恰由标记 t、组号 s 和块内偏移 d 恢复。B、G 为 2 的幂时，这两次除法正对应二进制位段切分；不是 2 的幂时仍可整除取模，但不能硬套整数位数。

□

3.15.2 主存器件分类与刷新开销

不变量·相联程度不直接等于延迟

全相联能并行比较多行，组相联只比较一个组；实际延迟还受比较器、布线和编码影响。固定实现才可比较快慢。映射限制候选位置，替换选择牺牲行，电路时序决定访问时间。

只读存储器的“只读”描述其正常使用或修改方式，不表示所有 ROM 永远无法改写。掩膜 ROM 由制造过程固定，PROM 可一次编程，EPROM 可通过特定方式擦除重写，EEPROM 支持电擦除；Flash 也是电可擦写的非易失存储，但擦除粒度通常较大。判断题先区分是否掉电保持、能否改写、擦写粒度与速度，避免把这些维度混成同一个分类。

刷新保持 DRAM 电容状态，读后恢复则由一次读出带来的状态变化引起；二者目的相关，但不能据“读后要恢复”推断只有被访问的行才需刷新。若 4096 行必须在 64ms 内各刷新一次，平均每 15.625 微秒需安排一行刷新；若每次刷新占 100ns，则理想均匀安排下占用比例约 0.64%。这是自拟预算示例；具体实现可能一条命令刷新多行或由芯片内部完成，要服从题设。

3.15.3 相联查找的并行性与延迟条件

相联映射描述能放在哪里以及怎样匹配标记，不直接决定所有比较的实际电路延迟。全相联可以并行比较多行标记，组相联可以只比较目标组中的若干行；比较器数、扇入、布线和编码都会影响时间。比较“最快”“最慢”时须固定具体实现，不能仅用相联度高低给出普遍时延定理。

写策略同样分成两次决策：写命中采用直写或回写，写缺失采用分配或不分配。回写的英文为 write-back，写分配为 write-allocate；二者分别处理命中更新和缺失装入，名字不能互换。脏块替换时才需写回，并非每次替换都必然产生一次主存写。

3.16 存储器分类、刷新与双端口访问

按存取方式，随机存取 RAM 可按任意地址访问，通常不因地址位置不同而有明显不同的寻址步骤；顺序存取 SAM 必须沿记录次序定位，如磁带；直接存取 DAM 先选大区域再在局部定位，如磁盘；相联存取按内容或标记比较选择信息。随机存取描述访问方式，与“断电是否保持”是两个分类维度。

SRAM 利用双稳态电路保存位，读取通常非破坏性，无需周期刷新；DRAM 利用电容电荷保存位，读出后要再生，并需在刷新周期内刷新所有行。读后再生恢复刚读出的电荷，定时刷新防止未读单元因漏电丢失信息，二者目的不同。DRAM 按行刷新时，刷新地址计数与行数有关，而不取决于存储器容量中的每一个字。

集中刷新在固定时段连续刷新全部行，形成较长死时间；分散刷新把正常存取和刷新交替安排，增加每次存取周期；异步刷新将刷新动作均匀分散在保持时间内。设有 128 行、每行刷新一次需 0.5 μs、要求 2 ms 内全部刷新，则相邻行刷新启动间隔可按 $2\text{ms}/128 = 15.625\,\mu\text{s}$ 安排，总刷新时间为 64 μs，占比 3.2%。

双端口 RAM 具有两套独立地址、数据和控制线路，两端可并行访问不同单元。同一单元上的读写或写写冲突，需要通过忙信号、仲裁或芯片规定的访问规则处理；“双端口”并不自动保证任何两次访问都无冲突。

3.16.1 ROM 各类别的可改写能力

课程连接·页与擦除块

闪存按页读取或编程，按块擦除；逻辑覆盖常转为异地写入并更新映射，磨损均衡不能消除擦除次数上限。

器件	内容写入或修改方式	需要注意的条件
掩膜 ROM	制造时写入	使用阶段不能修改
PROM	用户一次编程	写入后通常不能重新编程
EPROM	紫外线擦除后重写	通常整片擦除，修改不便
EEPROM	电擦除、电改写	改写速度与次数受器件限制
Flash	按块擦除、按规定粒度写入	写前擦除，读写和擦除粒度不同

3.17 SSD 的页、块与磨损均衡

SSD 由闪存翻译层和多个闪存芯片组成，芯片内再分块，块内再分页。系统给出逻辑块号，闪存翻译层 FTL 把它映射为物理页地址。NAND Flash 通常按页读取或编程、按块擦除；页与块大小取决于具体器件，课件例示页约 512 B～4 KiB、块约 16～512 KiB，这些是示例范围，不能作为所有 SSD 的固定参数。

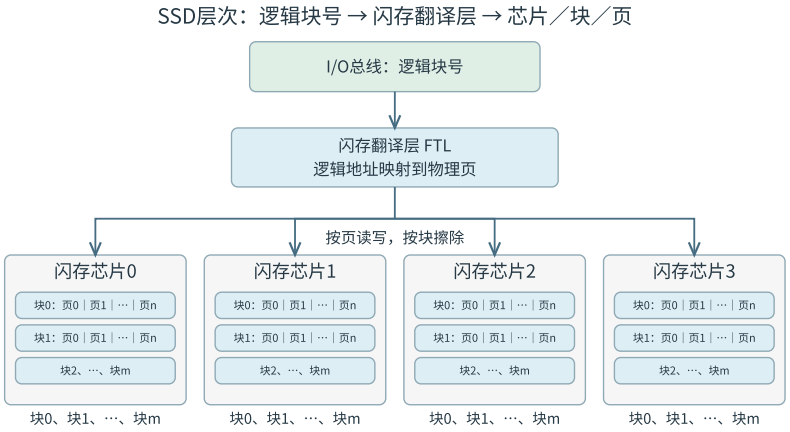


Figure 3.4: 系统逻辑块号经 FTL 映射到闪存芯片中的块与页

旧页中的数据需要更新时，控制器可把有效数据和新内容写入另一个已擦除块，更新映射，再把旧页标为失效。垃圾回收把仍有效的页集中搬走后擦除旧块，恢复空闲空间；额外搬移会产生写放大。读写快、无机械寻道、低噪声和抗震是相对机械磁盘的典型特点；擦写寿命、容量成本和写入管理是另一组比较维度。

磨损均衡通过分散擦写次数延长整体寿命。**动态磨损均衡**在发生写入时，优先使用擦除次数较少的空闲块；长期不改写的冷数据可能一直占着某些块。**静态磨损均衡**还会主动搬移冷数据，让擦写较少的块参加新写入，使各块使用更均匀。主动搬移增加了管理开销，不能把静态均衡理解为 “完全不移动数据”。

3.18 位扩展、字扩展与片选译码

课程连接·位扩展与字扩展

位扩展让多片提供同一地址的不同数据位；字扩展选择不同芯片提供更多地址，两者可组合。

芯片标称 $N \times w$ 表示 N 个单元、每单元 w 位。用 $n \times b$ 芯片组成 $N \times W$ 存储器，若两维均为整数倍，片数为 $(N/n)(W/b)$。

位扩展让若干芯片同时接收相同地址，各自负责不同数据位，共同构成更宽的存储字；片选相同，数据线分组连接。**字扩展**让各组芯片分别负责不同地址区间，低位地址接到组内所有芯片，高位地址经译码产生互斥片选。字位同时扩展时，先确定同组并行芯片数，再确定地址组数。

3.18.1 例：用 16K×8 位芯片组成 32K×16 位

目标字数扩大 2 倍，字长扩大 2 倍，共需 4 片。每组 2 片分别接 $D_7 \sim D_0$ 和 $D_{15} \sim D_8$；$A_{13} \sim A_0$ 并联到四片，A_{14} 选择两组。按字编址时，组 0 范围 0000H～3FFFH，组 1 范围 4000H～7FFFH。

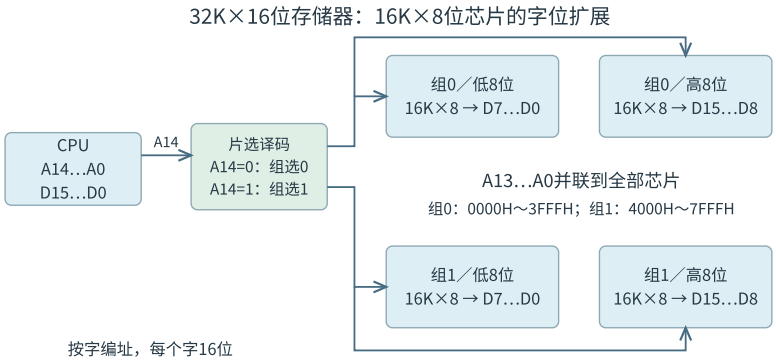


Figure 3.5: 同组两片作位扩展，A14 选择两组作字扩展

若按字节编址、每字 16 位，还需要说明最低地址位怎样区分字节以及字节使能信号如何连接，不能直接把上述按字地址范围照搬。片选有效电平由芯片和译码器图中的小圆圈、横线约定决定；逻辑表达式必须同时考虑访存使能和地址译码，避免 I/O 访问错误选中内存。

全译码使用全部需要判别的高位地址，地址对应关系唯一；部分译码省略某些高位，可能让多个地址选择同一物理单元，形成地址重叠；线选法直接用地址线选片，电路简单，但可用地址组合与地址连续性受限制。

3.19 交叉编址的连续访问时间

高位交叉用地址高位选择模块，同一连续区间常落在同一模块，主要扩容。低位交叉用低位选择模块，连续地址轮流分布到各模块，允许各模块启动时间交错。

设每个模块访问周期为 T，总线传送一个字需 τ，模块数 m 满足 $m\tau \geq T$，按连续地址访问且无其他冲突时，可每隔 τ 启动一个新字。读取连续 n 字的理想时间为 $T + (n-1)\tau$。若 $m\tau < T$，最先使用的模块尚未完成恢复便又被访问，须加入等待。

例如 $T = 200$ ns、$\tau = 50$ ns，至少 4 个模块才可连续交错启动；读取 8 字需 $200 + 7 \times 50 = 550$ ns。步长访问还要看模块号序列：模块数 4 且步长 4 时，总是访问同一模块，即使地址不同也不能获得连续逐字带宽。

3.20 Cache 替换：把命中后的状态也写出来

课程连接·命中也会改变 LRU

LRU 命中后需把命中块更新为最近使用；FIFO 命中一般不改变装入顺序。每次访问后写出状态，避免只数缺失。

直接映射的候选位置唯一，冲突时无需在多行中选择牺牲者；组相联只在所选组内替换，全相联在全部行内替换。有效位为 0 的空行优先使用。替换策略独立于写策略；被替换行若采用写回且脏位为 1，还要先回写。

FIFO 按装入时间淘汰最早装入行，命中不改变装入顺序；LRU 按最近使用情况淘汰最久未使用行，命中也要更新次序；LFU 按累计访问次数选择，需规定计数初值及并列时的规则；随机替换由硬件产生选择，成本较低。

4 行全相联 Cache 初始为空，访问主存块 $1,2,3,4,1,2,5,1,2,3,4,5$：

算法	命中位置（从 1 计数）	命中数／缺失数
FIFO	第 5、6 次	2 ／ 10
LRU	第 5、6、8、9 次	4 ／ 8

本序列中 LRU 命中 4 次、FIFO 命中 2 次，牺牲块和中间内容也不同。例如访问 5 前，FIFO 仍先淘汰 1，LRU 先淘汰 3；不能只核对最终命中率，就认为过程表正确。实现 LRU 可用访问次序栈或相应状态编码；题目采用计数器法时还须遵守其更新规则。

3.21 平均访问时间的两个模型

设 Cache 访问时间为 t_c、主存时间为 t_m、命中率为 h。若 Cache 和主存同时开始访问、命中时用 Cache 结果，则平均时间按 $ht_c+(1-h)t_m$ 计算。若先访问 Cache，确认未命中后再启动主存，则为 $t_c+(1-h)t_m$。还存在题目直接给出 “缺失损失” 的形式 $t_c+(1-h)p$，其中 p 须按定义判定是否含取块、回写和重新访问。

课件模型取 $t_m=5t_c$、$h=0.95$：同时访问时平均 $1.2t_c$，相对仅访问主存加速 $5/1.2\approx4.17$ 倍；先 Cache 后主存时平均 $1.25t_c$，加速 4 倍。这里的同时访问是计算模型，不是所有处理器的必然实现。

写直达在写命中时同时更新下层，写缓冲可以暂存待写内容；写回只更新 Cache 并置脏位，替换脏行时才写下层。写分配与非写分配处理的是写缺失：前者调入块后写，后者绕过当前 Cache 写入下层。常见组合属于工程选择，分析题目时按明确给出的策略推演。

3.22 磁盘记录与 RAID 的边界

磁盘由盘面、磁道、扇区组成，柱面是不同盘面上半径相同的磁道集合。一次访问通常包含寻道、旋转等待和数据传输；平均旋转等待在随机到达且均匀假设下为半圈。容量按有效盘面数 $\times$ 每面磁道数 $\times$ 每道扇区数 $\times$ 每扇区字节数计算，格式开销和保留区域需按题设处理。

RAID0 条带化提升并行带宽而无冗余；RAID1 镜像提高冗余但有效容量约减半；RAID3、RAID4 采用专用校验组织，RAID5 分布式校验避免固定校验盘成为所有写入的同一热点。冗余的容错范围与级别、故障数量有关，不能据 “有校验” 就断言任意多盘损坏都能恢复。写入更新还可能产生数据与校验的一致性开销。

3.22.1 计算与辨析自查

1. 四模块低位交叉中连续字和步长 4 访问的模块分布有何不同？2. 32K×16 位目标由 16K×8 位芯片组成需要几片？3. 为什么 FIFO 命中后不改队列而 LRU 要改？4. SSD 按页写、按块擦除会引出哪些管理动作？

课程连接·地址划分顺序

依题意先确定块内偏移，再根据组数确定组号；比较标记前还要验证有效位。

答案。1. 连续字轮流访问 0、1、2、3 号模块，步长 4 始终访问同一模块。2. 4 片，两片一组、两组字扩展。3. FIFO 记录装入时间，LRU 记录使用时间。4. 异地更新、地址映射、失效页管理、垃圾回收和磨损均衡。

3.23 主动回忆自测

3.23.1 配套自测

1. 原创补充：32 位字节地址、Cache 数据区 4 KB、块 16 B、2 路组相联，求偏移、组号、标记位数。假定另需有效位和修改位，先不计替换位，每行多少位？
2. 原创补充：两路某组按 LRU 访问 A、B、A、C、B，初始空，列每次命中情况与最近到最久次序。
3. 【配套例题】下列事件中，不是在 MMU 地址转换过程中检测的是（ ）。A. 访问越权　B. Cache 缺失　C. 页面缺失　D. TLB 缺失。

答案。1. 4、7、21 位；每行 $128+21+1+1=151$ 位。2. 缺失 [A]、缺失 [B,A]、命中 [A,B]、缺失 [C,A]、缺失 [B,C]。3. B。MMU 负责地址转换和相关保护；Cache 缺失属于 Cache 查找结果，不能与 TLB 缺失等同。

3.23.2 遮住答案复述知识关系

主动回忆·自拟

4. 自拟：块大小 64B、物理地址 32 位、Cache 共 16 行，采用 2 路组相联。组号多少位？与直接映射相比，标记位数怎样变化？
5. 自拟：一次查 Cache 为 2ns，命中率 95%，缺失后额外访问下一层 80ns，AMAT 是多少？如果“缺失时间 80ns”已经含查 Cache，公式又怎样写？
6. 自拟：4 体低位交叉、按字编址，字地址 0、4 连续访问为何可能冲突？是否任何两次同模块访问必冲突？
7. 自拟：TLB 未命中、页表存在位为 1、Cache 有效位为 0 时，分别发生哪两种缺失？是否缺页？
8. 自拟：回写 Cache 的一行被淘汰，是否必定写主存？还要看哪一位？
9. 自拟：DRAM 无需掉电也可能丢信息，Flash 掉电仍保留信息但为什么不能按 SRAM 方式任意覆盖写？

3.23.3 参考答案

自拟题核对答案

4. 8 组、组号 3 位，偏移 6 位，标记 23 位；直接映射 16 行，行号 4 位、标记 22 位，因此 2 路相联标记多 1 位。
5. 额外 80ns 时 $2 + 0.05 \times 80 = 6$ns；已含查 Cache 则 $0.95 \times 2 + 0.05 \times 80 = 5.9$ns。
6. 两者模 4 都为 0，若第二次请求到达时模块尚忙则冲突；若时间间隔足够则不冲突。
7. TLB 缺失和 Cache 缺失；页面已驻留，不发生缺页，仍需正常读取物理主存。
8. 不一定；只有脏位表明内容已修改且需要回写时才写回，干净副本可直接丢弃。
9. DRAM 电荷泄漏，需要周期刷新；Flash 编程和擦除粒度、允许的位变化及寿命条件不同，通常需先擦后写并由控制器管理。

课程连接·地址划分顺序

依题意先确定块内偏移，再根据组数确定组号；比较标记前还要验证有效位。

指令系统与 C 程序的机器级表示 4

章节脉络



本章围绕 “指令格式与编码 → 数据寻址 → 指令寻址与访问 → ISA 设计取舍 → 程序工具链 → 控制流与过程调用” 展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

不变量·操作码与操作数字段分开数

固定总长下，地址字段占用越多，可留给操作码的位越少。扩展操作码逐层使用未占用前缀，不是直接从全部位串数里减去指令条数。地址字段组合也占编码空间，容量必须按同一层级计算。

4.1 指令字段怎样决定可表示的操作和地址

定义 4.1.1 (概念与适用对象)

一条机器指令包含操作码以及完成该操作所需的其他字段。操作码规定做什么，地址字段可能给出立即数、寄存器编号、形式地址、位移或寻址方式，并非所有字段都是主存绝对地址。指令字长是指令编码的位数，机器字长是整数运算的基本位宽，两者不必相等。

按显式操作数地址字段数，可以有零地址、一地址、二地址、三地址指令。零地址指令也可以操作数据：栈式机器的加法隐含从栈顶取两个数；一地址指令可隐含使用累加器；二地址形式常让一个地址同时承担源和目的，但具体语义仍要由题目给出。“两个字段” 并不对应固定的访存次数：字段还可能指定寄存器或立即数。

数据传送指令移动或复制值；算术逻辑指令形成新值和可能的标志；转移指令改变后继执行地址；调用在转移前保留返回位置，返回根据保存的位置继续；输入输出及系统控制指令还可能有权限限制。通常的赋值和算术指令不会把源操作数 “取走后清空”。

固定长度操作码用 k 位可区分至多 2^k 种编码。扩展操作码把部分短编码保留为扩展前缀，用更长的操作码表示操作数较少的指令。已作为短指令使用的编码不能同时作长指令前缀，否则译码有歧义。

4.1.1 例 4-1 扩展操作码应逐层扣除哪些前缀

例 4.1.1 (配套例题)

【配套例题】设计某指令系统时，假设采用 16 位定长指令字格式，操作码使用扩展编码方式，地址码为 6 位，包含零地址、一地址和二地址 3 种格式的指令。若二地址指令有 12 条，一地址指令有 254 条，则零地址指令的条数最多为（ ）。

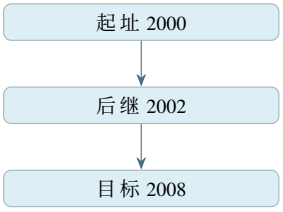
A. 0 B. 2 C. 64 D. 128

解析。二地址格式需要 12 位地址，短操作码为 4 位，共 16 种前缀，用掉 12 种，剩 4 种可扩展。每个前缀再接 6 位，共提供 $4 \times 64 = 256$ 种一地址操作码；用掉 254 种，剩 2 种再各扩展 6 位，得到 $2 \times 64 = 128$ 条零地址指令，选 **D**。

不能从 2^{16} 中直接减去 “12+254”，因为每条二地址指令还对应 2^{12} 种操作数字段组合，每条一地址指令对应 2^6 种组合。逐层保留前缀的算法直接反映译码过程，也方便处理其他字段长度。

最小反例·相对基准不一定是起点

最小反例：2B 指令从 2000H 取完，PC 已变 2002H，再加 6 应得 2008H。直接用指令起点加 6 得到 2006H，漏算了取指更新。负位移还须先按给定位宽作符号解释。



2B 指令取完后加 6，目标为 2008H。

4.2 从形式地址到有效地址

以下记指令地址字段为 A，寄存器 R 的内容为 (R)，主存地址 x 的内容为 M[x]；有效地址 EA 指最终操作数所在的主存地址。立即数不需要另求操作数的主存地址；寄存器寻址的操作数也在寄存器中，不必为凑访存次数而虚构一次主存访问。

寻址方式	操作数或有效地址	核对条件
立即	操作数在指令中	字段的符号扩展规则
直接	EA=A，操作数 =M[A]	A 的编码范围
一次间接	EA=M[A]	先取地址，再取操作数
寄存器直接	操作数 =(R)	字段是寄存器编号
寄存器间接	EA=(R)	寄存器保存的是地址
基址	EA=(BR)+ 位移	位移位宽及符号
变址	EA=A+(IX)	元素下标是否已乘元素宽度

寻址方式	操作数或有效地址	核对条件
相对	目标 =PC 基准 + 位移	PC 指当前还是后继地址

间接寻址得到的中间地址位宽可能大于指令地址字段，从而扩大可寻址范围，代价是多一次取地址的访存。基址可用于程序重定位，变址适合数组遍历；二者都可写成寄存器加位移的形式，但寄存器的用途不同，具体机器也可能用同一种编码机制实现。

4.2.1 例 4-2 相对转移用哪个 PC

例 4.2.1 (配套例题)

【配套例题】某机器字长为 16 位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一个字节为操作码字段，第二个字节为相对位移量字段。假定取指令时，每取一个字节 PC 自动加 1。若某转移指令所在主存地址为 2000H，相对位移量字段的内容为 06H，则该转移指令成功转移后的目标地址是（ ）。

A. 2006H　B. 2007H　C. 2008H　D. 2009H

解析。指令从 2000H 开始，占 2 B，取完后 PC=2002H。相对位移为 +6，目标为 2008H，选 **C**。若位移字段改为 FAH 且按 8 位补码解释，则位移为 −6，目标为 1FFCH；不能把 FAH 当作正 250。PC 基准必须以本题规定的更新时机为准。

数组寻址还需区分元素下标和字节偏移。按字节编址、int 为 4 B 时，`a[i]` 地址为 base+4i。某 ISA 若有 “比例变址”，可由硬件把索引乘 4；若没有，则由移位或其他指令形成 4i。这不意味着数组元素之间多留了 3 个空洞，四个字节共同存放一个元素。

4.2.2 访存次数要按执行路径来数

若指令为一个字、无 Cache、无预取，直接寻址的读操作数指令通常至少取指一次、取操作数一次；一次主存间接寻址再多读一次地址。若还写回主存，需另计一次写。若指令跨两个存储字、发生缺页或含两个主存操作数，次数会改变。“寄存器间接” 中的 “间接” 不会额外读一次主存地址，因为地址已经在寄存器中。

不变量·只数真实读写事件

无 Cache 无预取时，取指、取有效地址、取操作数和写回主存各是独立事件。某字段在寄存器中就不应虚构一次主存读取；指令跨字或有两个主存源时也不能沿用一字一源的计数。

4.3 RISC 与 CISC：设计倾向不代替具体实现

RISC 通常采用较规整的指令格式、较少的寻址方式和 Load/Store 风格：算术逻辑操作主要在寄存器间进行，访存由装入和存储指令承担。这有利于规整译码和流水处理。CISC 往往支持更丰富的指令形式与寻址方式，单条指令可以表达较复杂操作。

下表按共同维度比较教材中的典型设计。表中的 “较多”“较少” 只是同一历史设计背景下的倾向，并不按指令条数、某条指令或单项性能为所有处理器划定边界。

比较维度	RISC 的典型设计	CISC 的典型设计	做题时的适用边界
指令功能与使用	选用常见的简单操作，复杂功能由指令组合实现	指令功能较丰富，一条指令可表达较复杂操作	不用 “小于 100 条／大于 200 条” 等历史概数代替具体 ISA 定义
编码与寻址	格式较规整，常用定长编码，寻址方式较少	格式和寻址方式较多，常见变长编码	定长、变长须按题给编码确认；不能只凭 RISC 名称确定每条指令字节数
数据操作数访存	典型 Load/Store 模型中，装入和存储访问数据存储器，算术逻辑在寄存器间进行	多类指令可含主存操作数	这里说的是数据访问，取指也要读存储器；具体指令能否访存仍查其语义
寄存器与调用	通用寄存器较多，多用寄存器传参，减少数据访存	传统设计的通用寄存器相对较少，操作数来源较丰富	寄存器数由 ISA 给定；传参位置还受 ABI 约束，不由 RISC/CISC 名称单独决定
执行时间与 CPI	简单操作便于规整流水；教材常用多数简单指令单周期的理想模型	不同指令的执行时间差异往往较大	单条指令延迟、流水吞吐和平均 CPI 分别计算；访存等待、乘除及发射宽度都会改变结果
控制方式	多采用硬布线控制	复杂指令常借助微程序控制	两者是控制器实现方式，可结合使用，不能断言某类 ISA 只能选一种
流水线	规整编码和操作有利于流水处理	也可通过译码、分解内部操作等方式实现流水	采用流水线不是 RISC 独占的能力；阶段数、旁路和阻塞规则均由具体微架构决定
编译优化	复杂功能较多依靠编译器选指令、分配寄存器和调度	丰富指令给出更多选择，选指令与优化可能更复杂	两类都需要编译优化；目标代码条数较少不直接等于执行时间较短

做题时要按题目中 “多采用”“通常” 等措辞判断典型特征；删去这些限定，就会把设计倾向误读成必然条件。

课程连接·ISA 不等于微架构

是否采用流水线、旁路和哪种控制器主要涉及微架构实现，ISA 规定软件可见指令语义。两类层次相互影响但不能互代。指令简单有利于规整实现，不意味着只可能采用一种硬件结构。

4.3.1 例 4-3　RISC 设计能否支持流水线

例 4.3.1 (配套例题)

【配套例题】下列关于 RISC 的叙述中，错误的是（　）。

A. 多采用硬连线方式实现控制器
B. 通常采用 Load/Store 型指令设计风格
C. 难以采用流水线数据通路实现微架构
D. 多采用寄存器传递过程调用时的参数

解析。格式规整、操作较简单，通常有利于流水线实现，所以 C 错。A、B、D 描述的是常见设计倾向，不应把 “多采用” 读成 “只能采用”。答案为 **C**。

4.4 编译、汇编、链接怎样确定程序中的地址

第 1 章已给出工具链顺序，这里进一步说明为什么目标文件未必能直接运行。编译和汇编时，一个模块可能引用另一个模块的函数或全局变

量，汇编器尚不知道它的最终地址，只能留下符号与重定位信息。链接器合并相应节，解析符号，再按选定地址修改相关引用。

可执行文件包含机器代码，也包含装入所需的组织信息；CPU 并不把文件的每一个字节都当作指令。目标文件还可能含符号表、调试信息和重定位记录。高级语言的变量名在运行时未必仍对应一个固定的主存地址；优化后变量本身可能在寄存器中、被折叠成常量，或已经消失。

错项排除·目标代码不是最终运行状态

汇编得到可重定位目标文件，还可能缺跨模块符号地址。链接并安排引用后形成可执行文件，运行还需装入和环境。把汇编与链接交换，既无法解析源语言，也无法解释地址何时确定。

4.4.1 例 4-4　工具链顺序

例 4.4.1 (配套例题)

【配套例题】将高级语言源程序转换为可执行目标文件的主要过程是（　）。

A. 预处理 → 编译 → 汇编 → 链接
B. 预处理 → 汇编 → 编译 → 链接
C. 预处理 → 编译 → 链接 → 汇编
D. 预处理 → 汇编 → 链接 → 编译

解析。先处理源文件中的预处理指令，再将高级语言翻译成汇编表示，继而编码为目标文件，最后解析跨模块引用，选 **A**。这是题目采用的典型静态编译流程；动态链接把部分工作留到装入或运行时，但汇编编码仍必须在程序开始执行前完成。

4.5 分支和循环怎样改变 PC

机器执行选择结构时，先形成条件，再决定是否改变后继地址。条件可由比较指令设置标志，也可由分支指令直接比较寄存器。翻译 C 条件时必须保留 signed/unsigned 的区别：两者都用减法硬件，但转移条件并不相同。

下列教学指令约定整数均为 32 位，R1 存放 x，R2 存放 y，`bge` 按有符号数比较，标签代表指令地址；并假定本例的差值可用 32 位有符号数表示：

```
    bge R1, R2, ELSE     // x>=y 时跳过 then
    sub R3, R2, R1      // then: z=y-x
    jump END
ELSE:
    sub R3, R1, R2      // else: z=x-y
END:
```

这里要实现 `if(x<y) z=y-x; else z=x-y;`，分支条件取原条件的否定，于是满足原条件时顺序落入 then。转移目标是代码位置，不是把标签的字符存进 PC。

分析循环时，先确定入口测试、循环体、更新和回边。对以下机器级教学轨迹，R0 为数组基址，R1 为元素下标，R2 为和，R3 为元素个数，int 占 4 B：

```
    R1 <- 0; R2 <- 0
TEST:
    if R1 >= R3 jump EXIT
    R4 <- R1 << 2
    R5 <- M[R0 + R4]
    R2 <- R2 + R5
    R1 <- R1 + 1
    jump TEST
EXIT:
```

不变量·返回地址属于调用点

调用保存的是调用之后继续执行的位置；函数入口只是被调目标。递归各层可以在同一代码位置调用，却有独立参数和栈帧。恢复现场时必须与当前返回层次配对，不能只追踪函数名。

R3=0 时直接退出，不读 `a[0]`；R3=3 时读取偏移 0、4、8 而非 0、1、2。这两个边界足以检查循环上下界和地址单位。若变换为指针遍历，每轮指针加 4，终止地址为 base+4R3；终止地址并不是最后一个有效元素的地址。

逆向读代码时，先标出各寄存器的初值和更新式，再识别分支条件。循环回边指令的执行次数与循环体次数可能不同：入口先测的 n 次循环，测试通常执行 n+1 次。算动态指令数和 Cache 访问次数时，这个差别必须计入。

4.6 过程调用为什么要保存返回位置

调用至少要解决四件事：把参数交给被调过程，记录返回位置，给局部变量和临时值安排空间，将返回值交回调用者。ISA 提供调用、返回或寄存器转移等基础机制，ABI 进一步规定参数寄存器、返回值位置、栈对齐，以及哪些寄存器由哪一方保存。

返回地址可以放到栈，也可以先放到链接寄存器。若采用链接寄存器，叶子函数可能不访存保存返回地址；会再次调用其他函数的过程，通常要在覆盖链接寄存器之前保存其中的返回地址。“返回地址一定写主存” 并不是所有指令系统的共同特征。

以下采用一个明确的栈模型说明调用：栈向低地址增长，每槽 4 B，SP 指向当前栈顶；push 先 SP 减 4 再写，pop 先读再 SP 加 4；调用压入后继指令地址，返回弹出该地址送 PC。调用前 SP=1000H，返回地址为 2004H，则调用后 SP=0FFCH、M[0FFCH]=2004H。

若被调过程又保存旧帧指针 FP、再分配 8 B 局部区，SP 依次变为 0FF8H 和 0FF0H。此时局部变量不能覆盖 0FFCH 的返回地址。退出时先撤销局部区，恢复旧 FP，再取返回地址；SP 最终回到 1000H。图中的地址单位为字节。

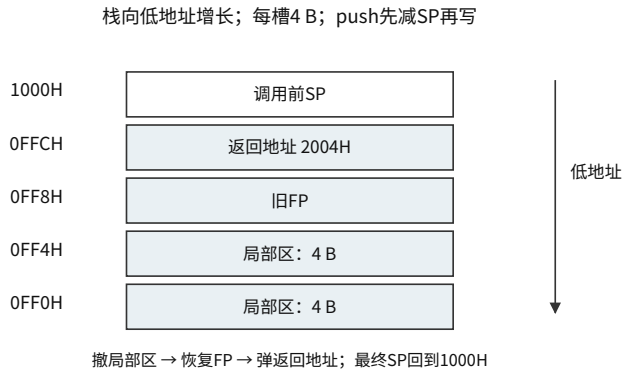


Figure 4.1: 调用、建立栈帧和返回时，SP 逐步移动

嵌套调用靠每次独立保存的返回位置区分 “回到哪里”；递归的各层也各有参数、局部变量和返回地址。局部变量同名不表示它们共用同一槽。调用者保存寄存器需在调用前保留自己还要使用的值；被调用者保存寄存器则在被调过程修改前保存、返回前恢复。

题目给出具体调用约定时，要按该约定追踪，不能凭熟悉的 x86、MIPS 或 RISC-V 经验补条件。特别要检查栈生长方向、SP 指向、入栈前后调整顺序和返回地址基准。这些细节都会影响地址计算与最终返回位置。

最小反例·转移不一定保存返回地址

最小反例：普通条件分支跳到标签，并不会自动为未来返回保留调用点。调用与分支都能改 PC，但保存返回位置的语义不同；把任意跳转都当成函数调用，会错误增加栈写和返回动作。

4.6.1 顺序、分支、调用和返回分别改变什么

执行方式	后继地址怎样得到	额外需要保留的信息	读代码时先核对什么
顺序执行	当前地址加指令字节数	不因普通顺序执行单独保存返回地址	变长指令必须实际数编码字节
条件分支	条件成立取目标，否则取顺序后继	通常无需调用返回地址	signed/unsigned 条件、位移符号、基准与缩放
调用	无条件转到被调入口	调用后的后继地址	返回位置存栈还是链接寄存器
返回	取先前保存的返回位置	还须按约定恢复栈和现场	返回地址来自哪层调用，不能重算成函数入口

相对寻址只规定 “基准地址加位移”，并不规定所有指令的位移都以字节为单位。例如下面 x86 的 call 用后继指令地址加有符号字节位移；某些定长 RISC 分支会把编码位移先左移两位。读机器码时应分别求出当前地址、指令长度、基准地址、扩展后的位移及缩放比例，最后核对目标。把汇编目标标签直接当成机器码里保存的绝对地址，会漏掉这条计算链。

4.6.2 例 4-5　递归调用的返回位置、机器编码与乘法溢出

例 4.6.1 (配套例题)

【配套例题】已知 $f(n)=n!=n\times(n-1)\times(n-2)\times\cdots\times2\times1$，计算 $f(n)$ 的 C 语言函数 f1 的源程序及其在 32 位计算机 M 上的部分机器级代码如下。

```
int f1(int n) {
    if (n > 1)
        return n * f1(n - 1);
    else
        return 1;
}
```

以下按 “行号／虚拟地址／机器指令（字节按地址递增排列）／汇编指令” 四列给出，每行是一条代码记录；省略行仍保留四列。

```
 1  00401000  55              push ebp
 …  …         …                 …
11  00401018  83 7D 08 01     cmp dword ptr [ebp+8], 1
12  0040101C  7E 17           jle f1+35h (00401035)
13  0040101E  8B 45 08        mov eax, dword ptr [ebp+8]
14  00401021  83 E8 01        sub eax, 1
15  00401024  50              push eax
16  00401025  E8 D6 FF FF FF  call f1 (00401000)
 …  …         …                 …
19  00401030  0F AF C1        imul eax, ecx
20  00401033  EB 05           jmp f1+3Ah (0040103A)
21  00401035  B8 01 00 00 00  mov eax, 1
 …  …         …                 …
26  00401040  3B EC           cmp ebp, esp
 …  …         …                 …
30  0040104A  C3              ret
```

其中，机器级代码行包括行号、虚拟地址、机器指令和汇编指令，计算机 M 按字节编址，int 型数据占 32 位。请回答下列问题。

（1）计算 f(10) 需要调用函数 f1 多少次？执行哪条指令会递归调用 f1？

（2）上述代码中，哪条指令是条件转移指令？哪几条指令一定会使程序跳转执行？

（3）根据第 16 行的 call 指令，第 17 行指令的虚拟地址应是多少？已知第 16 行的 call 指令采用相对寻址方式，该指令中的偏移量应是多少（给出计算过程）？已知第 16 行的 call 指令的后 4 字节为偏移量，M 采用大端方式还是小端方式？

（4）$f(13) = 6\,227\,020\,800$，但 f1(13) 的返回值为 1 932 053 504，为什么两者不相等？要使 f1(13) 能返回正确的结果，应如何修改 f1 的源程序？

（5）第 19 行的 imul 指令（带符号整数乘）的功能是 R[eax] ← R[eax] × R[ecx]，当乘法器输出的高、低 32 位乘积之间满足什么条件时，溢出标志 OF=1？要使 CPU 在发生溢出时转异常处理，编译器应在 imul 指令后加一条什么指令？

不变量·call 位移用后继地址校验

五字节 call 从 00401025H 开始，后继为 0040102AH；跳到 00401000H 所需位移为 −42，编码 FFFF FFD6H。把目标减起点会少算 5B。将求出的基准加位移回代，必须恰好恢复目标地址。

解析。先追踪源程序的调用层次，再读指令对 PC 和数据的作用。表中的省略号表示原题未展示的指令，行号差并不等于字节数。

（1）调用参数依次为 10、9、…、1，共 10 次，包含最外层调用；函数内部第 16 行 call 实际递归执行 9 次。参数为 1 时，第 12 行转到返回 1 的分支，不再递归。各层的返回位置虽都是调用点的后继地址，但栈帧和参数相互独立，返回时逐层乘回 2、3、…、10。

（2）第 12 行 jle 是条件转移，根据比较结果决定是否转移。第 16 行 call、第 20 行 jmp、第 30 行 ret 在执行时都会改变顺序后继：分别是调用、无条件跳转和返回。第 15 行 push 保存参数，本身并不转到 f1 入口。

（3）call 从 0040 1025H 开始，占 1 字节操作码及 4 字节位移，共 5 B，所以后继地址为

$$\texttt{00401025H} + 5 = \texttt{0040102AH}.$$

按后继地址为相对基准，位移为 $\texttt{00401000H} - \texttt{0040102AH} = -\texttt{2AH} = -42$，其 32 位补码为 FFFF FFD6H。表中后 4 字节按低地址到高地址为 D6、FF、FF、FF，低有效字节先存，因此为小端。复核：

`0040102AH` + (−42) = `00401000H`。不能用 call 自身起点减目标，也不能把 D6 当成正 214。

（4）13! 超过 32 位 int 上界 $2^{31}-1$。题给返回值是保留低 32 位后的机器结果：$6\,227\,020\,800 - 2^{32} = 1\,932\,053\,504$，它虽为正数，仍是溢出后的错误数学结果。标准 C 并没有规定有符号溢出必须这样回绕；这里解释的是原题给定的机器执行结果。

一种修改是把函数返回类型改为至少 64 位的 `long long`，使递归返回值及与 n 的乘法都按该较宽类型进行：

```
long long f1(int n) {
    if (n > 1)
        return n * f1(n - 1);
    return 1LL;
}
```

13! 可以用该类型精确表示。若只是先在 int 中乘完，再把溢出结果强制转换为宽类型，已经丢失的高位无法恢复。这里仅保证题目所问的 f1(13)，不意味着任意大的阶乘都能用固定 64 位整数表示。

（5）令精确 64 位乘积为高 32 位 H 与低 32 位 L。若 H 恰是 L 最高位的 32 位符号扩展，即 L 的符号位为 0 时 H 全 0、为 1 时 H 全 1，则截为 32 位仍保留真值，OF=0；否则 OF=1。等价地，乘积从第 63 位至第 31 位的高 33 位必须全 0 或全 1 才不溢出。只检查 H 是否为 0 并不够：可表示的负积需要 H 全 1。

题目要求的是发生溢出时转异常处理，因此应在 imul 后安排检查 OF 的**溢出自陷指令**：OF=1 时发出溢出异常并转入处理程序。普通条件转移 `jo` 能根据 OF 跳往一段代码，但跳转本身并不等于完成了异常入口动作。乘法器设置标志、指令检查标志、异常机制保存返回状态并转入口，是相接的三个步骤。

4.7 用语义、地址与状态三张清单读指令

机器级代码五步法

1. 列 ISA 给出的每条指令语义，区分它读寄存器、读主存、写主存还是只改 PC。
2. 标指令起址和字节长度，按题设确定顺序后继与相对基准。
3. 写有效地址，保留位移符号、索引比例和寻址级数。
4. 对分支列出真假两条路径；对调用单独记录返回地址、SP、FP 及需保存的值。
5. 用零次循环、一次循环、嵌套调用和负位移回代，最后数动态执行次数与访存。

4.7.1 扩展操作码为什么要保留前缀

译码互斥与编码空间

若某短操作码已被认作一条完整指令，同一前缀后的位就按其操作数字段解释；再把该前缀延长当作另一操作码，会让同一位串出现两种译码。因此扩展编码必须先留下未使用短前缀，再把它们的后续位作为更长操作码。每留下一个前缀并扩展 k 位，得到 2^k 个子前缀。逐层扣除已用子前缀，正是原例的容量算法。

不变量·编码数先核层级

扩展 k 位使一个保留前缀分裂成 2 的 k 次方个子前缀，已占用短格式对应的是整片子树。每一步统一数同样长度的前缀，才不会把指令条数与完整操作数字段位串混在一起。

□

课程连接·栈模型接到数据结构

栈的后进先出性质保持不变，但机器 SP 到底指向当前元素还是下个空位属于实现约定。数据结构中的 push 抽象不直接决定地址先减还是先写。题目给出约定后，再逐步画 SP 轨迹。

4.7.2 地址字段数不等于实际操作数数

隐含寻址不在指令中显式写出某个操作数的位置，例如约定累加器或栈顶。零地址算术指令仍可从栈顶取得两个操作数并把结果送回栈；只是这些位置由机器约定确定。一地址指令也可能隐含累加器，故题目问地址码个数时，不能直接回答参与运算的值有几个。

堆栈寻址以 SP 和约定确定元素位置。满栈/空栈、向低/高地址增长、先改 SP 还是先访存，都会改变读写地址。基址、变址和相对寻址虽然常都出现 “寄存器 + 位移” 的代数式，寄存器角色分别服务于重定位、数组下标与控制转移；代数相同不代表使用约定可互换。

4.7.3 指令类别与读写副作用

数据传送类把值在寄存器与存储器等位置间移动；算术逻辑类形成新值及可能的标志；移位类改变位位置；顺序控制类决定后继 PC；I/O 及系统控制类涉及端口、状态与受保护操作。具体指令是否改标志、访存或具有特权，须查给定语义，不能仅凭助记符字母猜测。

机器码解读和 C 语义需要两层核对。机器指令可以按规定生成截断结果，C 程序是否允许这种执行仍受溢出、对象范围与类型规则约束。把一次具体机器执行轨迹说清楚，并不等于证明对应 C 代码在所有实现上都有同样结果。

4.8 地址个数、指令长度与访存次数

地址字段数描述指令中显式给出的地址个数。三地址指令可分别给出两个源和一个结果地址；二地址指令常把一个源地址同时作为目的地址；一地址指令可隐含累加器；零地址指令可以不使用操作数，也可以隐含使用栈顶操作数。分析 “几个操作数” 时应包括隐含操作数，分析 “几个地址字段” 时只数编码字段。

若操作数和结果均在主存、指令一次取出，三地址运算可能需要取指 1 次、读源 2 次、写结果 1 次，共 4 次访存；寄存器操作数、间接寻址或多字指令都会改变次数。因此先列执行动作，再累计访问次数，不能把地址数直接当成访存次数。

指令字长、机器字长和存储字长分别描述指令编码、CPU 数据处理宽度和存储单元宽度。若存储字长 16 位，一条 32 位指令跨两个存储字，取指可能需要两次存储器访问；是否还能合并为一次总线事务，取决于具体存储组织。

4.8.1 扩展操作码的完整分配例

设 16 位指令，每个地址字段 4 位，要求 15 条三地址、12 条二地址、62 条一地址和 32 条零地址指令。基本 4 位操作码共有 16 种，先分配 15 种给三地址，剩余 1 种前缀扩展为 16 种 8 位操作码；分配 12 种给二地址，剩余 4 种再扩展为 64 种 12 位操作码；分配 62 种给一地址，剩余 2 种扩展为 32 种 16 位零地址操作码。

每次可继续扩展的数量为 “本层保留前缀数 $\times$ 下一段编码组合数”。已经分配的短操作码不能再作为其他操作码的前缀，否则译码器无法确定后续位是地址还是操作码。这里指令字长固定，变化的是操作码和

地址字段之间的分配。

课程连接·形式地址与有效地址

形式地址是指令字段，有效地址是计算后访问操作数的位置；立即寻址的字段本身表示操作数。

4.9 数据寻址方式的统一比较

下面把指令中的形式地址记为 A，寄存器内容记为 (R_i)，存储器单元内容记为 $M[A]$。执行阶段访存次数仅统计读取一个源操作数，取指和写回另计。

寻址方式	操作数或有效地址	执行阶段读主存次数
立即	操作数就是 A	0
直接	EA $=A$，操作数 $M[A]$	1
一次间接	EA $=M[A]$	2
寄存器	操作数为 (R_i)	0
寄存器间接	EA $=(R_i)$	1
相对	EA $=\mathrm{PC}_{\text{base}}+\mathrm{sext}(A)$	依操作数位置确定
基址	EA $=(\mathrm{BR})+A$	内存源通常 1 次

寻址方式	操作数或有效地址	执行阶段读主存次数
变址	EA $=(\mathrm{IX})+A$	内存源通常 1 次

相对寻址以题设规定的 PC 值为基准。常见机器用下一条指令地址作基准，位移为补码；若位移按字、半字对齐，还须先按约定缩放。基址寻址把一段程序或数据的起始位置交给基址寄存器，适合重定位；变址寻址把数组起点放在地址字段、元素变化量放在变址寄存器，适合循环访问。二者都表现为加法，寄存器含义和管理方式却不同。

堆栈寻址隐含使用 SP。采用向低地址增长、SP 指向栈顶元素的 32 位字栈时，压栈先使 SP 减 4 再写入，弹栈先读出再使 SP 加 4。若题目采用空栈顶指针或向高地址增长的约定，应重新写出操作次序。寄存器组组成的硬堆栈与主存组成的软堆栈在访存开销上不同。

4.10 x86 汇编：操作数顺序与宽度先行

课件采用 32 位 x86 示例，EAX、EBX、ECX、EDX 是 32 位通用寄存器；AX、AL、AH 分别对应 EAX 的低 16 位、最低 8 位和次低 8 位；ESP 和 EBP 常用于栈顶及栈帧基址。这里的具体寄存器约定属于示例 ISA，不能外推成所有机器的统一规则。

含义	Intel 格式	AT&T 格式
将立即数 5 送 EAX	`mov eax, 5`	`movl $5, %eax`
将 EBX 送 EAX	`mov eax, ebx`	`movl %ebx, %eax`
读地址 EBX+ECX×4+8 处 32 位数	`mov eax, dword ptr [ebx+ecx*4+8]`	`movl 8(%ebx,%ecx, 4), %eax`

Intel 格式目的在前、源在后；AT&T 格式源在前、目的在后，寄存器前有%，立即数前有 `$`，常用 b、w、l 后缀指定 8、16、32 位宽度。x86 的地址比例因子为 1、2、4 或 8，超出该集合的乘法不能直接当成这一寻址编码的比例因子。

`mov` 复制数据；`lea` 计算有效地址的算术表达式并把地址值送寄存器，不读取该地址中的数据；`add/sub` 修改目的数；`and/or/xor` 作逐位逻辑运算；`cmp` 计算减法标志而不保存差；`test` 计算与运算标志而

不保存按位与结果。移位时应区分 `shl/sal`、`shr` 和 `sar`，最后一个按算术右移保留符号扩展。

课程连接·调用边界

返回地址、保存的寄存器和局部变量各有职责；栈增长方向和参数位置必须采用题中调用约定。

4.10.1 条件转移如何解释相同标志位

以 Intel 格式 `cmp eax, ebx` 为例，标志来自 EAX−EBX。相等看 ZF；无符号小于看 CF；无符号小于等于看 CF 或 ZF；有符号小于看 SF $\neq$ OF；有符号小于等于看 ZF 或 SF $\neq$ OF。有符号大于要求 ZF=0 且 SF=OF，无符号大于要求 CF=0 且 ZF=0。

同样的寄存器位串可以被程序当作有符号或无符号数，硬件减法结果相同，所选分支条件不同。`loop` 在 32 位计数约定下先把 ECX 减 1，再判断是否非零，它本身不等价于“修改 ZF 后的普通 dec 指令”；在分析后续标志位时要遵守具体指令的副作用定义。

4.10.2 用机器级控制流还原循环

```
mov ecx, 0
mov eax, 0
L_test:
    cmp ecx, 10
    jge L_done
    add eax, dword ptr [ebx+ecx*4]
    add ecx, 1
    jmp L_test
L_done:
```

若 EBX 指向 10 个 32 位整数且运算结果在目标类型范围内，这段代码将数组元素累加到 EAX。读程序时依次确定初始化、循环条件、循环体和更新。条件转移写在循环开头时可能一次也不执行；写在循环末尾并由循环体起步时通常至少执行一次，二者对应的高级语言结构不同。

4.11 函数调用：参数、返回地址和栈帧

采用课件中的 32 位、向低地址增长的栈，以及常见的 EBP 栈帧形式：调用者安排参数并执行 `call`；`call` 把下一条指令地址压栈，再跳到被调用函数。被调用者执行 `push ebp; mov ebp, esp`，保存旧帧基址并建立新帧；需要局部空间时再减少 ESP。

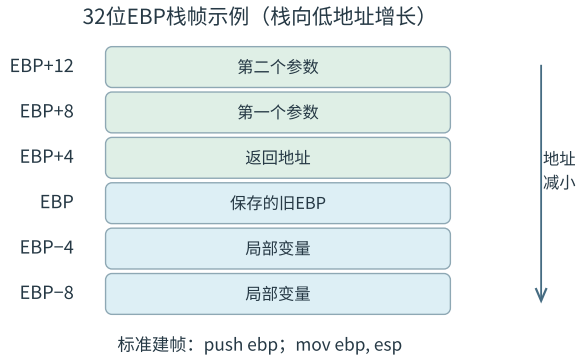


Figure 4.2: 32 位示例栈帧中，返回地址与参数位于 EBP 上方，局部变量位于下方

典型布局中，`[ebp]`为旧 EBP，`[ebp+4]`为返回地址，`[ebp+8]`、`[ebp+12]`为前两个栈参数，局部变量位于 `[ebp-4]`等位置。参数次序、哪些寄存器由谁保存、由谁回收参数空间，属于调用约定；这里按上述教学模型分析，不把某一布局当作全部编译器和优化级别的固定形式。

函数返回前常用 `mov esp, ebp; pop ebp` 恢复调用者栈帧，随后 `ret` 弹出返回地址到指令指针；常见整数返回值放在 EAX。递归调用每层拥有独立的返回地址和必要局部状态；全局变量则由各层共享。计算递归空间时应乘以最大同时存活的调用层数，而不是总调用次数。

4.11.1 计算与辨析自查

1. 16 位指令的三地址操作码为何最多分配 15 种后还能继续扩展？2. `lea eax,[ebx+4]`会读取地址 EBX+4 的内容吗？3. 有符号小于为什么不能只看 SF？4. 上述栈帧中返回地址位于哪里？

答案。1. 留出的一个 4 位前缀可再结合下一段 4 位形成 16 种长操作码。2. 不会，只计算地址值。3. 减法发生溢出时，结果符号与数学差的符号可能不同，须结合 OF。4. `[ebp+4]`，前提是已经执行标准建帧序列。

4.12 主动回忆自测

4.12.1 配套自测

1. 原创补充：沿用本章栈模型，初始 SP=1000H，连续 push 两个 4 B 值后，再 pop 一个值，SP 是多少？返回地址保存在链接寄存器时，是否每次调用都必然产生一次主存写？

答案。1. SP=0FFCH。否，是否写主存还取决于调用模型及是否需要溢出保存。

4.12.2 遮住答案复述知识关系

主动回忆·自拟

2. 自拟：说明零地址加法如何仍有两个源操作数；哪部分信息被隐含了？
3. 自拟：某 2B 转移指令从 2000H 开始，取完 PC 为 2002H，8 位

课程连接·按语义复查

地址计算、字节序和标志位分别检查，避免由指令名字直接猜数值结果。

补码位移 FAH。目标地址是多少?

4. 自拟：指令和操作数都占一个存储字，无 Cache 无预取，直接读操作数、一次主存间接读操作数、寄存器间接读操作数，各至少几次主存访问?
5. 自拟：原条件为 x<y，为何实现 if/else 时常先用 x>=y 跳到 else? 如果改成 unsigned，分支条件的解释要怎样改?
6. 自拟：返回地址存在链接寄存器时，递归调用为什么仍可能需要额外保存？调用后可以直接返回函数入口吗?

课程连接·按语义复查

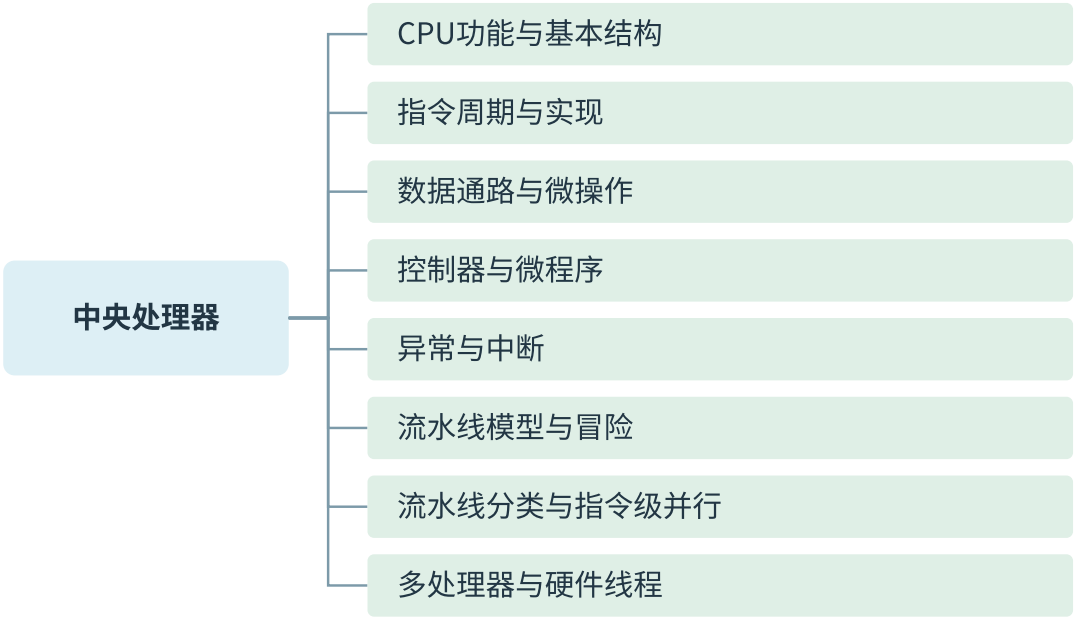
地址计算、字节序和标志位分别检查，避免由指令名字直接猜数值结果。

4.12.3 参考答案

2. 源来自约定的栈顶两项，目的为约定栈顶；指令没有显式地址字段，不表示没有操作数。
3. FAH 为 -6，目标 $2002H - 6 = 1FFCH$。
4. 分别至少 2、3、2 次：都需一次取指；主存间接额外读一次地址，寄存器间接的地址已在寄存器。
5. 取原条件的否定，使满足原条件者顺序进入 then，其他跳到 else。unsigned 须采用无符号比较条件，不能直接复用有符号标志组合。
6. 再次调用会覆盖链接寄存器，须在需要返回的旧值被覆盖前保存。返回到调用点后继地址，而非函数入口；各层栈帧与返回位置必须匹配。

中央处理器 5

章节脉络



课程连接·先看一次指令

跟踪指令从取指、译码、执行到可能的访存与写回；每一步由控制信号选择相应数据通路。

5.1 从指令语义到执行组织

课件先用 CPU 功能与结构图识别寄存器、运算器和控制器，再用指令执行导图连接取指、间址、执行和中断阶段。本章沿这条路径先解释一条指令怎样得到正确结果，随后把流水线、多发射、多处理器与硬件线程接入同一章，始终保留“值在何时产生、资源在何时被占用”的约束。

定义 5.1.1 (CPU 的功能与实现层次)

CPU 承担指令控制、操作控制、时间控制、数据加工和规定的异常/中断处理。运算器处理数据，控制器解释并组织执行，寄存器保存跨时刻状态；数据通路承载数据传送与运算，控制部件决定何时、按何种方式使这些动作生效。

ALU、累加器 ACC、通用寄存器组、移位器和计数器等可出现在运算部件的教学模型中；PC、IR、指令译码器、时序系统和微操作信号发生器等常放在控制器模型中。实际 CPU 划分依实现；例如 PSW 既包含运算标志，也包含处理器状态，不能仅凭图上位置认定其全部位都允许用户程序任意写。

课件把寄存器分为程序员可见和实现内部两层：通用寄存器、PC 及规定的程序状态可进入体系结构描述；MAR、MDR、IR 和临时寄存器通常属于实现内部。这里“可见”指软件模型的一部分，是否可由某条用户指令直接读取或写入仍由 ISA 与权限规定。

5.2 寄存器的名字说明它暂存什么

最小反例·地址宽与数据宽可不同

若按字节编址、可寻址 64MiB，地址需要 26 位；若一次访存交换 32 位，MDR 可为 32 位。MAR 为 26 位而 MDR 为 32 位完全自洽，不能因二者都接主存就要求等宽。

CPU 的基本任务是取指、解释指令并控制操作执行，同时处理规定的异常和中断。运算部件完成算术逻辑处理，寄存器保存当前操作需要的状态，控制器安排数据通路中的动作。每个部件的作用要放到一次实际传送中理解。

寄存器	保存内容及用途
PC	与后继取指位置有关的地址；更新规则由执行时序决定
IR	当前正在译码或执行的指令编码
MAR	本次主存访问的地址
MDR	与主存交换的指令或数据
通用寄存器组	指令能按编号访问的操作数、地址或临时结果
PSW/标志寄存器	运算标志及体系结构规定的处理器状态

“寄存器在 CPU 内” 不等于 “它属于通用寄存器组”。PC、IR 与通用寄存器的用途不同；MAR、MDR 在题目的抽象模型中标明访存接口，但这并不意味着所有现代芯片都用同样的名字设置独立物理寄存器。

MAR 位数由可寻址单元数决定，MDR 位数由该数据通路一次访存交换的位宽决定。若数据总线 32 位、按字节编址且有 26 位地址，MDR 可以为 32 位而 MAR 为 26 位，两者无须相等。MDR 刚完成取指时存指令，刚完成数据读时存操作数，它的内容随本次访存的类型改变。

5.2.1 例 5-1　哪些元件负责运算，哪些元件保存状态

例 5.2.1 (配套辨析)

【配套例题】数据通路由组合逻辑元件（操作元件）和时序逻辑元件（状态元件）组成。以下给出的元件中，属于操作元件的是（　）。

I. 算术逻辑部件（ALU）　II. 程序计数器（PC）
III. 通用寄存器组（GPRs）　IV. 多路选择器（MUX）

A. 仅 I、II　B. 仅 I、IV　C. 仅 II、III　D. 仅 I、II、IV

解析。ALU 按当前输入产生运算输出，MUX 按选择信号把某路输入送往输出，二者是组合逻辑。PC 和通用寄存器组保存跨时钟的状态，属于状态元件，选 **B**。寄存器组内部也有组合读出或选择电路，但这不改变它作为状态元件的分类。

5.3 指令周期、机器周期与时钟周期

时钟周期是同步 CPU 中状态推进的基本时间单位。指令周期是从一条指令开始执行到完成所需的整个过程。机器周期是某些实现中为取指、访存等基本操作划分的较大时间单位，不同机器的划分不同，不应把它当成跨体系结构固定的常数。

指令周期可能含取指、间址、执行、中断等阶段；不是每条指令都经过所有阶段。寄存器指令无需为操作数访问主存，间接寻址可能增加取有效地址的阶段，外部中断也只有在满足响应条件时才进入响应过程。

单周期 CPU 把一条指令的全部操作安排在一个时钟周期内，所以时钟必须容纳最慢指令的数据通路；简单指令也使用同样长的周期。多周期 CPU 把指令拆成若干步，周期由各步关键路径决定，不同指令使用不同步数。流水线让多条指令的不同阶段重叠，减少连续指令的平均完成间隔，但未必缩短一条指令自身的延迟。

比较维度	单周期	多周期	简单标量流水线
同一时刻处理的指令	一条	一条的当前步骤	多条各在不同阶段
时钟受什么路径约束	最慢完整指令	最慢基本步骤	最慢流水段及段寄存器开销
一条指令占几拍	1 拍，但该拍较长	随所需步骤变化	通常跨多拍；稳态可每拍完成一条
同一运算资源能否复用	同拍需要的不同用途要避免争用	可跨步骤复用	重叠阶段同拍所需资源要避免争用

例如 PC 增量和数据加法在同一长周期内都需要求和时，不能仅因“单周期一拍”就假定一个无复用路径的 ALU 能同时完成两项操作；多周期可以在不同拍复用，流水线则要核查两者是否会同时发生。CPI=1 的单周期与稳态 CPI 约 1 的流水线可能有完全不同的时钟长度，比较性能仍须回到执行时间。

若题目给“每次访存 2 周期”，应把它安排到取指和操作数读写等实际位置；不能拿“某类指令固定 3 个机器周期”代替具体数据通路的资源分析，实际步数仍以资源限制为准。

不变量·总线同拍只有一个驱动源

单内部总线上，R1 送 Y 与 R2 送 MAR 要求两个不同值同时驱动，发生冲突；R1 同时送 Y 和 MAR 则可能允许，前提是两目标都具备写入路径和独立写使能。

5.4 一条指令怎样变为微操作

微操作是数据通路能完成的基本动作，例如 PC 送 MAR、两寄存器值相加、结果写回。控制信号选择输出源和 ALU 功能，允许某个寄存器写入，或发出存储器读写请求。微操作是动作，微命令是发出该动作的控制命令，两者不是同一个对象。

下例采用单条 CPU 内部总线。一次总线传送只允许一个驱动源，但同一值可以同时送入多个目的寄存器。ALU 的一输入来自暂存器 Y，另一输入来自总线，结果先存 Z，再由 Z 上总线写回。这里先按图确认资源，再写时序，不另补未画出的专用路径。

> **按图排微操作**
>
> 先把机器指令写成寄存器传送语义；再列出每一步的源、目的和可用路径；同拍检查总线驱动冲突、端口数量及写使能；最后加入访存等待与结果可用时刻。若题目没有画某条旁路，就不能为了缩短周期自行增设。

5.4.1 例 5-2　从单总线图确定控制信号与取指时序

【配套例题】某 CPU 中部分数据通路如下图所示，其中，GPRs 为通用寄存器组；FR 为标志寄存器，用于存放 ALU 产生的标志信息；带箭头虚线表示控制信号，如控制信号 Read、Write 分别表示主存读、主存写，MDRin 表示内部总线上数据写入 MDR，MDRout 表示 MDR 的内容送内部总线。

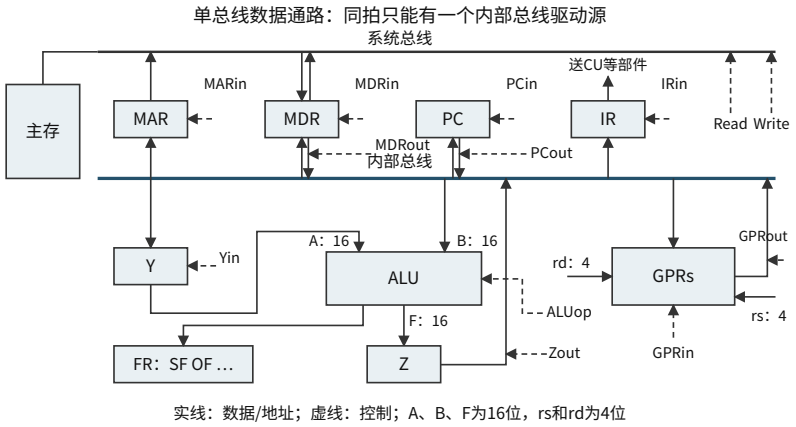


Figure 5.1: 单条内部总线连接 Y、ALU 与 Z；rs、rd 为 4 位寄存器编号

（1）设 ALU 的输入端 A、B 及输出端 F 的最高位分别为 A15、B15 及 F15，FR 中的符号标志和溢出标志分别为 SF 和 OF，则 SF 的逻辑表达式是什么？A 加 B、A 减 B 时 OF 的逻辑表达式分别是什么？要求逻辑表达式的输入变量为 A15、B15 及 F15。

（2）为什么要设置暂存器 Y 和 Z?

（3）若 GPRs 的输入端 rs、rd 分别为所读、写的通用寄存器的编号，则 GPRs 中最多有多少个通用寄存器？rs 和 rd 来自图中的哪个寄存器?已知 GPRs 内部有一个地址译码器和一个多路选择器，rd 应连接地址译码器还是多路选择器?

（4）取指令阶段（不考虑 PC 增量操作）的控制信号序列是什么?若从发出主存读命令到主存读出数据并传送到 MDR 共需 5 个时钟周期，则取指令阶段至少需要几个时钟周期?

（5）图中控制信号由什么部件产生？图中哪些寄存器的输出信号会连到该部件的输入端?

解析。(1) SF 直接记录结果最高位，所以 SF $= F_{15}$。OF 判断有符号运算是否越界。为简写，以下令 $a = A_{15}, b = B_{15}, f = F_{15}$，乘号表示逻辑与、加号表示逻辑或、上横线表示逻辑非：

$$\mathrm{OF}_{\mathrm{add}} = ab\overline{f} + \overline{a}\,\overline{b}f, \qquad \mathrm{OF}_{\mathrm{sub}} = a\overline{b}\,\overline{f} + \overline{a}bf.$$

加法只在两输入同号、结果异号时溢出；减法只在两输入异号、结果符号不同于 A 时溢出。例如 16 位 32767 加 1 得到 8000H，a=b=0、f=1，代入加法表达式得 OF=1；−32768 减 1 得到 7FFFH，a=1、b=0、f=0，代入减法表达式也得 1。SF 只保存结果的最高位，并不保证有符号运算的结果在数学上正确。

（2）同一时刻内部总线只能提供一个值，而 ALU 要同时接收两个输入。先把第一个输入存 Y，总线随后送第二个输入到 B 端，Y 提供 A 端。ALU 产生结果时总线还在送第二输入，若直接把结果也送回同一总线会争用；先用 Z 保存，下拍再由 Z 输出。Y 解决两个输入先后取得的问题，Z 解决结果输出的时机问题；二者并不是两个名字不同的通用寄存器。

（3）rs 和 rd 均为 4 位编号，最多 $2^4 = 16$ 个通用寄存器。它们来自指令寄存器 IR 中的对应字段。rd 决定本拍把结果写入哪一个寄存器，接写地址译码器；rs 决定从哪个寄存器读出，控制读出多路选择器。4 位编号能选 16 个寄存器，不表示每个寄存器只有 4 位。

（4）先用 PC 提供指令地址，访问主存，最后将读出的指令送 IR：

最小反例·同一动作不可重复计延迟

若题给从 Read 到主存数据送 MDR 共 5 拍，这 5 拍已经含送入 MDR。再加一次 “写 MDR” 会重复计数；但随后 MDR 经内部总线写 IR 仍是另一项真实传送。

控制步骤	有效控制信号	数据变化与所需时间
地址准备	PCout、MARin	PC 经内部总线写 MAR，1 周期
主存读	Read	主存按 MAR 地址读出，送 MDR，共 5 周期
装入指令	MDRout、IRin	MDR 经内部总线写 IR，1 周期

错项排除·编码对象先分清

32 条机器指令只需 5 位区分操作码，不说明控制存储器地址也只需 5 位。公共 2 步加每指令 4 步共有 130 位置，微地址需要 8 位；操作码、微地址和控制位数分别统计。

因此至少 $1+5+1=7$ 个时钟周期。5 周期已包含主存读出并送 MDR，无须再加一次 “写 MDR” 的额外周期。图中 MDRin 专指从内部总线写 MDR，并不出现在这条外部主存读入路径上。题目明确不考虑 PC 增量，也不必为更新 PC 多加一步。

（5）控制信号由控制器中的控制部件 CU 产生。IR 输出指令的操作信息，FR 输出 SF、OF 等状态，二者共同为控制决策提供输入。通用寄存器保存的是程序数据，PC 提供执行位置，不能因为它们在 CPU 内，就把所有寄存器都列为本图 CU 的判断输入。

5.4.2 例 5-3　间接操作数经过哪些部件

【配套例题】某指令功能为 $\mathrm{R}[r2] \leftarrow \mathrm{R}[r1] + \mathrm{M}[\mathrm{R}[r0]]$，其两个源操作数分别采用寄存器、寄存器间接寻址方式。对于下列给定部件，该指令在取数及执行过程中需要用到的是（　）。

I. 通用寄存器组（GPRs）　II. 算术逻辑单元（ALU）
III. 存储器（Memory）　IV. 指令译码器（ID）

A. 仅 I、II　B. 仅 I、II、III　C. 仅 II、III、IV　D. 仅 I、III、IV

解析。r1 的值和 r0 中的地址来自寄存器组；按 r0 值访问存储器得到第二个操作数；ALU 做加法并写回 r2，所以 I、II、III 必需，选 **B**。题目已限定 “取数及执行”，指令译码是此前确定操作的工作，不能把整个指令周期使用过的所有部件都加进来。

解数据通路题时，先写出指令的寄存器传送语义，再用图上的实际路径连接源和目的，最后安排周期。总线冲突、写入条件和内存延迟通常比 “背一张步骤表” 更能决定答案。不要为了省周期自行补出未画出的旁路。

5.5 控制信号由组合逻辑还是微程序产生

硬布线控制器根据操作码、当前节拍和状态条件，经译码与逻辑电路产生控制信号。它不必逐条读取控制存储器中的微指令，常有较短控制延迟；新增复杂指令可能需要修改更多逻辑。

微程序控制器把一条机器指令的控制过程组织为微指令序列，保存在控制存储器中。微指令的控制字段产生本拍的微命令，后继地址及条件字段决定下一条微指令。控制存储器通常是 CPU 内部的专用存储器，不是用户主存；微程序计数器或微地址寄存器也不是机器程序的 PC。

两个控制器实现同一 ISA 时，对同一机器指令应产生等价的软件可见结果；差别在于控制序列由逻辑电路直接形成，还是从控制存储器取出微指令形成。机器指令的操作码用于选择一种指令功能，微地址用于选择控制序列中的一步，微命令控制具体部件动作。因而 “机器指令有 32 条”“控制存储器有 130 个位置”“本拍需发 5 条微命令” 数的是三个不同对象，各自的位数不能互相套用。

微指令的直接控制方式为每条控制线分配一位，便于同拍并行，但字宽较大。字段直接编码把互斥微命令分组，每字段经译码选一个命

令，需要保留 “不发该组命令” 的编码。字段间接编码还借助其他字段共同解释。水平型倾向于显式并行多个微命令，垂直型以较短编码表示操作，往往需要更多微指令才能完成同一控制任务；不能笼统地说后者并行性更高。

最小反例·4 条命令不只 4 种状态

一个互斥字段含 4 个微命令，仍需 “不发该组命令” 的第 5 种状态，所以通常至少 3 位。互斥只限制同字段同时发一条，不能省掉本拍暂不用该字段的状态。

5.5.1 微程序控制器的状态怎样推进

控制存储器 CM 保存微指令；微地址寄存器 CMAR（也常称微程序计数器）保存下一次要访问的微地址；读出的微指令进入 CMDR/微指令寄存器，其中控制字段经直接控制或译码发出微命令。后继逻辑依据下地址字段、条件和节拍形成下一微地址。CMAR 与程序 PC 分别指向控制存储器和机器程序空间，不能共用 “指令地址” 的名称而忽略所属层次。

后继微地址可由断定法读取下地址字段，也可顺序递增；机器指令操作码用于确定执行微程序入口，取指或中断等公共微程序入口可由硬件规则给出。课件用固定 0 号作为公共取指入口，是该教学模型的约定，不是所有微程序 CPU 的统一物理地址。

水平、垂直与混合微指令是控制字组织的分类。直接控制用较多位明确允许并行微命令；字段编码用较短编码选择互斥动作；混合形式在紧凑编码与并行控制之间折中。“字短” 不等于 “一条机器指令执行更快”，应比较微周期数与各微周期时长。

5.5.2 例 5-4　互斥类为什么还要一个空操作编码

【配套例题】某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有 33 个微命令，构成 5 个互斥类，分别包含 7、3、12、5 和 6 个微命令，则操作控制字段至少有（　）。

A. 5 位　B. 6 位　C. 15 位　D. 33 位

解析。每组除了要能发出其中一条命令，还要能一条都不发。各组位数为 $\lceil \log_2(7+1) \rceil = 3$、2、4、3、3，总计 15 位，选 **C**。例如若某组有 4 条命令，2 位只有 4 种编码，没有 “本拍不用该组” 的位置，通常要 3 位。

5.5.3 例 5-5　控制存储器地址字段怎样估位宽

【配套例题】某计算机采用微程序控制器，共有 32 条指令，公共的取指令微程序包含 2 条微指令，各指令对应的微程序平均由 4 条微指令组成，采用断定法（下地址字段法）确定下条微指令地址，则微指令中下地址字段的位数至少是（　）。

A. 5　B. 6　C. 8　D. 9

解析。公共取指微程序只存一份，控制存储器至少需 $32\times4+2=130$ 个微指令位置，地址至少 $\lceil \log_2 130 \rceil = 8$ 位，选 **C**。32 种机器指令只需要 5 位编码，不表示微指令的存储地址也只需 5 位；两者编码的对象不同。

取指虽然不靠 “上一条指令的操作码” 决定读哪条指令，但仍需要控制信号来选择 PC、发出存储器读并写入 IR。流水线图有时省略这些固定发生的控制动作，但这不代表取指、译码电路不需要任何控制。题目说 “数据通路不包含控制部件” 时，是按功能划分数据处理通路与信号生成部分；这也不意味着数据通路能脱离控制运行。

5.6 异常、中断与返回位置

课程连接·缺页使当前指令重试

OS 调入缺页后，原访问尚未完成，处理器通常从该指令重试；设备完成中断对应一条已完成指令的边界。返回点取决于事件与 ISA 的精确状态规则，不由当前取指 PC 猜测。

本讲义按 408 的常用语境区分：异常与当前执行的指令有关，在执行过程中检测，例如缺页、非法指令、算术异常；外部中断来自指令执行之外的事件，例如设备完成、定时器请求，通常在指令边界接受响应。广义的 “中断机制” 有时统称两者，遇到题目要先辨清其用词范围。

故障 fault 通常可在修复后重新执行，例如缺页后调入页面；陷阱 trap 是有意发出的通知或调试事件，处理完后通常从后继位置继续；终止 abort 表示严重情况，通常不能可靠恢复。不同 ISA 的异常分类和保存 PC 的细则不同，此处不把每一种算术异常固定归入同一类别。

响应异常或中断时，硬件需保存足够的返回信息、切换到规定状态并找到处理程序入口。通用寄存器现场往往由入口汇编代码和软件继续保存。中断服务例程是软件；“中断隐指令” 则是对硬件自动响应动作的教材称呼，不是程序员在代码中插入的一条普通指令。

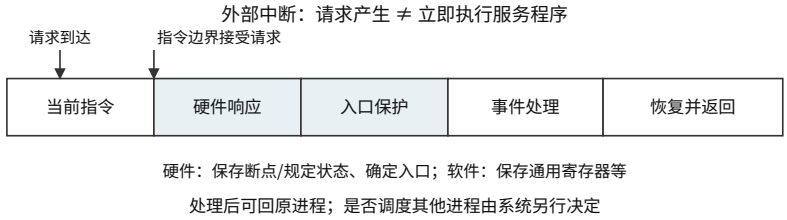


Figure 5.2: 外部请求先等待响应边界，再由硬件入口转入软件处理

5.6.1 例 5-6　缺页后为什么重试当前指令

【配套例题】异常事件在当前指令执行过程中进行检测，中断请求则在当前指令执行后进行检测。下列事件中，相应处理程序执行后，必须回到当前指令重新执行的是（　）。

A. 系统调用　B. 页缺失　C. DMA 传送结束　D. 打印机缺纸

解析。页缺失意味着当前指令需要访问的页面尚未就绪，指令的原定访问还未完成。处理程序调入页面后，应重新执行这条指令，选 **B**。系统调用已经发出了服务请求，不能不加区分地重做调用指令；DMA 完成和打印机事件来自外设，不要求重做某条已完成的普通指令。

外部中断的返回地址通常是下一条尚未执行指令的位置，不能写成 “刚执行完的那一条”。发生故障时可能保存当前指令起点，发生陷阱时可能保存后继位置。给出流水线或变长指令的题目，还要看体系结构怎样保证可恢复状态，不能单凭此时取指 PC 的数值推断精确断点。

处理器进入内核态不必切换到另一个进程。一次设备中断可能处理后返回原进程；调度器也可能决定切换执行主体。模式切换与进程上下文切换相关但不等价。第 7 章进一步讨论设备请求、屏蔽和中断嵌套。

5.6.2 基本控制巩固

1. 【配套例题】相对于微程序控制器，硬布线控制器的特点是（　）。A. 指令执行速度慢，指令功能的修改和扩展容易；B. 指令执行速度慢，指令功能的修改和扩展难；C. 指令执行速度快，指令功能的修改和扩展容易；D. 指令执行速度快，指令功能的修改和扩展难。
2. 原创补充:沿用本章单总线图,能否在一拍同时执行 `R1→Y`、`R2→MAR`? 能否同时执行 `R1→Y`、`R1→MAR`?

答案。1. D，比较的是教材所述常见控制实现的控制开销与修改难度。2. 前者不能，两源争用总线；后者可以，同一来源同时送两个写入端，只要各目的具备独立写使能。

不变量·第一条付满延迟，后续按间隔

理想 k 段、n 条指令总时长为 (k+n−1) 拍：第一条需 k 拍，剩余 n−1 条每拍完成一条。不能给每条都加一次填充延迟，也不能把阶段数直接当任何基线下的加速比。

5.7 延迟和完成间隔是两个量

把一条指令分为 k 段，每段使用自己的部件或占用自己的时段，让多条指令的不同阶段重叠执行，称为指令流水线。同一条指令内部的先后依赖依然存在：必须先取得地址，才能读取相应数据。它主要提高连续指令的吞吐率。

定理 5.7.1 (理想标量流水线的总时间)

理想 k 段流水线，每段耗时一个时钟周期 τ，无停顿且每周期启动一条指令，完成 n 条指令需

$$T_{\text{pipe}} = (k + n - 1)\tau.$$

由第一条延迟与后续完成间隔计数

前 k 个周期使第一条指令通过全线，此后每周期再完成一条。填充时间只在开始付一次，并非每条指令各付一次。吞吐率为 n/T_{pipe}，n 足够大时趋近 $1/\tau$；单条延迟约为 $k\tau$。加入停顿 s 周期后，总时间变为 $(k+n-1+s)\tau$；这里假定每个停顿周期都使最终完成时间推后一个周期。

□

若非流水方案中每条指令依次完整走完这 k 段、每段也占用同样的 τ，则非流水时间为 $nk\tau$，加速比为 $nk/(k+n-1)$。若原方案时钟不同、各段耗时不等或另有资源复用，基线要重新计算；流水级数本身并不等于加速比。

5.7.1 例 5-7　最慢的一段决定时钟下限

【配套例题】某计算机的指令流水线由四个功能段组成，指令流经各功能段的时间（忽略各功能段之间的缓存时间）分别为 90 ns、80 ns、70 ns 和 60 ns，则该计算机的 CPU 时钟周期至少是（　）。

A. 90 ns　　B. 80 ns　　C. 70 ns　　D. 60 ns

解析。相邻阶段寄存器在同一节拍推进，必须留足最慢段 90 ns，选 **A**。若实际阶段寄存器开销为 t_r，则时钟周期通常至少为 $\max_i(t_i) + t_r$；题目已说明忽略这项开销。

在本题时间下，理想完成 10 条指令需 $(4 + 10 - 1) \times 90 = 1170$ ns。若非流水方案每条直接耗时 $90 + 80 + 70 + 60 = 300$ ns，10 条为 3000 ns，加速比约 2.56；若基线规定每段都占 90 ns，基线时间又会不同。比较时必须明确分母采用哪种机器。

若每条指令都有效使用 k 个等时段，n 条共占用 nk 个“段 × 周期”，时空图总面积为 $k(k+n-1)$ 个“段 × 周期”，所以理想流水线效率为 $E = n/(k+n-1)$。有空闲段、停顿或不同功能指令时，应按实际有用面积重新计算，不能把这个理想式用于所有时空图。加速比比较两种方案时间，效率比较段的有效利用，二者不是同一个比例。

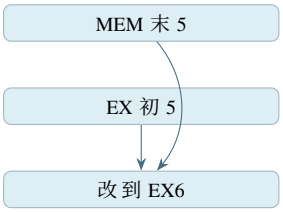
5.7.2 课件中的流水线分类

按使用层次，部件级流水线把算术运算拆段，处理机级流水线把指令过程拆段，处理机间流水线让多台处理机分担任务阶段。按功能可分单功能与多功能；按功能配置能否在同一时刻并行用于不同功能，分静态与动态流水线；按各段是否存在反馈回路，分线性与非线性。非线性流水线可能让一项操作重复经过某些段。

“动态流水线” 在上述分类中讨论功能配置，“动态调度” 则讨论就绪指令能否提前执行，不能仅因都含 “动态” 就当作同一技术。超标量、超流水与超长指令字又是多发射或细分阶段等技术维度，随后分别说明。

最小反例·尚未产生的值不能转发

LOAD 的值到第 5 拍 MEM 末才形成，下一条本应第 5 拍 EX 开头使用它，两者时刻不合。已有 MEM→EX 旁路仍需把消费者推至第 6 拍；“有转发” 不能抹去因果顺序。



同拍初早于同拍末，故需推迟消费者

5.8 阶段寄存器保存的不只是一份数据

五段教学流水线常写为 IF 取指、ID 译码/寄存器读、EX 执行或地址计算、MEM 访存、WB 写回。算术指令也可能经过 MEM 这一拍但不访问数据存储器；存储指令通常不在 WB 写通用寄存器。阶段名只描述一种通路划分，并不意味着每条指令都在各段做同一件事。

阶段之间的寄存器保存下一段所需的操作数、立即数、目的寄存器编号、指令地址及控制信息。若只传数据不传目的编号，结果就可能写到错误寄存器；若指令已经阻塞，写使能却继续向下游传递，就可能重复写入或错误访存。

时钟负责推进状态，控制信号决定状态能否写入、组合部件执行哪种功能。阶段寄存器常需写使能、保持和清空控制，因此 “控制仅作用于组合部件、时钟仅作用于寄存器” 的割裂描述不能当作普遍规则。气泡是在某阶段插入一个不改变有效状态的操作，冻结则让原状态保持，两者效果不同。

5.9 冒险先按冲突对象分类

结构冒险来自同一拍内争用数量不足的硬件资源。例如取指和数据读取同时需要单端口统一存储器；可通过停顿、增加端口或分离指令与数据存储来处理。寄存器组同拍读写是否冲突，取决于端口数及前后半拍的时序约定。

数据冒险来自指令间数据依赖与执行时序的矛盾。写后读 RAW：后指令读前指令产生的新值；读后写 WAR：后指令过早写破坏前指令尚未读取的旧值；写后写 WAW：两次写的完成顺序颠倒，使最终值错误。在固定五段、顺序发射、ID 读寄存器且 WB 顺序写回的简单流水线中，通常需要处理 RAW，而既定次序使 WAR、WAW 不会发生。变为乱序执行或不同延迟写回后，这个结论不再普遍成立。

控制冒险来自后继指令地址未确定，常见于条件分支、跳转、异常和中断处理。无条件转移仍需在某个时刻获得目标地址，因而也可能造成前端控制冒险。“不判断条件” 并不等于 “不会停顿”。

5.9.1 转发能省去等待写回，但不能送来尚未产生的数据

转发把已经形成的结果从阶段寄存器等位置直接送到后续指令使用点，不必等写回寄存器组。例如 ALU 结果在 EX 结束时产生，后继指令下一拍进入 EX，可以从 EX/MEM 寄存器取得该结果。

load 的数据通常到 MEM 结束才产生。紧随其后的指令若下一拍 EX 就需要该值，即使有转发也来不及，必须插入一个等待周期。转发路径是否通往存储数据端、分支比较端，还要看具体电路；不能一见“支持转发”就推断所有相关都已解决。

错项排除·段寄存器还要携带控制

只传操作数不传目的编号，结果可能写错寄存器；保持数据却让写使能继续推进，可能重复写入。流水段状态要同时说明值、目的、有效位与控制信息，不只数“几个数据寄存器”。

5.9.2 同一组 RAW，在不同实现中等到哪一拍

题设实现	消费者取得值的条件	排时序时使用的约束
无转发，寄存器同拍不能读写	必须等生产者 WB 结束，再在 ID 读	消费者 ID 至少比生产者 WB 晚 1 拍
无转发，寄存器同拍先写后读	生产者 WB 前半拍写，消费者 ID 后半拍读	可以让消费者 ID 与生产者 WB 同拍
ALU 结果向 EX 转发	生产者 EX 末已有结果，下一拍送消费者 EX	不必为该值等到寄存器 WB
load 结果向 EX 转发	生产者 MEM 末才有读出数据	消费者 EX 须在该 MEM 之后，紧邻 load 仍可能停 1 拍

“相关”由程序中谁产生、谁使用同一个值决定；“冒险是否导致停顿”还由结果产生点、使用点、转发路径和寄存器时序共同决定。因此，无转发题不能套用 load-use 只停一拍的结论，有转发题也不必机械等到所有源写回。STORE 的数据可能在 ID 读取、在 MEM 使用，分支可能在 ID 或 EX 比较，须先看清具体通路的约定。

5.9.3 例 5-8　装入后立即使用需要一个气泡

【配套例题】下列指令序列中，指令 I1 和 I3、I2 和 I3 之间发生数据相关。假定采用“取指、译码/取数、执行、访存、写回”五段流水线方式，那么在采用转发技术时，需要在指令 I3 之前加入（　）条空操作指令才能使这段程序不发生数据冒险。

```
I1: add  r1, r0, 1    // R[r1] <- R[r0] + 1
I2: load r3, 12(r2)   // R[r3] <- M[R[r2] + 12]
I3: add  r5, r3, r1   // R[r5] <- R[r3] + R[r1]
```

A. 3　B. 2　C. 0　D. 1

解析。I1 在第 3 拍 EX 末形成 r1 结果，到 I3 需要时已经可用；I2 到第 5 拍 MEM 末才有 r3 数据，而无停顿的 I3 本应在第 5 拍 EX 使用它，时间相撞。把 I3 的 EX 推迟到第 6 拍即可，因此选 **D**。

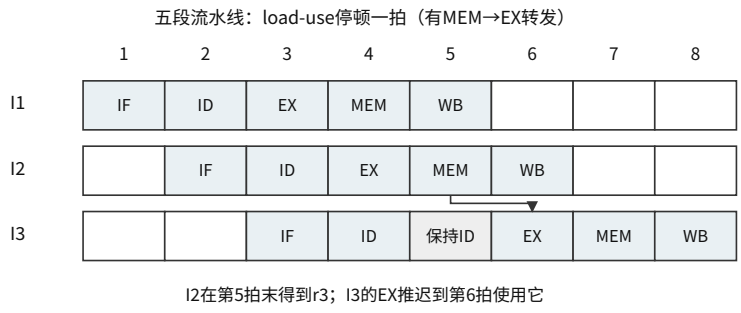


Figure 5.3: load 到 MEM 末才有数据；空泡把使用时刻推迟到下一拍

指令	成功推进的阶段与周期	等待状态
I1	IF1，ID2，EX3，MEM4，WB5	无
I2	IF2，ID3，EX4，MEM5，WB6	无
I3	IF3，ID4，EX6，MEM7，WB8	第 5 拍保持 ID，EX 为气泡

不变量·产生点必须早于使用点

先把每个值的产生段末与消费者所需段初标在时间线上，再检查是否有实际转发路径。STORE 数据端、分支比较端和 ALU 输入端可能不同，不能把一种 load-use 模板用于全部指令。

采用寄存器组同拍先写后读的时序，I1 第 5 拍 WB 写回的 r1 可在该拍被保持在 ID 的 I3 读到；若实现不同，须另有 WB 到 ID 的旁路。第 5 拍 EX 放入气泡，I3 在 ID 保持，下一拍取得装入结果。这里按题目所指的常见五段转发模型，把 r3 的使用点设在 EX；若某图要求在 ID 比较，所需等待和转发路径还可能不同。

5.9.4 例 5-9　同一段序列里同时有数据与控制等待

【配套例题】在采用 “取指、译码/取数、执行、访存、写回”5 段流水线的 RISC 处理器中，执行如下指令序列，其中 s0、s1、s2、s3 和 t2 表示寄存器编号。

```
I1: add  s2, s1, s0  // R[s2] <- R[s1] + R[s0]
I2: load s3, 0(s2)   // R[s3] <- M[R[s2] + 0]
I3: beq  t2, s3, L1  // if R[t2] = R[s3] jump to L1
I4: addi t2, t2, 20  // R[t2] <- R[t2] + 20
I5: L1: ...
```

若采用转发（旁路）技术处理数据冒险，采用硬件阻塞方式处理控制冒险，则在指令 I1～I4 的执行过程中，发生流水线阻塞的指令有（　）。

A. 仅 I3　　B. 仅 I2、I4　　C. 仅 I3、I4　　D. I2、I3、I4

解析。I1 的 ALU 结果可以及时转发给 I2 计算地址，所以 I2 不因这项 RAW 等待。I2 是 load，I3 马上使用 s3，仍有 load-use 等待。I3 的分支结果未确定时，硬件阻塞后继 I4，形成控制等待。因此发生阻塞的是 I3、I4，选 **C**。分析一条序列不能只找到第一组相关就停下；每个使用值的位置和后继 PC 都要检查。

5.9.5 三种处理办法各改变什么

硬件阻塞保持相关指令及其前方的必要状态，在前级暂停、后级放气泡，让生产者先前进。编译调度把不依赖该结果、且移动后不改变程序语义的指令填进等待间隔；无法填满才用 NOP。不能把有副作用或可能越界访存的指令随意移过分支。

分支预测先选择一条路径继续取指，正确时减少等待，错误时清除错误路径上尚不能提交的结果并从正确位置重新取指。若每条指令发生预测错误的概率为 p、每次平均损失 d 周期，且其他开销已包含在基本 CPI 中，可用 $\mathrm{CPI} = \mathrm{CPI}_0 + pd$；“分支比例” 并不等于 “所有指令的预测失败比例”。

旁路主要让结果更早可用，不直接决定分支目标地址；某些设计把数据转发给分支比较器，缩短因数据相关而产生的分支等待，但不等于所有控制冒险都由旁路消除。题目需按具体控制路径判断。

5.10 多发射、动态调度与按序提交

最小反例·改名字不能提前算出值

前指令产生 r1，后指令确实需要这个结果，即使把目的映射为物理寄存器 p7，后指令仍须等 p7 得到值。重命名消除 WAR/WAW 名字冲突，不能消除真实 RAW 因果依赖。

超标量 CPU 每周期可发射多条指令，需要相应的译码、调度、执行和写回资源。是否真能每周期发满，取决于数据依赖、分支、功能部件和访存带宽。最大发射宽度 w 只给出理想吞吐率上限，不能把 w 直接当作任意程序的加速比。

超流水把阶段划得更细，试图缩短时钟周期。阶段寄存器开销和冒险损失会限制收益。超长指令字 VLIW 把若干可并行的操作交给编译器打包，硬件和指令格式仍需支持相应并行执行，并非 “只改编译器，硬件没有要求”。

动态调度可让后面的就绪指令先执行，减少因前面一条长延迟指令而造成的闲置。寄存器重命名为每次逻辑写入分配不同的物理寄存器，可去除 WAR、WAW 这类假相关，但不能消除真实 RAW：消费者必须等对应生产者的值。按序提交使软件看到的状态仍符合程序顺序，并帮助实现精确异常。

例如前指令还要读取旧 r1，后指令将写新 r1 时，为新值分配另一物理寄存器，可避免覆盖旧值；若后指令确实需要前指令算出的 r1，改名并不能凭空产生结果。区分 “名字相同” 和 “值确有依赖” 比背三组字母更有用。

5.11 一条指令处理多个数据，还是多个指令流

Flynn 分类按指令流和数据流区分：SISD 为单指令流单数据流；SIMD 为单指令流多数据流；MISD 为多指令流单数据流；MIMD 为多指令流多数据流。分类描述执行组织，不是按机器装了几个屏幕、几块内存来判断。

向量处理器用一条向量指令对一组数据元素重复执行同类操作，可采用向量流水和多个通道，属于常见 SIMD 组织。它有启动代价，长向量能更好地摊薄这笔代价；向量长度不等于同时存在同样多套完整处理器。SIMD 也不能自然地处理这样的任务：每个元素都要求完全不同的控制流程；分支发散可能降低有效利用率。

MIMD 允许不同处理单元执行不同指令流。共享物理地址空间的多处理器属于 MIMD 的一种，分布式存储系统也可属于 MIMD。SMP 中的 “对称” 是各处理器在系统功能等方面地位对等，不表示每个时刻执行同一条指令。

5.12 硬件线程、多核与共享内存

硬件多线程让一个核保存多套线程体系结构状态，以便在一个线程等待时运行另一线程。细粒度多线程可按周期交替线程；粗粒度多线程常在较长延迟事件时切换；同时多线程 SMT 允许同周期从不同线程发射操作，利用已有执行资源。它们都不要求芯片必须有多个核。

多核是在一个处理器芯片内集成多个执行核心。各核可以独立推进指令流，通常还共享某些 Cache、互连或内存控制资源。增加核数并不保证单线程加速，也不意味着功耗必然高于任意单核设计；工艺、频率、负载和资源共享均影响结果。

共享内存多处理器让各处理器使用同一物理地址空间，但各核寄存器、部分 Cache 仍可私有。共享地址空间只保证各处理器能访问同一对象，不保证多个读改写操作自动具有原子性。并发程序还需要同步，

Cache 一致性与存储顺序也要分别满足相应规则。操作系统的进程同步讲义会沿用这一边界。

课程连接·共享地址并不自动互斥

多核共享物理地址空间，仍可能并发读到同一旧值并分别写回。OS 同步中的原子操作与锁解决复合更新的协调；Cache 一致性不等于所有读改写序列自动原子。

5.12.1 例 5-10　硬件多线程不等于多核

【配套例题】下列关于并行处理技术的叙述中，不正确的是（　）。

A. 多核处理器属于 MIMD 结构
B. 向量处理器属于 SIMD 结构
C. 硬件多线程技术只可用于多核处理器
D. SMP 中所有处理器共享单一物理地址空间

解析。单核也能保存多套线程状态并共享执行部件，所以 C 错，选 **C**。A、B 是按指令流和数据流分类，D 说的是共享物理地址空间；都没有断言所有资源完全相同或所有访问自动同步。

5.12.2 例 5-11　无转发流水线的阻塞传播与最短调度

【配套例题】某 16 位计算机中，带符号整数用补码表示，数据 Cache 和指令 Cache 分离。下面给出了指令系统中部分指令的汇编格式及对应功能，其中 Rs 和 Rd 表示寄存器，mem 表示存储单元地址，(x) 表示寄存器 x 或存储单元 x 的内容。

```
ADD   Rs, Rd
SHL   Rd
SHR   Rd
LOAD  Rd, mem
STORE Rs, mem
```

上面每行给出一种汇编格式；下表逐项给出各助记符对应的名称和功能，不改变源、目的操作数的次序。

名称	助记符	指令功能
加法	ADD	(Rs) + (Rd) → Rd
算术/逻辑左移	SHL	2(Rd) → Rd
算术右移	SHR	(Rd)/2 → Rd
取数	LOAD	(mem) → Rd
存数	STORE	(Rs) → mem

该计算机采用 5 段流水方式执行指令，各流水段分别是取指 IF、译码/读寄存器 ID、执行/计算有效地址 EX、访问存储器 M 和结果写回寄存器 WB。流水线采用 “按序发射、按序完成” 方式，没有采用转发技术处理数据相关，并且同一个寄存器的读和写操作不能在同一个时钟周期内进行。请回答下列问题。

（1）若 int 型变量 x 的值为 −513，存放在寄存器 R1 中，则执行指令 `SHR R1` 后，R1 的内容是多少（用十六进制表示）?

（2）若某个时间段中有连续 4 条指令进入流水线，在其执行过程中没有发生任何阻塞，则执行这 4 条指令需要多少个时钟周期?

（3）若赋值语句为 `x=a+b`，x、a、b 均为 int 型变量，其存储单元地址分别为 [x]、[a]、[b]。对应指令序列及流水过程如下，I3 的 ID 段和 I4 的 IF 段被阻塞的原因各是什么?

```
I1  LOAD  R1, [a]
I2  LOAD  R2, [b]
I3  ADD   R1, R2
I4  STORE R2, [x]
```

最小反例·一核也能保存两线程状态

一个核心可保存两套 PC 和寄存器状态，按周期或等待事件交替执行，也可 SMT 跨线程发射。线程状态数与物理核心数不同，硬件多线程不要求先有多个核心。

指令	成功推进的阶段与周期	因依赖或前级冻结而等待
I1	IF1，ID2，EX3，M4，WB5	无
I2	IF2，ID3，EX4，M5，WB6	无
I3	IF3，ID7，EX8，M9，WB10	第 4～6 拍在 ID 端等待
I4	IF7，ID11，EX12，M13，WB14	第 4～6 拍未获准取指；第 8～10 拍在 ID 端等待

上表将原图时空格紧凑写成“阶段名 + 周期数”，另列等待状态：I3 第 4～6 拍在 ID 端等待，I4 第 4～6 拍尚未获准取指，第 8～10 拍又在 ID 端等待；这些等待状态保留了原图中的阻塞信息；其他未列时刻表示该指令尚未进入或已经离开流水线。

（4）若赋值语句为 `x=x*2+a`，x 和 a 均为 unsigned int 型变量，其存储单元地址分别为 [x]、[a]，执行这条语句至少需要多少个时钟周期？要求模仿上图，画出对应指令序列及流水线执行过程示意图。

解析。本题与例 5-8 的转发模型不同。全题统一采用如下约束：源寄存器若在第 w 拍 WB 写回，则消费者最早第 w+1 拍在 ID 完成取数并向后推进；在此之前可以停留于 ID 端等待，IF/ID 寄存器保持，后继取指也被阻塞。按题中过程，所有指令都要走完五段才算完成，即使某一段不产生实际数据读写。

（1）−513 的 16 位补码为 `FDFFH`，即 `1111 1101 1111 1111`。算术右移复制符号位，得到 `1111 1110 1111 1111`，即 `FEFFH`，真值 −257。表中“除以 2”说明算术右移的数量关系；对负奇数按位右移相当于向负无穷取整，不能套用 C 整数除法向零截断，答成 −256。

（2）无阻塞时，第一条用 5 拍，后 3 条每隔 1 拍完成，共 $5+4-1=8$ 拍。指令 Cache 与数据 Cache 分离，题干未给取指与数据访存争用，不应额外添加。

（3）I3 要读 I1 写入的 R1 和 I2 写入的 R2，存在两条 RAW 依赖。I1 在第 5 拍、I2 在第 6 拍 WB 写回，且同一寄存器不许同拍读写，所以 I3 在第 4～6 拍于 ID 端等待，第 7 拍才可完成取数，而非第 6 拍。I3 占用的 IF/ID 状态尚未释放时，若 I4 继续取指，会覆盖这条待推进指令，因此 I4 的 IF 也推迟到第 7 拍。后一处是前方数据等待向取指端传播，不能误说成 STORE 在 IF 就要读 R2。

等 I3 在第 10 拍 WB 写出和，I4 才可在第 11 拍 ID 读取 R2，随后经过 EX、M、WB，在第 14 拍流出。原图的两段空白应分别追溯到它等待的生产者与受阻的资源，不能把空白一概叫作访存延迟。

（4）用两个寄存器保存 x 和 a，让 x 先装入，以便尽早开始连续依赖的移位与相加：

```
I1  LOAD  R1, [x]
I2  LOAD  R2, [a]
I3  SHL   R1          // 也可用 ADD R1, R1
I4  ADD   R1, R2      // R2 <- 2*x + a
I5  STORE R2, [x]
```

课程连接·状态与控制分工

寄存器保留当前状态，控制器依状态与指令信息发出控制信号；保存状态与产生信号是不同功能。



Figure 5.4: 无转发且寄存器不能同拍读写：分别等待 WB 后的下一拍，最短 17 拍

I1 第 5 拍 WB，故 I3 最早第 6 拍 ID、第 9 拍 WB。I4 依赖 I3 的 R1，最早第 10 拍 ID、第 13 拍 WB；I2 已在第 6 拍写回，不会再额外拖延 I4。I5 最早第 14 拍 ID，至第 17 拍流出，故至少 17 拍。

这个排程还给出了下界证明：本题给定指令中，需要两次 LOAD、一次翻倍、一次相加和一次 STORE；x 的“LOAD→ 翻倍 → 相加 →STORE”链不能颠倒。即使其余工作都尽早完成，这条链也要求 ID 分别不早于第 6、10、14 拍；所列排程恰好达到下界。若先装 a 再装 x，关键链反而晚一拍。STORE 在第 16 拍 M 已写入存储器，但按原题五段完整流出的统一口径，答案仍为 17 拍。

5.12.3 流水线巩固

1. 原创补充：理想 5 段流水线周期 2 ns，完成 20 条指令，中间有 3 个导致整体延长的停顿周期，求总时间与实际吞吐率。
2. 原创补充：若一条指令的结果要到 MEM 末才形成，而消费者在下一拍 EX 开始使用，有 MEM→EX 转发，是否仍需等到生产者 WB 结束？若消费者 EX 与生产者 MEM 是同一拍呢？

答案。1. $(5+20-1+3)\times 2=54$ ns，吞吐率 $20/54\approx 0.370$ 条/ns。2. 前者不需要，可在下一拍使用 MEM 末的结果；后者来不及，需要推迟消费者。关键是结果形成时刻与使用时刻，而不是 “二者有相关” 这句判断本身。

5.13 CPU 内的状态与控制信息

CPU 既执行数据运算，又负责指令控制、操作控制、时间控制和中断处理。通用寄存器可由指令显式指定；PC、IR、MAR、MDR、微地址寄存器等承担特定组织功能，其中哪些对程序员可见要按 ISA 区分。PSW 保存条件码及控制状态，既包含算术结果信息，也可能包含中断允许、特权状态等。

取指周期根据 PC 取得指令并更新顺序地址；间址周期按需要取得

有效地址；执行周期完成指令规定动作；中断周期在满足条件时保存返回所需状态并进入服务程序。不是每条指令都经过全部这些周期。机器周期用于描述某种基本操作组织，时钟周期是时序基准，三者不能按名称认定固定倍数。

课程连接·单总线冲突

同一拍通常只能由一个源驱动总线，多个目的同时接收还需各自路径和写使能。

5.14 单总线数据通路的节拍安排

内部单总线通常同一时刻只允许一个部件驱动总线，多个寄存器可以在满足时序时同时接收同一总线值。ALU 若需要两个输入，而总线一拍只能给出一个源，须由暂存寄存器保存另一操作数；ALU 结果再暂存在 Z 一类寄存器中，后续送总线。

例如计算 $R_1 \leftarrow R_1 + R_2$，可依次组织：$R_1 \rightarrow Y$；$R_2 \rightarrow$ BUS 并使 ALU 执行 $Y + \text{BUS} \rightarrow Z$；$Z \rightarrow R_1$。如果把两个不同源同时接到同一总线，或在结果尚未稳定时提前写入目的寄存器，都违反数据通路条件。专用通路、多总线结构可以增加并行传送能力，但还要检查每条路径和写使能的实际连接。

5.14.1 取指与间址微操作的依赖

取指中常见 PC $\rightarrow$ MAR、$M[\text{MAR}] \rightarrow$ MDR、MDR $\rightarrow$ IR 以及 PC 顺序更新。访存完成前，MDR 中的旧内容不能当作本条新指令。指令进入 IR 后，操作码才可用于译码；若地址字段本身用于间接寻址，还需把它送 MAR，再读出有效地址。

能否把 PC 加 1 与读存储器放在同一节拍，取决于 PC 增量器是否独立、总线是否被占用以及控制器的时序方案。课件节拍表是某一具体组织的实例，应先理解依赖关系，再照图列允许的并行动作。

5.15 硬布线控制器从何处获得控制信号

硬布线控制器把指令操作码、时序节拍、机器周期状态和有关条件标志输入组合逻辑，生成各寄存器写使能、总线输出使能、ALU 操作选择及存储器读写控制。设计时先分析指令微操作，再安排节拍，最后把“某指令、某周期、某节拍、某条件下发出某信号”合成为逻辑表达式。

例如某控制信号在指令 A 的 T_1 或指令 B 的 T_2 发出，可概括为 $C = A \cdot T_1 + B \cdot T_2$；若还取决于 ZF，应把条件乘入对应项。这里的加、乘是逻辑或、与。硬布线速度通常较快，但指令和控制条件增加时，设计、修改及验证复杂度也增加。

5.16 微程序控制：区分四种地址

主存地址定位机器指令或数据，PC 保存程序级指令地址；控制存储器 CM 保存微指令，CMAR 保存微指令地址；CMDR 保存当前微指令，操作控制字段输出微命令，顺序控制字段参与确定下一微地址。主存和控存是不同逻辑空间，机器指令地址与微指令地址也不是同一数值体系。

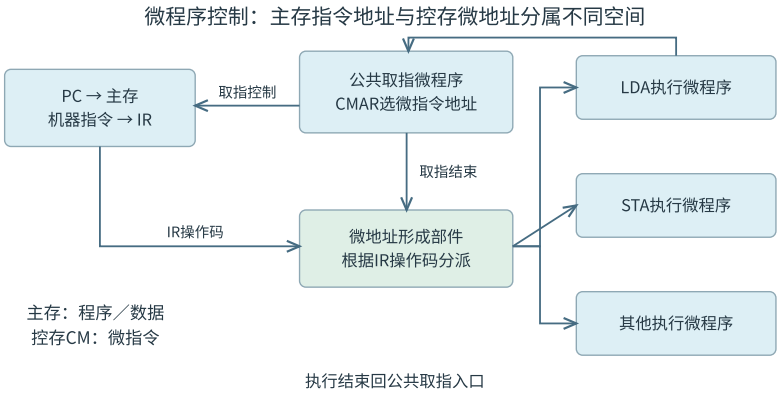


Figure 5.5: 公共取指微程序先运行，再由操作码映射到相应执行微程序入口

取指微程序可以由全部机器指令共享。取指结束后，根据 IR 操作码形成执行微程序入口；执行微程序结束后，通常回到公共取指入口。遇到条件转移、间址或中断时，还可能根据测试条件和周期状态选择相应微程序分支。

5.16.1 微指令编码与下地址形成

直接控制法为每个控制信号分配一位，译码简单、并行性强而字长较长。字段直接编码把互斥微命令分到同一字段，不同字段可并行；一个含 k 个可选微命令的字段若需额外表示“本字段不发命令”，至少 $\lceil \log_2(k+1) \rceil$ 位。相关编码还可利用字段之间的依赖进一步压缩，但增加译码层次。

水平型微指令一般允许同拍多个微操作并行，控制字段较宽；垂直型采用更强编码，字长较短而执行微操作的并行程度受到限制。这里的水平、垂直是控制信息组织方式，不是微程序在纸上的排版方向。

下一微地址可由当前微指令下地址字段直接给出，也可由微地址计数器顺序增量、指令操作码映射、条件测试或硬件强制入口形成。下地址位宽按控存实际需要区分的微地址数量估算，而不是按机器指令条数直接取对数。

设 32 条机器指令，各自执行段平均 4 条微指令，共享取指段 2 条，且这些段互不重叠，则至少需存储 $32 \times 4 + 2 = 130$ 条微指令，顺序地址字段至少 8 位。若微程序共享其他公共段、还含中断段或间址段，须据实际布局重新统计。

5.16.2 微程序控制单元的设计步骤

先列每条机器指令所需微操作并安排可并行的节拍，再补入“读下一微指令、形成微地址”所需的控制动作；选择控制字段编码和下地址形成方式，确定微指令字长，最后分配控存地址并编写各条微指令码。用当前微指令控制数据通路的同时，还应确保下一个微地址及时稳定。

静态微程序通常固化于 ROM，动态微程序允许通过可改写控存调整解释规则。毫微程序在微指令之下再设一层解释，以进一步压缩或组织控制信息。微程序修改依赖硬件支持及控制存储器类型，不能把修改普通程序文件等同于修改 CPU 微程序。

不变量·微指令字段各有职责

控制字段决定本拍发出的微命令，顺序字段参与形成下一微地址；操作码映射只用于需要选择微程序入口的阶段。

5.17 五段流水线中的装入、存储和转移

课程连接·值何时可用

装入结果在访存完成后可用；旁路路径和使用时刻共同决定需要几个气泡。

IF 取指，ID 译码并读取源寄存器，EX 进行算术运算或计算有效地址，MEM 访问数据存储器，WB 写回寄存器。不是每条指令都在每一段执行同样的实质工作，但简化固定五段模型可保留阶段位置以统一时序。

装入指令在 EX 算地址、MEM 取数据、WB 写寄存器；存储指令在 EX 算地址、MEM 写数据，通常没有寄存器写回；算术指令在 EX 产生结果，后续沿阶段寄存器传递；条件转移在哪段确定方向和目标，应按题设明确。阶段寄存器除数据外还须保存后续需要的目的寄存器号、控制位和异常信息。

在理想等长 k 段、每段时间 Δt、连续输入 n 条指令的模型中，总时间 $(k+n-1)\Delta t$，吞吐率 $n/[(k+n-1)\Delta t]$，相对完全串行的加速比 $nk/(k+n-1)$，效率 $n/(k+n-1)$。这些公式的前提是无冒险、无额外等待且阶段时间相同；不等长阶段通常按最慢段及寄存器开销确定时钟。

5.17.1 硬件多线程与共享内存组织

粗粒度多线程通常在较长停顿时切换线程，切换后还可能需要清空并重新填充流水线；细粒度多线程按较短固定粒度交替发射不同线程指令，用更多独立指令掩盖停顿；同时多线程 SMT 允许同一周期从多个线程选择指令占用不同发射槽。每个线程需要自己的体系结构状态，而功能部件等资源可以共享。

共享内存多处理器共享物理地址空间，可用 LOAD/STORE 访问共同主存；多计算机系统各有私有主存，通过消息传递通信。多核表示一块芯片集成多个处理核，是物理集成方式；SIMD 表示一个指令流控制多个数据通路，是并行控制方式，不能把二者当作同一分类轴。

5.17.2 三种硬件多线程的发射比较

时钟	细粒度：每拍选择一个线程	粗粒度：长停顿时换线程	SMT：一拍可选多个线程
i	A 的 j、j+1	A 的 j、j+1	A 的 j、j+1；B 的 k、k+1
i+1	B 的 k、k+1	A 的 j+2、j+3，发现 Cache 缺失	A 的 j+2；B 的 k+2；C 的 m
i+2	A 的 j+2、j+3	调度，从 A 切换到 B	A 的 j+3；C 的 m+1、m+2
i+3	B 的 k+2、k+3	B 的 k、k+1	按就绪指令及资源继续选择

该示例中的细粒度多线程允许同一拍从选中的线程发射多条指令；关键是这一拍仅选择一个线程。SMT 则可以在同一拍把不同线程的就绪指令安排到多个执行槽。粗粒度切换时可能需要重填流水线，因此切换的成本与粒度相关。

5.17.3 计算与辨析自查

1. PC、MAR、CMAR 保存的地址分别属于什么空间？2. 含 7 个互斥微命令及空操作的字段需要几位？3. 理想 5 段流水线执行 20 条指令需几拍？4. 为什么 load-use 冒险可能在有转发时仍需停顿？

答案。1. PC 为机器指令地址，MAR 为本次主存访问地址，CMAR 为控存微指令地址。2. 3 位，正好 8 种编码。3. $5+20-1=24$ 拍。4.

使用者需要操作数时，前一装入指令的数据可能尚未从 MEM 产生，转发只能传已有数据。

不变量·调度下界来自不可颠倒链

无转发且不许同拍读写时，LOAD→翻倍 → 相加 →STORE 链的各次 ID 必须晚于相应生产者 WB。先给关键链下界，再排出达到该下界的合法时序，才能证明 “最少几拍”。

5.18 主动回忆自测

以下均为自拟检查。沿章导图，先说明对象与前提，再写结论。

1. PC、IR、MAR、MDR 分别暂存什么？为何 MAR 与 MDR 位宽无须相同？
2. 一条指令是否总经过取指、间址、执行与中断四阶段？
3. 单内部总线同拍可否由两个不同源送两个不同目的？同一源送两个目的呢？
4. 机器指令、微指令与微命令各控制或表示哪个层次？PC 与 CMAR 有何区别？
5. 某互斥字段有 4 个微命令，为什么按通常约定至少需 3 位？
6. 故障处理后重执行当前指令，与外部中断后返回后继位置，依据是什么？
7. 理想 5 段、每段 2ns、20 条指令加 3 拍整体停顿，总时间和吞吐率分别是多少？
8. 在 EX 形成的 ALU 值与 MEM 末形成的 load 值，紧邻消费者所需等待为何可能不同？
9. 寄存器重命名能否消除真实 RAW？单核能否采用硬件多线程？
10. SIMD、MIMD、超标量与 SMT 分别按哪种并行维度分类？共享地址空间是否使读改写自动原子？

5.18.1 自测答案

1. PC 与后继取指位置有关，IR 存当前指令，MAR 存访存地址，MDR 存与主存交换的值；地址位数取决于可寻址单元数，MDR 位数取决于交换数据通路。2. 不总是；间接寻址、外部中断等有相应条件。3. 不同源通常冲突；同源多目的在独立写使能和实际路径允许时可以。4. 机器指令规定 ISA 操作，微指令组织一个控制步，微命令驱动具体部件；PC 访问机器程序，CMAR 访问控制存储器。5. 4 个命令加 “不发本组命令” 共 5 种状态，需 3 位。6. 故障尚未完成原定操作，可恢复时需要重试；外部中断通常发生在指令完成边界，但具体保存 PC 规则依 ISA。7. $T=(5+20-1+3)\times 2=54$ ns，吞吐率约 0.370 条/ns。8. 按结果形成时刻、消费者使用时刻及转发路径判断；数据尚未形成就无法转发。9. 不能消除真实值依赖；单核可保存多套线程状态并共享执行资源。10. SIMD/MIMD 按指令流与数据流，超标量按每周期多发射，SMT 按同周期跨线程利用执行部件；共享地址空间仍需原子操作与同步。

6 总线

章节脉络



6.1 从概念、性能到事务时序

本章沿课件导图的四组关系展开：先由 “共享通路” 辨认总线的组成、分类与结构，再把位宽、周期和传送率写成可计算的量，随后按申请、寻址、传输、结束四阶段追踪一次事务。定时方式回答 “何时允许推进”；仲裁回答 “由谁取得使用权”，两者分属不同问题。

定义 6.1.1 (总线的共享性)
总线是一组能被多个部件分时共享的信息传送通路。连接在同一条公共通路上，不表示多个主设备可以同时任意驱动线路；同一时刻允许的驱动者、接收者及信号解释由协议决定。

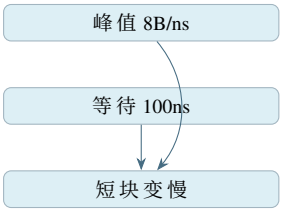
总线特性可以按四个维度检查：机械特性规定连接器、尺寸和形状；电气特性规定电平、负载等；功能特性规定每根线承载什么；时间特性规定信号建立、采样及撤销关系。它们分别回答 “如何接、什么电平、传什么、何时有效”。

按位置与用途可分片内总线、连接 CPU/主存/I/O 接口的系统总线，以及部件或系统间的通信通路；按数据形式分串行与并行；按定时分同步、异步等。一个具体接口可以同时具有 “串行”“包传输”“多通道” 的属性，分类维度不能混为一个互斥列表。

单总线结构让多个部件共享一条系统通路，连接简单但易形成争用；双总线、三总线以不同层次分担主存和 I/O 等流量。桥接器完成不同总线间的数据缓冲、协议转换和控制。增加通路能减少某类冲突，仍须核对桥接和主存是否成为瓶颈。

最小反例·峰值相同，事务速率可不同

两条总线数据拍都每 ns 传 8 B，一条每块额外等待 100 ns，另一条只等 1 ns；二者峰值相同，短块的有效速率却相差很大。分母是否包含等待必须先由题问确定。



相同数据拍峰值，等待改变事务有效速率

6.2 位宽、频率、传送率分别给出什么

总线是部件之间传送信息的公共通路。按信息用途可区分地址、数据和控制：地址说明访问哪个单元或端口，数据线承载被交换的值，控制线

说明操作类型、有效时刻及应答等。命令字虽然在语义上“控制设备”，CPU 写命令字时仍可能在**数据线**上传送；不能只凭名称判断该走哪组线。

数据线上可以传指令、操作数、状态字、命令字和中断类型号。它们如何解释取决于传送双方和操作类型。MAR 保存待访问地址，MDR 暂存交换的值；主存并不靠“某个数据看起来像地址”来判定这次访问的性质，而是由相应线路与控制时序确定。

总线位宽通常指并行数据线根数。串行传送逐位使用通道，并行传送同一时刻通过多条线传若干位；多个串行通道并用，并不等于一条传统并行总线。系统可用桥接部件连接不同速度和协议的通路，靠近 CPU 的部件通常要求更高带宽；有多条总线，并不表示它们共用同一时钟。

最小反例·串行不等于慢

一条高速串行链路与一条低频 8 位并行总线，谁带宽更大还需各自的传送率。位宽只决定每次能送多少，线路频率、编码和协议开销同样影响结果，不能用串/并行名称代替计算。

带宽先写每次字节数与每秒次数

若有效数据宽度为 W 位，每秒实际发生 R 次数据传送，理想单方向峰值带宽为 $R \times W/8$ B/s。R 与时钟频率不一定相同：每周期一次为 R=f，每周期两次为 R=2f；若已给 MT/s，则已经是每秒传送次数，不能再乘“每周期次数”。

总线周期是一次规定的总线操作所占的时间，可能有多个时钟周期。带宽为每次传送的有效数据量除以这次所用时间；控制、地址、等待、校验与空闲开销会使有效数据率低于只算数据节拍的峰值。

6.2.1 例 6-1　一个总线周期占两个时钟周期

【配套例题】假设某系统总线在一个总线周期中并行传输 4 B 信息，一个总线周期占用 2 个时钟周期，总线时钟频率为 10 MHz，则总线带宽是（　）。

A. 10 MB/s　　B. 20 MB/s　　C. 40 MB/s　　D. 80 MB/s

解析。时钟周期 100 ns，一次总线传送需 200 ns，传 4 B，因此带宽 $4/(200 \times 10^{-9}) = 20 \times 10^6$ B/s，选 **B**。若直接用 4 B×10 MHz，就忽略了每次传送占两拍。

6.2.2 例 6-2　已给传送率时不重复乘每周期次数

例 6.2.1 (自拟数值例)

某 64 位并行总线每时钟周期传 4 次数据，实际数据传送率已明确给为 1333 MT/s，求单方向理想峰值带宽。若改给的是 333.25 MHz 时钟频率，结果是否相同?

解析。每次 64 位 =8 B；1333 MT/s 就是每秒 1333×10^6 次，带宽为 $1333 \times 10^6 \times 8 = 10.664$ GB/s。若给 333.25 MHz 且每拍 4 次，则先算 $R = 4f = 1333$ MT/s，结果相同。若改成 1333 MHz 时钟、仍每拍 4 次，才会得到 42.656 GB/s。三个单位条件必须分别使用，不能把“频率”字样覆盖题干明示的物理量。

多通道带宽只有在各通道能够并行传送且工作分配允许时才相加。全双工总线要区分单方向带宽和两个方向合计带宽；校验位若不属于有效载荷，不应计入有效数据量的分子。带宽题第一步宜写出“每次有效几字节、每秒几次”，再统一单位。

6.3 一次总线事务从地址到应答

不变量·先数有效数据拍

32 B 块经 64 位数据线传送，需要 4 个有效数据单位。突发减少的是地址、准备等重复代价，不能把 32 B 整块误作一个 8 B 数据拍；每拍几次传送还要另核时钟口径。

总线主设备发起事务，从设备按地址被选中并完成操作。CPU 常是主设备，DMA 控制器也可在取得使用权后成为主设备。主从关系只就本次事务而言，不等于某个设备永久只能发送或只能接收。

一次读事务可以分为：主设备发地址和读命令；从设备译码并准备数据；从设备在数据线上给出有效数据；主设备采样，双方结束本轮操作。写事务则由主设备给出要写的数据。地址、数据若复用同一组线路，就需不同时间片分别传送，不能把这些线路同时算成两份独立的并行位宽。

6.3.1 同步定时：固定采样时刻，必要时插等待

同步总线用共同的时钟约定信号的有效和采样时刻。最短周期必须覆盖信号传播、译码、设备准备及建立时间等延迟。若从设备来不及，可按协议插入等待状态；“同步”不等于所有事务长度永远相同，也不表示每个设备可随意提供互不协调的时钟。

突发传送只给出一个首地址，再连续传送多个相邻数据，分摊地址和启动开销。非突发则可能为每个数据重复地址及准备过程。因此“读一块多长时间”应先数数据拍数，再判断地址、等待代价要付几遍。

6.3.2 例 6-3　不支持突发，就要反复准备每个数据

【配套例题】某存储器总线宽度为 64 位，总线时钟频率为 1 GHz，在总线上传输一个数据或地址需要一个时钟周期，不支持突发传送方式。若通过该总线连接 CPU 和主存，主存每次准备一个 64 位数据需要 6 ns，主存块大小为 32 B，则读取一个主存块所需的时间是（　）。

A. 8 ns　　B. 11 ns　　C. 26 ns　　D. 32 ns

解析。一拍 1 ns，每个 64 位数据为 8 B，一块需要 4 个数据。每个数据都单独付地址 1 ns、准备 6 ns、传送 1 ns，共 8 ns，总计 $4 \times 8 = 32$ ns，选 **D**。11 ns 对应只付一次地址和准备、再连传 4 次，是突发模型，题目已排除。

6.3.3 例 6-4　峰值带宽与整次事务的有效速率

【配套例题】某存储器总线的时钟频率为 420 MHz，总线宽度为 64 位，每个时钟周期传送 2 次数据；其总线事务支持突发传送方式，最多传送 8 次数据，第 1 个时钟周期传送地址和读/写命令，从第 4 个至第 7 个时钟周期连续传送 8 次数据。该总线的总线带宽（最大数据传输率）为（　）。

A. 3.84 GB/s　　B. 6.72 GB/s　　C. 30.72 GB/s　　D. 53.76 GB/s

解析。问的是数据节拍的最大传送率，每周期 2 次、每次 8 B，峰值 $420 \times 10^6 \times 2 \times 8 = 6.72$ GB/s，选 **B**。

若另问这次 8 次数据的事务有效速率，才把地址和等待计入分母：数据 64 B、事务 7 周期，$64/(7/(420 \times 10^6)) = 3.84$ GB/s。A 正是按整次事务口径算出的结果，不能用它回答本题已经说明的峰值带宽。第 4 至第 7 拍共 4 拍，每拍 2 次，正好 8 次，不是 8 拍。

6.3.4 峰值、事务速率与 CPU 占用采用不同分母

不变量·撤销也要确认

全互锁中，从设备看到请求撤销后才撤应答。少看最后一步会把半互锁误作全互锁；画四个边沿并逐项标出依赖，比按波形长度猜类别可靠。

总线峰值带宽按数据节拍可持续搬多少有效字节来计，事务有效速率还把本次地址、等待等时间算入；设备速率是设备产生或消耗数据的速度；CPU 占用率则是 CPU 执行查询、服务或设置收尾的时间比例。这四个量可能出现在同一道题中，应先分别写成“什么量除以什么时间”。

例如在例 6-4 中数据节拍峰值 6.72 GB/s、整次事务有效速率 3.84 GB/s，都不能直接当成某个设备的实测速率；若设备只提供 40 MB/s，系统还受这个供数速度限制。反过来，DMA 可让设备数据直接经过总线进入主存，同时 CPU 仅做少量设置收尾，所以不能把总线使用率等同于 CPU 使用率。端到端吞吐要核最慢环节，资源占用要核实际争用的时段。

当一次突发在等待期间必须保持总线占有权时，等待也消耗总线资源；若题设明确采用分离事务、允许准备期间让出总线，则总历时与实际占用时间还会不同。画事务时间线时，应同时标出“在等什么”和“谁仍占有总线”，不能仅凭延迟长短直接推断总线忙碌比例。

6.3.5 异步定时：用握手确认进度

异步总线不用统一周期来强制双方同拍完成，而用请求和应答等信号交换进度。速度较慢的设备晚些应答，主设备相应等待。异步仍受顺序约束，也不能随时改变信号而不通知对方。

全互锁的四阶段握手可以写为：主设备置请求；从设备准备好后置应答；主设备看到应答后撤请求；从设备看到请求撤销后撤应答。各阶段都等待对方作出相应反应，因而不依赖预设的固定延迟来猜测对方已结束。

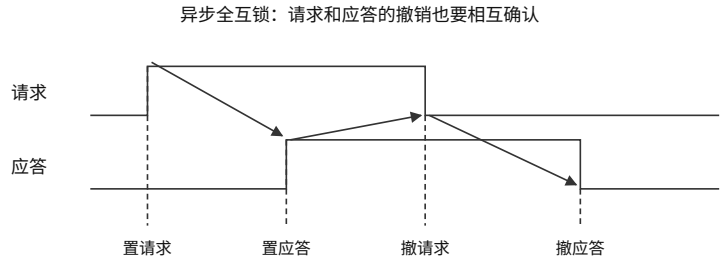


Figure 6.1: 全互锁握手的撤销顺序也由对方信号确认

非互锁方式中，主设备发出请求后按预定时间撤回，不等应答；从设备发出应答后也按时撤回，不等请求撤回。半互锁方式中，主设备等待应答出现才撤请求，而从设备仍可按时撤应答。全互锁方式中，从设备还必须等请求撤回才撤应答。给出时序图时，按图中信号的建立、撤销条件来识别，不单凭信号名称猜测。全互锁通常多了等待环节，不能因为最可靠就断定它最快。

半同步通信将时钟节拍与等待/应答结合：正常情况下按时钟节拍推进，慢设备可请求延长，握手状态在规定边沿采样。这里的“半同步”不等于一半数据串行、一半数据并行，两个分类维度不同。

6.3.6 如何读总线时序图

先标出地址何时稳定、读写命令何时有效、数据何时有效和何时被接收，最后数出从事务开始到完成的时间。若图中低电平有效，应以标出的有效电平判断，不可把所有上升沿都当成开始。不同信号能否同拍出

现由协议规定；“地址/数据复用” 和 “独立地址数据线” 会改变哪些操作可以重叠。

串行异步字符传送每帧另加起始位和停止位，校验位可选。若每字符 7 个数据位、1 个校验位、1 个起始位和 1 个停止位，则每字符实际发送 10 位，480 字符/s 需要 4800 bit/s，而有效字符数据只有 3360 bit/s。不要把物理线路传送率与有效数据率混为一谈。

错项排除·主设备角色随事务

DMA 可以发起一次主存写，CPU 也可能暂不占总线。主从描述谁发起并控制本次交换，并不表示某设备永远只能发送或永远只能接收。

6.3.7 事务速率巩固

1. 【配套例题】某存储器总线采用同步通信，时钟频率 50 MHz，突发传 8 个字，每字 4 B。1 周期接收地址、3 周期等待、8 周期传 8 个字。数据传输速率为（ ）。A. 114.3 MB/s　B. 126 MB/s　C. 133.3 MB/s　D. 144.3 MB/s。
2. 【配套例题】：将第 1 题改为每个字都独立付 1 周期地址、3 周期等待、1 周期数据，读同样 8 字需要多少时间，有效速率多少?

答案。1. C。共 12 周期，每周期 20 ns，32 B/240 ns ≈ 133.3 MB/s。2. 共 40 周期 =800 ns，32 B/800 ns=40 MB/s。两题块长相同，改变的是启动代价的重复次数。

6.4 总线仲裁与分离事务

总线传输在申请分配阶段处理多个主设备的使用权竞争，随后寻址、传输并结束。CPU 和 DMA 都可能成为主设备，主设备与从设备的角色针对一次事务；主设备申请总线，不表示它一定是本次数据的接收者。

课件将仲裁分为集中与分布两类。集中仲裁由一个仲裁部件授予使用权：链式查询以允许信号沿设备链传递，靠近仲裁端的请求通常先被满足，控制线少但固定优先级可能使低位请求长期等待；计数器定时查询依次给出设备号，请求者在号码匹配时取得资格，起始编号和轮转规则决定公平性；独立请求为各设备设置请求/允许通路，仲裁灵活但线路及逻辑开销较大。分布仲裁由参与者依共同规则竞争或判定，不能因此推断系统没有任何仲裁规则。

课件把总线仲裁与标准列为拓展。本章保留其机制联系，不凭文件名替读者确认某一考试年度的正式范围。总线标准还约定接口、电气、功能、时序等兼容条件；串行或并行只说明传输组织，不能单独决定峰值带宽或延迟。

分离事务允许主设备发出请求后释放总线，待从设备准备好再申请并返回结果；因此 “总历时” 与 “总线实际占用时间” 可以不同。它与普通同步/异步定时的分类角度不同，还需要标识请求与响应的对应关系。

定时/组织	推进依据	等待是否占通路	不能直接推出
同步	规定时钟沿；慢设备可插等待	由事务协议决定	每事务固定一拍
异步全互锁	请求/应答建立和撤销均互相确认	由握手占用规则决定	不需要时序约束
半同步	时钟节拍与等待/应答结合	需看等待状态的保持规则	一半串行一半并行
分离事务	请求与响应分开争用通路	准备期间可释放	总历时等于占用时长

6.5 总线按位置、信息和连接方式分类

课程连接·分类依据

数据、地址与控制描述信息类型；片内、系统与通信总线描述所处层次，分类角度各自独立。

总线是一组由多个部件共享的信息传输线路及配套协议。片内总线连接芯片内部部件，系统总线连接 CPU、主存和 I/O 接口，通信总线连接计算机、外设或系统之间的通信端点。数据总线传数据，地址总线选择存储单元或端口，控制总线规定读写、请求、应答、时序和状态。

地址总线宽度主要影响可给出的地址组合数，数据总线宽度影响一次可并行传送多少位。CPU 发起普通访存时地址由 CPU 输出；DMA 控制器成为主设备时也可发地址，所以“地址线总由 CPU 单向输出”只适用于限定的主设备关系。总线主设备启动事务，从设备响应事务，I/O 设备不必永远是从设备。

总线标准需要同时规定机械特性、电气特性、功能特性和时间特性：接插件尺寸与引脚布局属于机械规范；电压、负载等属于电气规范；各线表示什么信息属于功能规范；信号何时建立、何时采样属于时间规范。只做到插头相同并不能保证协议兼容。

6.5.1 单总线、多总线与桥接器

单总线组织连接简单、扩展方便，但不同部件争用同一通路，且速度受多方协调限制。多总线组织把 CPU—主存高速流量与较慢 I/O 流量分开，桥接器完成不同总线间的缓冲、转换和控制。增加总线可以减少争用，却也增加连接和控制成本；跨桥事务可能引入额外延迟。

串行传输每条数据通道按先后发送位，并行传输用多条线同拍传多位。串行链路可以使用多条 lane 并行传包；因此“串行协议”与“只有一条数据通道”不是同一判断。并行线间偏斜、串扰和接插件复杂度，会限制频率和距离；较少线数的高速差分串行链路可通过更高信号速率取得较高带宽。

6.6 四种定时方式的完成条件

同步定时用共同的时钟规定信号产生和采样时刻，简单而要求各模块满足统一时序；必要时用等待状态延长事务。异步定时不依靠统一固定等待时间，而用请求和应答信号确认进度，适合速度差异较大的部件；握手也有控制开销。

异步握手可按发送方撤销请求是否等待应答，以及接收方撤销应答是否等待请求撤销，区分不互锁、半互锁和全互锁。全互锁四阶段为：主设备置请求、从设备置应答、主设备撤请求、从设备撤应答。后一动作依赖前一动作，形成完整的建立和释放确认。

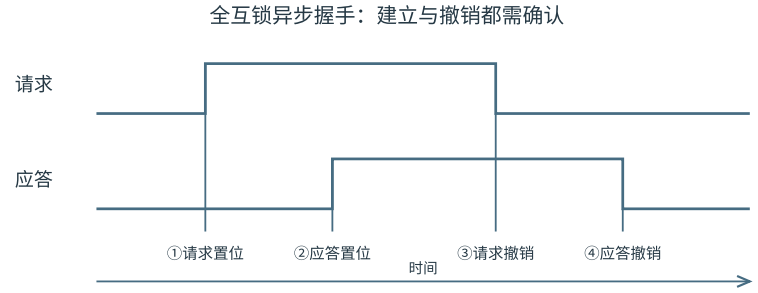


Figure 6.2: 全互锁握手中，请求和应答依次建立并依次撤销

半同步通信保留时钟基准，再由从设备用等待信号延长数据准备

阶段，兼顾采样规则和速度适应。分离事务把请求和返回数据分成两次总线占用；从设备内部准备数据时释放总线供其他事务使用，之后再申请总线返回结果。分离事务需要标识与排队机制对应请求和响应。

课程连接·集中式仲裁

链式查询、计数器定时查询和独立请求方式的连线与优先级规则不同，应分别判断。

6.7 拓展：三种集中式仲裁

多个主设备同时请求总线时，需要仲裁决定使用权。优先级决定谁先获得，公平性关系到低优先级设备会不会长期等待，二者不是同一个指标。

方式	选择机制	特点与限制
链式查询	允许信号沿设备链传播	结构简单，近仲裁器者通常优先；链路故障敏感
计数器定时查询	用计数编码轮询设备号	起始计数方式可改变优先顺序；需编码线和比较逻辑
独立请求	每个设备独立请求和允许线	响应快、优先级灵活；线路较多

按课件约定，n 个设备的链式查询可用请求、忙、允许共 3 条控制线；计数器方式需 $\lceil \log_2 n \rceil + 2$ 条；独立请求若每设备各有请求和允许并共用忙线，则为 $2n + 1$ 条。不同教材图的公共信号统计口径可能不同，计算必须先列线的种类。

6.8 拓展：常见总线和接口标准

标准	主要对象或组织	识别要点
ISA ／ EISA	早期 PC 扩展	传统并行总线，位宽及传送组织随版本变化
VESA 局部总线	早期高速外设	连接高速显示等局部流量
PCI	共享并行局部总线	支持总线主控、突发传输及配置机制
AGP	图形接口	为图形加速设计的历史接口
PCI Express	点到点串行链路	可多 lane，全双工；带宽取决于代际与 lane 数
USB	主机与外设	串行、支持集线器、即插即用和热插拔
SATA	存储设备	串行存储接口，标称速率按具体版本解释

标准	主要对象或组织	识别要点
SCSI	存储及其他设备接口体系	支持设备寻址、命令与控制机制
RS-232	串行通信	规定连接、电平及有关信号
PCMCIA ／ CardBus	便携设备扩展	历史上的卡式扩展标准

表中的名称用于识别结构和适用对象。某一课件列出的带宽是具体版本或理想条件下的值，不能当作该标准所有设备的统一实测速率。

按编码效率、包头、方向和通道数修正标称物理速率，才能得到有效载荷带宽。

例如 32 位数据通路、每秒 33×10^6 次有效数据传送时，理论数据率为 $33 \times 10^6 \times 4 = 132 \times 10^6$ B/s；若事务还需地址拍、仲裁和等待，有效事务带宽低于该峰值。DDR 表示每时钟周期可在两个边沿传数据，已给传送率时须避免再次乘 2。

课程连接·单位复核

先确定一次传输的字节数和每秒传输次数，带宽再据此计算；频率不必然等于事务完成率。

6.8.1 计算与辨析自查

1. 为什么数据线数不能直接决定最大寻址容量？2. 全互锁握手的四个动作按什么顺序发生？3. 分离事务释放的是什么资源？4. 8 设备独立请求按上述计线口径需要多少条控制线?

答案。 1. 寻址组合由地址编码及编址单位共同决定，数据线主要影响每次传送位数。2. 请求置位、应答置位、请求撤销、应答撤销。3. 设备准备数据期间的共享总线使用权。4. $2 \times 8 + 1 = 17$ 条。

6.9 主动回忆自测

沿导图回答，再核对单位与条件

以下为自拟检查，先遮住答案，按导图主枝回答。

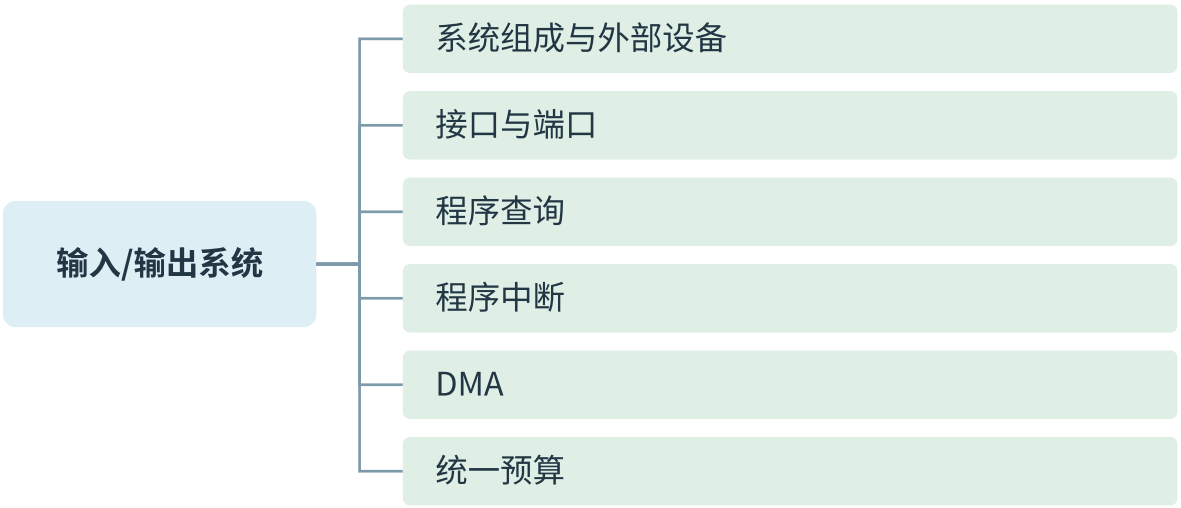
1. 地址、数据、控制三类线的分工，是否按所传信息名称里有没有 “控制” 二字决定?
2. 为什么 32 位、每拍两次的 100 MHz 总线，理想峰值是 800 MB/s，而不是 400 MB/s?
3. 给 200 MT/s 后，能否因 “每拍两次” 再乘 2?
4. 连读 8 个字时，突发方式节省的是哪类重复开销?
5. 全互锁握手四个事件的先后顺序是什么?
6. 一次读请求等待 100 ns，能否直接断言总线被独占 100 ns?
7. 仲裁与定时分别解决哪两个问题?
8. 双向合计带宽与单方向带宽相差多少，是否总能用同一个倍数转换?

6.9.1 自测答案

1. 不能；命令字、状态字等编码值可走数据线，控制线规定事务类型和时序。2. 每次 4 B，每秒 $2 \times 100 \times 10^6$ 次，乘得 800 MB/s，前提是全部数据位均有效。3. 不能，MT/s 已经计入实际传送次数。4. 首地址、启动与准备等是否只付一次，由题设确定；数据拍仍要逐项数。5. 请求置位、应答置位、请求撤销、应答撤销。6. 不能，分离事务可在设备准备时释放总线。7. 谁获得使用权；何时采样与推进。8. 需看全/半双工、两个方向是否能同时达到峰值以及题目统计口径，不可无条件翻倍。

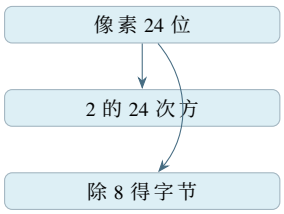
输入/输出系统 7

章节脉络



最小反例·24 位不等于 24 种颜色

每像素 24 位时最多有 2^{24} 种编码，而不是 24 种。容量计算乘 24 得到位数，再除 8 变字节；若题给的是灰度级 G，应先算编码位数 $\lceil \log_2 G \rceil$。



位数决定编码数，容量还须把 bit 换成 B

7.1 沿“谁等待、谁搬运、谁收尾”理解 I/O

课件把输入/输出系统分为硬件、软件与控制方式：设备和接口负责把物理事件变成可交换的数据；I/O 指令或驱动程序发出命令；查询、中断与 DMA 决定 CPU 何时介入、谁实际搬运。先固定设备速率、接口缓冲大小和每次服务粒度，再比较方式，才有统一的计数基础。

定义 7.1.1 (I/O 硬件、指令与通道)

I/O 硬件包括输入设备、输出设备、外存和 I/O 接口/控制器。I/O 指令是 CPU 执行、用于访问端口或控制 I/O 的机器指令；通道指令由专用通道执行，通道程序通常在主存中，CPU 启动后由通道组织相应设备工作。二者不是同一指令集的两个名称。

通道在硬件支持下承担比逐字 DMA 更完整的 I/O 组织任务，减少 CPU 的细粒度参与；它仍需软件准备、设备协议与完成通知。这里作为课件导图的背景分支保留，不把某一种历史实现推广为现代所有设备的结构。

7.2 外部设备及其缓冲需求

输入设备把用户或环境的信息送入系统，例如键盘、鼠标；输出设备把结果表现出来，例如显示器、打印机；外存提供较大容量的持久数据保存，磁盘与固态盘的具体访问机制回指存储系统一章。设备分类说明主要用途，不表示具有复合功能的设备只能归入一个绝对类别。

显示器参数包括屏幕尺寸、分辨率、每像素灰度或颜色位数、刷新频率。若每帧包含 N 个像素、每像素 b 位且无压缩，则一帧显存至少需

Nb 位，按每秒 F 帧连续刷新所需的有效像素数据率为 NbF 位/s；空白区、编码、总线等额外开销需按题设另计。字符点阵库保存每种字符的字模，帧缓冲保存当前屏幕的像素，两者统计对象不同。

例 7.2.1 (自拟：从分辨率算显存与数据率)

图像为 1920×1080 像素，每像素 24 位，每秒刷新 60 帧，不计压缩、空白区与传输编码。求单帧有效数据量、双缓冲容量及连续像素数据率。

解析。单帧为 $1920\times1080\times24/8=6220800$ B；双缓冲需要 12441600 B；数据率为 $6220800\times60=373248000$ B/s。24 位是每像素的编码位数，不是 “24 种颜色”；双缓冲使存储容量翻倍，并不要求把每秒实际输出帧数也乘 2。

打印机可按是否击打、按字符/行组织、按针式/喷墨/激光等机制分类。分类维度不同，不宜将 “非击打” 和 “行式” 作为互相排斥的并列项。接口中的缓冲可吸收设备与总线的瞬时速率差；持续输入超过长期消耗能力时，有限缓冲终会填满，不能靠 “有缓冲” 保证永不丢数据。

最小反例·同地址不等于同寄存器

某端口地址读出状态、写入命令，是由读写方向和接口译码区分的两项动作。只看数字地址相同就说状态寄存器被命令字覆盖，忽略了访问类型这一条件。

7.3 端口是接口中可访问的位置

CPU 通常通过 I/O 接口访问外部设备。接口把系统总线上的读写操作转换为设备所需的信号，并提供数据缓冲、状态记录、命令控制、地址识别及必要的中断或 DMA 连接。设备本体和接口并不是一回事：磁盘驱动器是设备，网络控制器、打印机适配器等承担接口功能。

CPU 能访问的接口寄存器或相应位置称为 I/O 端口。数据端口交换数据，状态端口报告就绪、忙或错误，控制端口接收启动、复位等命令。读状态与写控制可能共用同一个端口地址，由读写方向区分；不能因为地址相同就认为两个端口是同一个寄存器。

统一编址（存储器映射 I/O）把端口放进主存地址空间，用访存指令访问；独立编址为 I/O 设单独地址空间，通常使用专门 I/O 指令。同一个数值地址在独立编址下可分别对应内存位置和端口，在统一编址下则要避免地址译码冲突。端口地址不是设备内部磁盘扇区地址。

7.3.1 例 7-1　命令字走哪组线

【配套例题】下列选项中，在 I/O 总线的数据线上传输的信息包括（　）。

I. I/O 接口中的命令字　II. I/O 接口中的状态字　III. 中断类型号

A. 仅 I、II　B. 仅 I、III　C. 仅 II、III　D. I、II、III

解析。三者都是本次总线交换的编码值，可以由数据线承载，选 **D**。读写控制线表达 “本轮读或写” 等时序和操作性质，并不逐位承载所有名称里带 “控制” 二字的信号。

接口与设备侧交换数据、控制和设备状态，接口向 CPU 发送中断请求。打印机一般不会向其接口 “接收中断请求信号” 来获知下一个字符；这里要辨清信号的发送端与接收端。串并转换、缓冲和速度匹配也在接口或设备控制电路中完成，主机内部总线并行并不要求磁盘写入盘面的位也并行。

7.4 查询方式：等待和搬运都由 CPU 安排

不变量·最长间隔受缓冲期限限制

R B/s 填满 q B 缓冲需要 q/R 秒；定时查询必须在覆盖前读走数据，还要计入实际检查与搬运延迟。平均每秒查够次数并不自动保证某个最长间隔不会超过期限。

程序查询方式先启动设备，再读状态端口，未就绪就继续检查；就绪后由 CPU 执行指令，在数据端口与寄存器或主存之间搬运，并更新地址与计数。数据没准备好时，CPU 若一直轮询，等待时间也占用了执行机会。

```
设置设备命令、目标地址和剩余数量
while 剩余数量 > 0:
    while 状态端口表示未就绪:
        继续检查
    从数据端口读一个单位并存入目标地址
    目标地址 += 单位大小
    剩余数量 -= 1
```

定时查询则在规定时刻检查，其余时间可以执行其他工作。是否会丢数据取决于设备速率、接口缓冲容量和最长查询间隔，不能由忙等程序的表现推出“查询方式下 CPU 永远不能运行其他程序”。

7.4.1 例 7-2　定时查询占多少 CPU 时间

【配套例题】某计算机处理器主频为 50 MHz，采用定时查询方式控制设备 A 的 I/O，查询程序运行一次所用的时钟周期数至少为 500。在设备 A 工作期间，为保证数据不丢失，每秒需对其查询至少 200 次，则 CPU 用于设备 A 的 I/O 的时间占整个 CPU 时间的百分比至少是（　）。

A. 0.02%　　B. 0.05%　　C. 0.20%　　D. 0.50%

解析。每秒至少执行 $500 \times 200 = 100000$ 个查询周期，CPU 每秒有 50×10^6 个周期，比例为 0.002，即 0.20%，选 **C**。200 次/s 只决定查询次数，不是设备每秒搬运 200 B；题目也未要求把每次查询的 500 周期再当作 500 条指令。

若另给每个数据的搬运周期数，还要判断它已包含在查询程序开销中，还是必须另加。计算 CPU 占用率时，用 CPU 实际花掉的时间作分子，设备在外部独自准备数据的时间不直接算作 CPU 执行时间。

7.5 中断方式：先请求，再响应，再运行服务程序

中断 I/O 允许 CPU 启动设备后继续做其他工作，设备或接口就绪后提出中断请求。CPU 在允许响应的边界检查未屏蔽请求，完成硬件响应，再执行中断服务程序，由程序读取或写入数据端口。等待不再要求 CPU 持续轮询，但每次软件搬运数据仍要 CPU 执行指令。

请求需要保存在接口或中断控制器的触发器中，因为外设产生事件时 CPU 未必能立即响应。中断请求的产生、被允许响应、被调度执行以及处理完成是四个不同的时间点。CPU 可能正在执行指令，处于关中断状态，或已有更高优先级请求需要先处理。

7.5.1 硬件保存断点，与软件保护现场分开

典型硬件响应包括暂时禁止相应中断、保存返回位置及规定的状态、确定中断服务入口。中断向量是用于定位处理程序的信息，中断向量地址是保存向量的位置，处理程序入口地址是最终执行代码的位置。三者在某种实现中存在固定联系，但并不总是同一数值。

服务程序进入后保存其将修改且须恢复的通用寄存器，处理事件，再恢复并返回。单重中断处理期间不允许同类可屏蔽中断嵌套；多重中断则在保护必要现场、设置屏蔽后重新允许某些请求打断当前服务程序。入口处的现场保护未完成就开放嵌套，可能覆盖尚未保存的返回信息。

不变量·每层恢复自己的屏蔽字

嵌套服务像调用栈逐层返回。C 退出应先恢复 B 的屏蔽状态，B 再恢复 A；直接恢复最外层全开放状态可能允许原本不该打断 B 的请求，破坏题设处理关系。

在教材采用的一种单重中断序列中，软件部分是“保护现场 → 事件处理 → 恢复现场 → 开中断 → 中断返回”，保存断点和初始关中断已经由硬件响应完成。实际 ISA 可能由中断返回指令原子恢复状态和允许位；不能把教材序列中的“开中断”和返回之间可被中断的窗口当作所有机器的共同实现。

7.5.2 屏蔽改变允许谁打断谁

中断响应优先级通常由硬件仲裁关系决定，多个未屏蔽请求同时到来时据此选一个。处理优先级描述服务过程中允许哪些请求打断当前服务，可由各服务程序设置的屏蔽字确定。屏蔽会改变参与本次仲裁的集合，不会把仲裁电路本身的固定比较顺序改写。

7.5.3 例 7-3　响应顺序与完成顺序为什么可以不同

原创补充

有 A、B、C 三个请求源，硬件响应优先级 A>B>C。屏蔽字按 A、B、C 排列，1 表示屏蔽。要求处理优先级 C>B>A，服务程序均在保存现场并设置屏蔽字后开中断。给出三者的屏蔽字，并追踪三请求同时待处理时的一次嵌套过程。

解析。 A 处理中应屏蔽 A 自己，允许 B、C，故为 100；B 处理中屏蔽 A、B，允许 C，故为 110；C 处理中三者都屏蔽，故为 111。初始三者均允许响应时，硬件先选 A，不能因希望 C 处理优先就把首次响应写成 C。

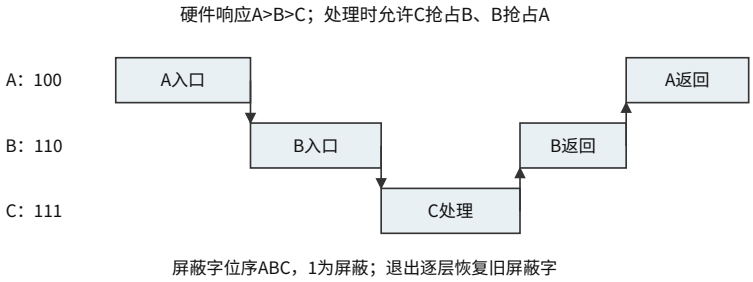


Figure 7.1: 屏蔽只改变可参与响应的集合；嵌套返回按栈次序进行

1. A 保护现场、设 100、开中断，B 和 C 仍待处理，硬件在二者中选 B。
2. B 保护现场、设 110、开中断，C 可以响应，转入 C。
3. C 设 111 完成处理，恢复 B 的状态和屏蔽字，返回 B。
4. B 完成后恢复 A，A 最后完成。服务开始顺序 A、B、C，完成顺序 C、B、A。

题目要的是给定规则下的模型。各层必须保存并恢复旧屏蔽字，不能在 C 退出时直接恢复成最外层的全开放状态。屏蔽字的位序和 0/1 含义也必须从题干确定。

7.5.4 例 7-4　允许延迟不是额外的服务开销

错项排除·块完成一次，单位传送多次

1024 B 块、4 B 接口缓冲可分 256 个数据单位搬运，而正常完成通知可仅一次。DMA 减少 CPU 事件数，不要求把整块压成一个总线数据拍；突发是否合并事务还需题设。

【配套例题】若某设备中断请求的响应和处理时间为 100 ns，每 400 ns 发出一次中断请求，中断响应所允许的最长延迟时间为 50 ns，则在该设备持续工作过程中，CPU 用于该设备的 I/O 时间占整个 CPU 时间的百分比至少是（　）。

A. 12.5%　　B. 25%　　C. 37.5%　　D. 50%

解析。每个请求实际占 CPU100 ns，请求间隔 400 ns，长期占用比例为 $100/400 = 25\%$，选 **B**。最长响应延迟 50 ns 约束请求产生后必须多快开始响应，不是额外要执行 50 ns 的服务程序，不能加到 100 ns 上。

若题给设备数据率 R B/s、每次中断交换 q B、每次 CPU 开销 t 秒，在每次均交换 q B 且持续工作的模型中，请求率为 R/q，CPU 比例为 $(R/q)t$。若结果超过 100%，说明该模型下 CPU 无法及时服务；不能把它继续当作可达到的利用率。

7.6 DMA：数据不经过 CPU 寄存器逐次搬运

DMA 控制器在设备接口与主存之间组织直接传送。CPU 仍需通过驱动程序设置方向、主存起始地址、传送数量和设备等参数，启动设备；传送过程中由控制器形成主存地址、请求总线、搬运数据并更新计数；结束或出错后再通知 CPU 处理。DMA 减少逐个数据的指令开销，不表示 CPU 从头到尾完全不参与。

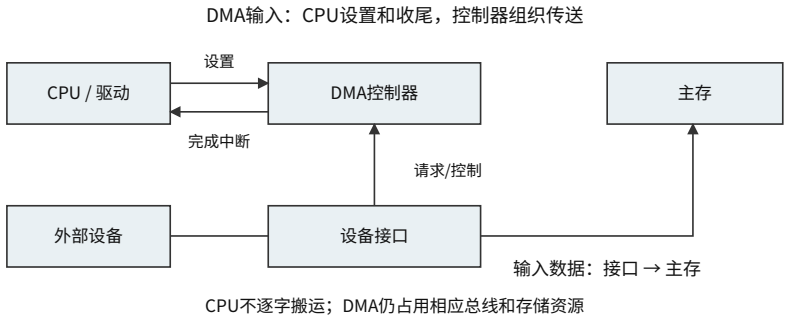


Figure 7.2: CPU 设置和收尾，DMA 传输阶段在接口与主存之间搬运

暂停 CPU 方式可在一块数据传送期间集中占用相关总线；周期挪用方式在设备准备好一个传送单位后申请一次或若干次总线使用；交替方式按约定把周期分给 CPU 和 DMA。CPU 能否同时访问某个存储体、Cache 或另一条总线，要由具体资源结构决定。只要 CPU 本拍不需要被占用的资源，它仍可能继续内部运算。

周期挪用中，设备准备数据后向 DMA 控制器请求服务，DMA 控制器再向系统申请总线。教材说 “DMA 请求” 时可能指设备侧请求，也可能指总线申请，解题要看请求的发送者和接收者，不能只认四个字母。

DMA 控制器通常包含主存地址计数器、传送长度计数器、数据缓冲寄存器、请求触发器、控制/状态逻辑以及完成中断机构。地址计数器决定下一次主存位置，长度计数器判断块是否完成；缓冲容量 q B 决定一次可交换的数据粒度，整块长度 Q B 决定 CPU 何时收到完成通知。两种 “计数” 不能交换使用。

先统一事件粒度，再列预算

设设备连续速率为 R B/s，接口每次可交换 q B，每次 CPU 服务耗时 t 秒，则理想事件率为 R/q 次/s，CPU 开销比例为 $(R/q)t$。若 DMA 每块 Q B 才完成一次且每块软件开销为 u 秒，则 CPU 开销比例为 $(R/Q)u$。另以总线每次占用 b 秒列总线占用 $(R/q)b$；这第三个式子不自动等于 CPU 开销。

课程连接·CPU 预算与总线预算分列

每块软件预处理/后处理影响 CPU 比例，DMA 实际访存影响总线和主存占用。CPU 可能在其私有 Cache 中继续运算，所以总线忙碌比例不能无条件等同 CPU 损失比例。

这些式子都先假定事件持续、粒度固定且无额外未计开销；若比值超过 1，说明所需服务能力超过预算。缓冲是否溢出还要核最长响应时间：平均 CPU 开销低于 100% 不是实时期限一定满足的充分条件。

7.6.1 例 7-5　周期挪用不等于整块期间封住主存

【配套例题】若设备采用周期挪用 DMA 方式进行输入和输出，每次 DMA 传送的数据块大小为 512 字节，相应的 I/O 接口中有一个 32 位数据缓冲寄存器。对于数据输入过程，下列叙述中，错误的是（　）。

A. 每准备好 32 位数据，DMA 控制器就发出一次总线请求
B. 相对于 CPU，DMA 控制器的总线使用权的优先级更高
C. 在整个数据块的传送过程中，CPU 不可以访问主存储器
D. 数据块传送结束时，会产生 “DMA 传送结束” 中断请求

解析。每次缓冲 32 位即 4 B，512 B 需要 128 次传送。周期挪用只在取得总线的时段占用资源，其余时段 CPU 仍可访问主存，不是整块传完前一直封锁，所以选 **C**。B 描述本题中 DMA 优先取得总线的模型，不表示任何系统的任意外部请求都永久压过 CPU。

一次总线占用时间为 b、每次搬 q B 时，DMA 数据率 R 对应的总线占用比例约为 $(R/q)b$，前提是这些占用不重叠且没有额外启动代价。CPU 实际损失还取决于它原本是否需要总线，因此不能把 “DMA 占总线 10%” 无条件改写成 “程序变慢 10%”。

7.6.2 例 7-6　DMA 的准备与完成仍需要软件

【配套例题】下列关于 DMA 方式的叙述中，正确的是（　）。

I. DMA 传送前由设备驱动程序设置传送参数
II. 数据传送前由 DMA 控制器请求总线使用权
III. 数据传送由 DMA 控制器直接控制总线完成
IV. DMA 传送结束后的处理由中断服务程序完成

A. 仅 I、II　B. 仅 I、III、IV　C. 仅 II、III、IV　D. I、II、III、IV

解析。I 是 CPU 上的驱动程序做传送前准备，II、III 是 DMA 硬件实际传输，IV 是完成通知后的软件收尾，四项均正确，选 **D**。这条链把 “CPU 不逐字搬运” 和 “CPU 完全不参与” 清楚区分开。

方式	CPU 介入时机与粒度	数据搬运主体	主要约束
忙等查询	持续检查，逐单位搬运	CPU 指令	等待和搬运均占 CPU 执行机会
定时查询	定期检查，每次查询及搬运	CPU 指令	查询间隔与服务时长须满足缓冲期限
中断 I/O	每请求被允许响应后，交换 q B	CPU 服务程序	请求率 × 每次响应处理时间
DMA	每块 Q B 前设置、结束后收尾	DMA 控制器	CPU 开销、总线与主存占用分别计算

适用性取决于设备速率、传送粒度、缓冲和系统实现。字符型低速设备通常适合查询或中断，网络和块设备的大批量传送常用 DMA；中断方式并非适用于所有设备，DMA 也没有固定的速度门槛。

比较三种方式时，可先固定同一设备数据率 R。若接口一次仅能容纳 q B，则数据积满时间为 q/R；定时查询的查询间隔要小于这个时间，中断方式每秒约响应 R/q 次。若 DMA 每块 Q B 才让 CPU 收尾一次，CPU 事件率变成 R/Q；但控制器可能仍按 q B 多次请求总线，不能把 “每块一次完成中断” 误作 “整块只有一次总线传送”。CPU 事件数、设备准备次数和总线事务数是三个不同计数。

最小反例·平均低占用也会超期限

每秒总共只需 1ms 服务，看似占用很低；若某次数据必须在 10 微秒内取走，而 CPU 最长关中断 100 微秒，仍可能丢数据。平均预算与最坏响应延迟必须分别核对。

7.6.3 例 7-7　同一接口缓冲下比较查询、中断和 DMA 预算

【配套例题】假定计算机的主频为 500 MHz，CPI 为 4。现有设备 A 和 B，其数据传输率分别为 2 MB/s 和 40 MB/s，对应 I/O 接口中各有一个 32 位数据缓冲寄存器。请回答下列问题，要求给出计算过程。

（1）若设备 A 采用定时查询 I/O 方式，每次输入/输出都至少执行 10 条指令。设备 A 最多间隔多长时间查询一次才能不丢失数据？CPU 用于设备 A 输入/输出的时间占 CPU 总时间的百分比至少是多少？

（2）在中断 I/O 方式下，若每次中断响应和中断处理的总时钟周期数至少为 400，则设备 B 能否采用中断 I/O 方式？为什么？

（3）若设备 B 采用 DMA 方式，每次 DMA 传送的数据块大小为 1000 B，CPU 用于 DMA 预处理和后处理的总时钟周期数为 500，则 CPU 用于设备 B 输入/输出的时间占 CPU 总时间的百分比最多是多少？

解析。数据率中的 M 按 10^6 计，缓冲 32 位 =4 B，CPU 每秒有 500×10^6 个时钟周期。先分别求设备填满缓冲的时间、CPU 每次服务时间，再求长期比例，不把设备工作时间直接当作 CPU 忙碌时间。

（1）A 每准备 4 B 需要

$$\Delta t_A=\frac{4}{2\times10^6}=2\ \mu\text{s}.$$

因此最多每 2 μs 就必须完成一次查询与取数；间隔更长，单个缓冲可能被下一组数据覆盖。每秒至少服务 $2\times10^6/4=5\times10^5$ 次；每次至少 10×4=40 周期，故 CPU 占用比例至少为

$$\frac{(5\times10^5)\times40}{500\times10^6}=4\%.$$

每次这项服务最少耗时 80 ns，2 μs 是设备产生一缓冲数据的间隔，不是 CPU 每次都要执行 2 μs。

（2）B 每 4 B 只允许 $4/(40\times10^6)=0.1\ \mu$s，相当于 50 个 CPU 周期；一次中断响应与处理却至少需 400 周期 =0.8 μs。每次服务还未完成，后续数据已连续到来，不能持续及时处理。按请求率核对，所需 CPU 比例为

$$\frac{(40\times10^6/4)\times400}{500\times10^6}=8=800\%.$$

这表示所需服务能力超过 CPU 预算，不能解释成实际 CPU 利用率能达到 800%。给定单缓冲与逐次中断模型下，设备 B 不适合这种中断 I/O 方式。

（3）DMA 把 CPU 事件粒度改成 1000 B 一块。以设备满速计，每秒最多 $40\times10^6/1000=40000$ 块；每块 CPU 预处理和后处理 500 周期，

所以 CPU 开销比例最多为

$$\frac{40000 \times 500}{500 \times 10^{6}} = 4\%.$$

课程连接·准备数据和传送数据

设备准备一批数据的时间与总线传送该批数据的时间各自计算；是否能重叠取决于设备与接口机制。

一块设备传送时间为 25 μs，CPU 设置收尾共 1 μs，二者相除也得 4%。这一项没有再乘 CPI，因为题干已直接给时钟周期数。第（1）问的 4% 是由 “至少 10 条指令” 得到下界，本问的 4% 按最大设备速率得到上界；数值相同，含义不同。

DMA 的 4% 统计 CPU 设置收尾，并未把 DMA 占用总线的时间当作 CPU 执行时间。若另问整机吞吐是否可达 40 MB/s，还须核对总线与主存带宽；若另问 CPU 程序因此慢多少，还要看 CPU 同时是否需要这些资源。第 6 章的总线预算与本题 CPU 预算应分别算，再按共享资源关系连接。

7.6.4 配套辨析巩固

1. 【配套例题】下列关于中断 I/O 方式的叙述中，错误的是（ ）。A. 中断屏蔽字用于确定中断响应的优先级；B. 保存断点和程序状态字在中断响应阶段完成；C. 保存通用寄存器和设置新中断屏蔽字由软件实现；D. 单重中断方式下中断处理时 CPU 处于关中断状态。
2. 【配套例题】DMA 控制 I/O 方式下，设备的输入/输出由 DMA 控制器控制完成，此时 DMA 控制器控制的数据传输通路位于（ ）。A. CPU 和主存之间；B. CPU 和 DMA 控制器之间；C. 设备接口和主存之间；D. 设备接口和 DMA 控制器之间。
3. 原创补充：设备数据率为 1 MB/s，每次中断交换 4 B，开销为 2 μs，CPU 占用多少？若改为每 1024 B 一块 DMA，假设 CPU 每块仅需 20 μs 设置及收尾，CPU 开销比例是多少？暂不计总线竞争。

答案。 1. A。按本章教材口径，屏蔽字规定允许的抢占关系和处理优先级，固定的响应仲裁次序不能由它直接改写。2. C。3. 中断为 $(10^{6}/4) \times 2 \times 10^{-6} = 50\%$；DMA 为 $(10^{6}/1024) \times 20 \times 10^{-6} \approx 1.953\%$。后者只是 CPU 设置收尾开销，尚不包含 DMA 占用主存总线对程序造成的影响。

7.7 外部设备怎样形成 I/O 请求

输入设备把外界信息转换为数字编码，例如键盘扫描按键并形成按键编码，鼠标把位移和按键状态转换为报告，扫描仪和摄像设备把光学信息采样编码。输出设备把编码转换为显示、声音或纸面记录；显示帧缓冲保存像素数据，显示控制器按时序读取并驱动显示。

若分辨率为 $W \times H$、每像素 b 位、刷新率 f 帧/s，仅有效像素的单帧容量为 $WHb/8$ B，最低像素载荷数据率为 $WHbf/8$ B/s。实际接口还可能包含消隐、包头、编码和压缩等因素，题目给出额外条件时需分别计入。

针式打印利用撞针、色带形成字符，适合多联复写；喷墨通过喷出墨滴成像，彩色及介质要求依实现而定；激光打印利用感光鼓成像、转印和定影。外设的机械或介质工作速度通常与 CPU 不同，接口中的数据缓冲和状态控制用于协调这种差异。

7.8 I/O 软件、接口与端口的分工

课程连接·端口与接口

端口是 CPU 访问接口寄存器的地址入口，接口提供设备状态、控制和缓冲等功能。

I/O 软件包括驱动程序及上层管理程序，I/O 硬件包括设备、接口及连接通路。接口承担设备选择、命令译码、数据缓冲、状态记录、格式和速度匹配，并可发出中断或 DMA 请求。端口是 CPU 可读写的接口寄存器或相应地址位置，接口通常含多个端口。

数据端口保存待传或已接收数据，状态端口报告就绪、忙、错误等状态，控制端口接收命令或工作方式。控制命令字和状态字都可能经数据总线传送；控制总线上的读写信号用于说明这次总线事务的性质，两种 “控制信息” 处于不同层次。

统一编址把 I/O 寄存器映射到与主存同一地址空间，用普通访存指令访问；独立编址设置单独 I/O 地址空间并用专门 I/O 指令访问。统一编址需要通过地址译码区分内存和设备区域；独立编址需要区分存储器事务和 I/O 事务的控制规则。

通道是一种可执行通道程序的专用 I/O 处理部件。CPU 发出启动命令，通道按通道指令组织一系列 I/O 操作；DMA 控制器主要负责设定数据块的传送，通道可承担更复杂的控制组织。它们都能减少 CPU 逐字搬运的工作量，实际能力取决于具体系统。

7.9 查询方式的可行性先于占用率

独占查询中，CPU 不断检查状态并等待，外设准备期间也不能转做普通程序；定时查询在查询间隔可执行其他程序，但必须保证缓冲区在下次查询前不会溢出。若设备每 t_d 产生一项、缓冲只能容纳一项，则查询间隔应满足及时取走的要求；存在多项缓冲时还要考虑连续到达及服务时间。

设 CPU 主频 50 MHz，一次查询需 100 周期。鼠标每秒查询 30 次，占用率为 $30 \times 100/(50 \times 10^6) = 0.006\%$。若磁盘每秒输入 2 MiB、每次查询传 4 B，则每秒需 2^{19} 次查询，占用周期 100×2^{19}，占比约 104.86%。这说明该查询组织即使耗尽 CPU 时间也来不及处理，不能把超过 100% 的值解释为实际可达到的 CPU 占用状态。

7.10 中断判优、屏蔽和现场恢复

中断请求标记用于记住待处理事件；判优电路在多个请求同时存在时选择响应对象。响应优先级是硬件在当前时刻选谁；处理优先级决定哪个服务程序允许被哪个中断打断，可通过屏蔽字组织。二者可以不同。

课件约定屏蔽位 1 表示屏蔽、0 表示允许；必须在例题中先写清位序和电平语义。某服务程序运行时通常屏蔽本级及不允许打断它的源，只允许规定的更高处理优先级进入；进入服务程序时保存旧屏蔽字，返回前恢复。屏蔽字改变的是请求可见性，不会抹掉已经发生的请求事件。

多重中断的一般顺序为：硬件关中断、保存断点并进入服务入口；软件保护现场和旧屏蔽字，装入新屏蔽字；在安全位置开中断，运行服务主体；结束时关中断，恢复屏蔽字和现场，再按返回规则恢复执行。若过早开放中断，尚未保存的状态可能被下一层覆盖。

非屏蔽中断与可屏蔽中断的区别在于是否受普通中断允许控制约束。非屏蔽不表示任何时刻、任何指令内部都立即任意打断；响应时点和保存规则由体系结构规定。

7.10.1 例：设备准备与服务程序怎样重叠

课程连接·DMA 仍需控制信息

DMA 控制器需知道内存地址、传送计数、方向和设备状态；数据传送期间 CPU 与 DMA 如何争用总线须依题设。

CPU 主频 50 MHz，CPI 为 4；设备每 0.5 ms 准备一个字符，每字符含 1 位起始、7 位数据、1 位奇校验及 1 位停止，共 10 位。设备持续工作时最多每秒形成 2000 字符。

中断响应需 10 周期，服务程序 20 条指令，第 15 条启动设备准备下一字符。按课件的稳定重复近似，每字符完成间隔为 $25000 + 10 + 15 \times 4 = 25070$ 周期，因为最后 5 条服务指令与下一字符准备重叠。1000 字符总历时约 25070000 周期；CPU 实际用于响应和服务的时间为 $1000(10 + 20 \times 4) = 90000$ 周期。

若从第一次启动设备准备第 1 个字符开始计时，到第 1000 次中断服务程序完全结束，且各次均采用上述理想时序，则最后 5 条服务指令没有下一字符的准备过程可供重叠，精确总历时为 $1000 \times 25070 + 5 \times 4 = 25070020$ 周期。这里 “约” 表示采用课件的稳定重复周期近似；题目改变起止事件时，应按新端点补计启动或排空时间。

7.11 DMA 控制器的寄存器和传送阶段

DMA 控制器通常包含主存地址寄存器、传送字数计数器、数据缓冲寄存器、控制与状态逻辑以及设备相关地址信息。CPU 预处理设置方向、主存起始地址、长度和设备参数并启动设备；数据准备好后，DMA 申请总线，在授权后直接在设备和主存之间搬运，自动更新地址和计数；完成或出错后通知 CPU 后处理。

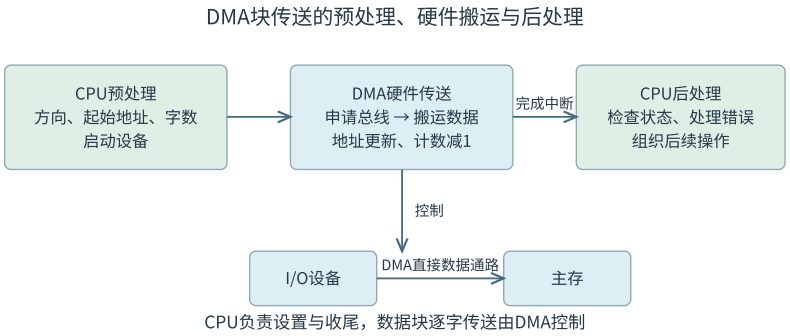


Figure 7.3: CPU 设置传送参数，DMA 逐次获得总线传数据，块结束后请求 CPU 处理

停止 CPU 访存的成组传送在一段时间内集中占用总线，控制简单但 CPU 访存受阻；周期挪用在需要时借用一个或若干存储周期，CPU 正在访存时通常先让该次访问完成；交替访存按固定时隙分配 CPU 和 DMA，减少重复仲裁但增加时序组织要求。

DMA 请求是获取总线或主存访问机会，中断请求是要求 CPU 改变执行流去运行服务程序。DMA 传数据不逐字执行 CPU 搬运指令，块完成通知仍可采用中断。DMA 占用总线时，CPU 若能在寄存器或 Cache 中继续执行某些操作，并不一定完全停止；是否发生停顿须看具体访问和总线关系。

7.11.1 计算与辨析自查

1. 写控制端口时，命令字为什么仍可走数据总线？ 2. 查询占用率算出 105% 意味着什么？ 3. 中断响应优先级和处理优先级分别决定什么？ 4. DMA 块传送前后 CPU 还要做什么？

答案。 1. 命令字是本次事务传送的数据，读写性质由控制线及地址共同说明。2. 该方案处理能力不足。3. 前者选择同时请求时先响应谁，后者限定服务程序间的嵌套关系。4. 预处理设置参数并启动；后处理检查结果、处理错误和唤醒后续工作。

课程连接·先列时间线

分别标设备准备、CPU 服务、总线传送的开始与完成时间，再判断是否可以重叠。

7.12 主动回忆自测

以下为自拟检查。请沿 “设备—接口—控制方式—预算” 回忆，不先看答案。

1. I/O 端口、接口、设备和磁盘扇区地址之间分别是什么关系?
2. 统一编址与独立编址，改变的是地址空间与指令中的哪两项约定?
3. 为什么定时查询允许 CPU 在两次查询间执行其他程序，而忙等查询会持续占用 CPU?
4. 硬件中断响应与软件服务程序分别保存哪些类型的信息？是否所有 ISA 完全相同?
5. 已知响应优先级 A>B>C，能否只据此确定三者最终完成顺序?
6. DMA 中，块长 1024 B、接口缓冲 4 B，块内可能有多少次数据单位传送、多少次块完成通知?
7. 显存容量和刷新数据率分别多乘了哪一个参数？双缓冲为何不能直接让输出帧频翻倍?
8. 设备每秒 1 MB、每次中断 4 B、每次开销 2 微秒，改为每 1024 B 一块、每块软件开销 20 微秒的 DMA，CPU 预算分别是多少?

7.12.1 自测答案

1. 端口是 CPU 可访问的接口位置；接口连接总线与设备；扇区地址标识设备内部数据位置，不能与 CPU 端口地址混同。2. 统一编址共享存储器地址空间并用访存指令；独立编址有独立端口空间，通常用专门 I/O 指令。3. 前者在期限允许下间歇检查，后者循环执行检查；均仍需安排实际搬运。4. 典型硬件保存断点与规定状态，软件保存其需要恢复的通用寄存器等；具体保存集合和开中断/返回时序依 ISA。5. 不能，还需屏蔽规则、允许嵌套关系、请求到达时间与服务时长。6. 可有 256 次 4 B 单位传送，正常块完成通常一次通知；总线突发合并会改变事务数，须看具体控制器。7. 有效容量为像素数 $\times$ 位数，刷新有效速率再乘帧频；双缓冲多存一帧，不自动多输出一帧。8. 中断 50%；DMA 约 1.953%，均仅按题定 CPU 执行开销，另核总线与实时期限。